Exact Derandomization of Logarithmic Space: $\mathsf{L} = \mathsf{RL} = \mathsf{BPL}$

OpenAI

September 23, 2026

Abstract

For every fixed randomized machine that runs in polynomial worst-case time and logarithmic space, we give a deterministic algorithm that approximates its acceptance probability to any fixed inverse-polynomial accuracy in logarithmic space and polynomial time. This proves $\mathsf{L} = \mathsf{RL} = \mathsf{BPL}$. For length-$n$ inputs and unary $q \geq 1$, the acceptance probability is approximated to error 2^{-q} in deterministic space $O(\log(n+2)+q)$ and time $(n+2)^{O(1)}2^{O(q)}$. For fixed inverse-polynomial accuracy, the construction starts with a coordinate of $(I-S)^{-1}e$, where S is the substochastic matrix on time-labelled configurations with transitions only to later times and e is the final-time accepting indicator. A hierarchy of corrections and vertex copies transfers transition mass into accumulated rewards. These rewards are estimated recursively in a shared random environment of $O(\log(n+2))$ bits that specifies sampled vertex sets and endpoint fingerprints. Conditional averaging uses walks with a uniform spectral gap to suppress the noise of estimates that share one environment across recursive calls. Compressed row lists, adaptive fingerprints and a telescoping $O(\log(n+2))$ bound on suspended records control the storage those calls retain. Among valid environment encodings whose sampled set contains the designated start vertex, more than three quarters give accurate estimates. Every environment is evaluated by a halting logarithmic-space computation, so the algorithm enumerates polynomially many estimates over these encodings and outputs their median.

Contents

1 Introduction **3**

2 The transition model and resource parameters **8**

3 Reduction to fixed-precision approximation **10**

4 Sampling matrices for retained vertex sets **12**

5 The exact correction and copy hierarchy **14**

6 Random sparse tables and their interfaces **24**

7 Conditional averaging in the shared environment **34**

8 Additive estimation and conditional prefix bounds **38**

9 Compression of additive row estimates **51**

10 Adaptive fingerprints within budget blocks **61**

11 Finite numerical queries with bounded suspended storage **72**

12 Uniform controller implementation **84**

13 Fixed-precision approximation and accepting computations **104**

14 The effective compiler **109**

References **116**

1 Introduction

A logarithmic-space machine can remember a bounded number of positions in its input, but cannot keep a polynomial-length computation or random tape. Can fresh random bits increase what such a machine can decide? We answer this question by giving a deterministic logarithmic-space simulation of every polynomial-time randomized logarithmic-space computation with bounded error.

We use machines with finite control, an endmarked read-only input, finitely many work tapes, and fresh independent fair bits. Input heads are confined between the endmarkers. All randomized running-time bounds hold on every random tape. Space counts traversed writable cells in bits, including every simultaneously live counter, numerical register and suspended call; visiting a blank work cell counts toward space. A transducer has a one-way write-only output tape. Throughout, n is the input length and logarithms have base two.

The class L consists of languages decidable by deterministic machines in $O(\log(n+2))$ work space. Such a halting deterministic machine has only polynomially many configurations and hence runs in polynomial time. The class RL permits polynomial-time randomized machines in this space, with acceptance probability zero on no inputs and at least $1/2$ on yes inputs. For BPL, the corresponding probabilities are at most $1/3$ and at least $2/3$.

Theorem 1.1 (Exact logarithmic-space derandomization)**.**

$$\mathsf{L} = \mathsf{RL} = \mathsf{BPL}.$$

The proof approximates arbitrary acceptance probabilities. This gives the two-sided statement directly, and also applies to computations whose acceptance probabilities have no decision gap. Its quantitative form separates the memory needed for the input from the memory needed for the requested accuracy.

Theorem 1.2 (Acceptance-probability approximation)**.** *Fix a randomized machine $\mathcal{M}$ with polynomial worst-case running time and $O(\log(n+2))$ work space. Let $p_{\mathcal{M}}(x)$ be its acceptance probability on an input x of length n. There is a uniform deterministic algorithm which, on input $(x, 1^q)$ with $q \geq 1$, outputs an integer $0 \leq a \leq 2^{q+2}$ satisfying*

$$\left| a2^{-(q+2)} - p_{\mathcal{M}}(x) \right| \leq 2^{-q}.$$

It uses $O(\log(n+2)+q)$ work space and $(n+2)^{O(1)}2^{O(q)}$ time. The constants depend only on $\mathcal{M}$.

In particular, every fixed inverse-polynomial accuracy is attained in logarithmic space and polynomial time. The precision parameter is unary; the running-time bound is exponential in general q. We first prove the fixed inverse-polynomial result and then obtain the single variable-precision algorithm by padding. Section 3 also derives total deterministic separators for bounded-error promise problems. Section 13 constructs accepting runs when the acceptance probability has a specified inverse-polynomial lower bound.

The simulation is also effective at the level of machine descriptions. Given a source program and supplied polynomial-time and logarithmic-space bounds, one terminating compiler produces a deterministic decider and explicit numerical bounds for its space and time (Theorem 14.1). The supplied bounds are hypotheses on the source. Section 14 constructs this compiler by specializing one fixed deterministic library.

1.1 Historical context

The complexity of probabilistic Turing machines was developed by Gill [18], including measures of both time and tape usage. Random walks made the logarithmic-space question concrete: Aleliunas, Karp, Lipton, Lovász and Rackoff obtained polynomial-time randomized algorithms for undirected connectivity and explicitly asked whether polynomial-time randomized logarithmic space could be simulated in deterministic logarithmic space [3]. Early work also studied probabilistic machines without a polynomial-time restriction. Borodin, Cook and Pippenger obtained a quadratic-space deterministic simulation in this broader setting [7]. The distinction between a space bound and simultaneous time and space bounds is essential throughout this history.

Pseudorandom generators attack the problem by replacing a long sequence of fresh random bits with the output generated from a short seed. Ajtai, Komlós and Szemerédi gave early logarithmic-space constructions that hit small-space tests using short random strings [2]. Babai, Nisan and Szegedy subsequently obtained pseudorandom generators through multiparty communication bounds [6]. Nisan's generator uses $O(S \log R)$ seed bits for a space-S computation reading R random bits [21]. For logarithmic space and polynomially many bits this gives $O(\log^2 n)$ seed length; direct seed enumeration has a quasipolynomial time bound. Nisan's later simultaneous simulation achieves polynomial time and $O(\log^2 n)$ space, for two-sided as well as one-sided bounded error [22]. For $S(n) \geq \log n$, Nisan and Zuckerman showed that a space-S computation using $\mathrm{poly}(S)$ random bits can be simulated using only $O(S)$ random bits and $O(S)$ space [24]. The restriction on the original number of random bits is substantive; for logarithmic space it permits polylogarithmically many bits.

Saks and Zhou reduced the deterministic space needed for general bounded-error logarithmic-space computation to $O(\log^{3/2} n)$ [31]. Cai, Chakaravarthy and van Melkebeek established a simultaneous tradeoff: for each fixed rational $0 \leq \alpha \leq 1/2$, the simulation uses time $n^{O(\log^{1/2-\alpha} n)}$ and space $O(\log^{3/2+\alpha} n)$ [10]. The polynomial-time endpoint has squared-logarithmic space. Hoza subsequently improved the general space bound to $O(\log^{3/2} n/\sqrt{\log \log n})$ [19, Theorem 1.6 and Corollary A.5]; like the bound of Saks and Zhou, this space containment asserts no polynomial time bound. The same paper also constructs an improved weighted pseudorandom generator, in which signed or real weights replace an ordinary probability distribution; the space bound above does not need this new generator [19, Section 1.5].

Acceptance-probability estimation also connects derandomization to stochastic matrix products and inverses. Ahmadinejad, Kelner, Murtagh, Peebles, Sidford and Vadhan developed high-precision small-space algorithms for random-walk probabilities, with stronger bounds for Eulerian graphs [1]. Cohen, Doron, Sberlo and Ta-Shma later used varying precision across recursion to improve the space complexity of long stochastic matrix products [16]. The construction below works with a finite inverse on a time-layered configuration graph; its correction and copy hierarchy controls the information retained while individual entries are evaluated.

Structural restrictions on a computation provide another route. For undirected connectivity, Nisan, Szemerédi and Wigderson obtained deterministic space $O(\log^{3/2} n)$ [23], subsequently improved to $O(\log^{4/3} n)$ by Armoni, Ta-Shma, Wigderson and Zhou [4]. Reingold's logarithmic-space algorithm settles undirected connectivity [27, 28]. Reingold, Trevisan and Vadhan studied regular directed graphs and gave logarithmic-space path search for Eulerian directed graphs [29]. Their oblivious walk constructions impose additional labeling conditions. General directed configuration graphs need not have these properties, so those results did not settle the unrestricted randomized computation problem.

The relation between one-sided and two-sided derandomization also requires care. A read-once branching program is a layered deterministic computation that reads the input symbols once in

a fixed order (bits in the binary case). Its length is the number of steps, and its width is the largest number of states in a layer. A 1/2-hitting set for a class of programs contains an accepting input for every member that accepts at least half of all input strings. Cheng and Hoza proved that one uniformly logarithmic-space-enumerable family of 1/2-hitting sets $H_n \subseteq \{0,1\}^n$ for all width-n, length-n read-once branching programs over the binary alphabet would yield $\mathsf{L} = \mathsf{BPL}$ [13, Theorem 1.3]. The premise is a hitting-set construction; it is not merely the language-class equality $\mathsf{RL} = \mathsf{L}$.

Recent results refine both general and structured models. Chen, Cohen, Doron, Khaskelberg and Ta-Shma improve error reduction for weighted generators; in the polynomial-width regime their construction has seed length $\log|\Sigma| + O(\log^2 T + \log(1/\delta))$, where T is branching-program length, Σ its alphabet and δ the error [12]. Cheng and Wu obtain simultaneous polynomial-time and polylogarithmic-space simulations for regular branching programs, with extensions to polynomial-time randomized logarithmic-space machines whose two-way read-only random tape visits each cell at most constantly often, and to such machines with read-once randomness and an auxiliary stack whose push, pop and idle schedule is independent of the random bits. Consecutive stays on a random-tape cell count as one visit [14, Theorems 5–6]. Hardness-versus-randomness approaches additionally give conditional, certified and per-instance alternatives; Pyne and Tell survey this line of work [26].

Effectivity has its own substantial antecedents. Pyne, Raz and Zhan construct a universal deterministic estimator whose $O(S(n))$ space guarantee follows from $\mathsf{prBPL} \subseteq \mathsf{SPACE}[O(S(n))]$, for space-constructible $S(n) \geq \log n$ [25, full version, Theorem 5.1]. Here prBPL denotes promise problems solvable in polynomial time and randomized logarithmic space with bounded error. The compiler below uses a fixed separator established in this paper and supplies explicit source-specific numerical resource bounds. Its construction should be read in this context of universal simulation, with the source bounds supplied as part of the compiler input.

1.2 Proof strategy

The core result is fixed inverse-polynomial approximation (Theorem 3.1). Fix the source machine and the accuracy exponent, and put $B = \Theta(\log(n+2))$. We construct a finite list of estimates indexed by $O(B)$-bit descriptions. More than three quarters are accurate, and every estimate can be evaluated by a halting deterministic computation in $O(B)$ space. Their median gives the desired approximation. The descriptions encode the possible random *environments*, of which there are at most $2^{O(B)}$. An environment consists of a fixed number of *sampling matrices* (Section 4): random matrices over a prime field of size $2^{O(B)}$, conditioned on their positive-degree coefficient columns being linearly independent. Applied to the vector of powers of a vertex identifier, each matrix decides whether that vertex is *retained* at a prescribed sampling rate. A vertex is retained with probability comparable to the rate, and, conditioned on the retention of one identifier, a distinct identifier is retained with probability at most a constant times its rate (Lemma 4.1). The environment also contains $O(B)$ independent fair bits, used as fingerprint seeds (Section 10). The median list uses only the encodings that retain the designated start of the final reward query; it is under this conditional law that the stated majority is accurate (Section 13). Each estimator task carries an integer *statistical budget* M. Its error bound in the metrics of Section 6 is $C_b 2^{-p}$ for a fixed constant C_b, with precision p growing linearly in M (Equation (8.10)). Every other estimator task it queries has a smaller budget. The top-level task has budget $M_{\mathrm{top}} = O(B)$. The proof must control both estimation error in a shared random environment and the storage retained across recursive calls; enumeration requires termination even on inaccurate choices.

Two resource costs in the preceding literature help locate this approach. Direct enumeration

of Nisan's generator uses an $O(\log^2 n)$-bit seed [21]. Saks and Zhou's matrix-powering algorithm, specialized to logarithmic space, has $O(\sqrt{\log n})$ recursive levels using $O(\log n)$ space each [31, Section 1, pp. 377–379]. Here we enumerate an $O(\log n)$-bit environment for an input-specific probability estimator. Reusing that environment makes the recursive errors dependent, while keeping a full endpoint or numerical word at each level would exceed the desired space bound. The construction below addresses these costs together: conditional averaging and the block comparison control the shared errors; compression, outgoing and incoming port labels and allowance differences control the retained storage. This is a direct estimation argument.

Transferring transitions into rewards. Section 2 represents the computation by a substochastic matrix S on time-labelled configurations V_0, with final accepting indicator e. Every transition advances time, so $p_0 = (I - S)^{-1}e$ is a finite sum and records acceptance probabilities at all configurations. Section 5 successively replaces transitions by accumulated rewards. For a current transition matrix C, the identity

$$D = C - E + EC, \qquad I - D = (I + E)(I - C)$$

allows a nonnegative correction E to update a reward W to $(I+E)W$ while preserving the represented probability vector. The aim is to make the remaining transition contribution small.

Correction and copying serve complementary purposes. A finite detector calculation suppresses correction in columns with many substantial entries. Where fully enabled, correction makes entries small. The remaining large entries are divided among copies of their destinations. The detector bounds how many vertices need active extra copies, so each stage reduces the entry bound by 2^H while at most doubling the number of vertices incident to positive entries, for fixed $H \geq 4$ (Proposition 5.5). Correction itself needs only constantly many iterations: at its comparison fixed point, a positive lower bound on the residual at corrected entries bounds their number in each row. Strictly increasing paths through that support give the factorial truncation bound of Lemma 5.1. The remaining transition mass in each row is at most the entry bound times the number of active vertices. Transporting rewards through the copy projections therefore yields accumulated rewards W_l with

$$0 \leq p_0(x_0) - W_l(z_l) \leq |V_0|\, 2^{-(H-1)l},$$

where x_0 is the start and z_l its copy obtained by choosing the designated copy labelled zero at each stage. Thus $L = O(B)$ stages suffice; the full stage domains still have polynomial size. This is the target proved in Proposition 5.7.

Estimating in one shared environment. The hierarchy's exact matrices and conditional means define targets for recursive queries. Section 6 represents their contributions by random tables on the retained vertex sets of Section 4. Cutoffs suppress crowded sampled rows and, for corrections, crowded columns. On the support of a true contribution, the conditional mean of its cutoffs is bounded away from zero, so division by that mean recovers the contribution in expectation. Clipped denominators keep values bounded, and the bounds on row support and incoming degree hold even when estimated inputs are inaccurate. Substitution of earlier estimates introduces bias, which Section 8 bounds.

The incoming bound connects the error and space arguments. After a correction move, the algorithm can discard the old source and query a right factor at the intermediate vertex, retaining only a bounded inverse port for the return. In the error analysis, the same incoming bound lets us sum out the old source, at a cost given by the ratio of sampling rates. Lemma 8.3 gives this cost, and the precision assignment in Section 8 pays it.

To approximate the conditional means, Section 7 constructs walks preserving the specified source and endpoint retention tests. It derives a uniform spectral gap for the required finite affine actions from Shalom's property (T) results [33, Theorems 2.1 and 2.6]. The resulting conditional averaging contracts eighth-moment noise exponentially in walk length. Its steps can be described independently of the conditioning vertices, with validity checked when those vertices are visited (Lemma 7.3); this permits their use with one current vertex address.

Differences and row compression. At parent budget M, the additive schedule in Section 8 averages differences between child estimates at budgets m and $m-1$. Both approximate the same sample term formed from the true tables, so their difference has small absolute norm. Their conditional means telescope, and a walk whose length is proportional to the slack $M-m$ reduces the remaining noise. Contributions at the same endpoint are combined before taking the difference. The same slack bounds the immediate path description, linking the averaging available to the storage it requires.

Section 9 groups comparable slacks and represents each row by a finite list of exceptions to one common default. Small entries are replaced by that default. At the first overflow, where capacity forces the threshold above its prescribed floor, either many retained entries are truly large, an event rare even conditional on a specified endpoint, or the prefix errors account for the excess. The analysis uses this rarity for the deterministic conditional-mean error and the absolute moment bound for the noise, which may depend on the overflow event. Stability of the later updates then controls the bounded row functions needed by the hierarchy (Proposition 9.5).

Fingerprints for adaptive endpoints. Even these lists cannot be evaluated by saving a full endpoint at every recursive level. Section 10 uses short fingerprints at small slacks, combining polynomial evaluation and affine hashing [32, 11]. With top budget $O(B)$, sharing one $O(\log B)$-bit seed over each block of $\Theta(\log B)$ budgets uses $O(B)$ bits. A fixed-pair collision bound is insufficient because earlier comparisons affect later candidates.

The analysis therefore replaces fingerprint comparisons by exact equality throughout the current block, retaining the actual procedures below it. This comparison calculation supplies a small set of candidate endpoints independent of the current fingerprint. An actual path escaping that set contains an edge where a smooth support function is one and its counterpart in the comparison calculation is zero. The lower-call error bound pays for such escapes; inside the set the ordinary collision bound applies. Only new increments are combined by fingerprint: old values stay at their own endpoints. Thus the collision cost is proportional to the small consecutive-budget differences, rather than to accumulated values. Together with the compression estimate, this coupled induction proves the moment bound for the actual estimator in Theorem 10.2.

Total evaluation in logarithmic space. Sections 11 and 12 implement these estimates using one vertex cursor and numerical queries returning one bounded digit at a time. Lists and ports are fixed independently of the requested digit, so greater numerical precision approximates the same estimator value. Incoming accesses use the forest traversal of Cook and McKenzie [17, Proposition 1], also used in the reversible simulation of Lange, McKenzie and Tapp [20]. Here the traversed tree is the component of a terminal program's success state; its canonical initial states enumerate the forward ports ending there. Returning to that success state recovers the endpoint without a saved address (Section 12.3).

Even a short fingerprint index would cost too much if saved at every call with small budget slack. Lemma 12.9 compares two fingerprints by toggling and reading a shared B-bit vector on

repeated endpoint visits. The initial contents cancel and are restored. This adds only constant local control beyond the endpoint descriptions, and nested comparisons reuse the same vector. This vector is counted in the work space.

The remaining records are charged to differences between caller and child resources. If their recursion allowances are w, w' and their digit places are ν, j, the suspended caller uses at most

$$A_{\text{space}}(w - w') + D_{\text{space}}(\nu - j)$$

bits for fixed $A_{\text{space}}, D_{\text{space}}$, including local copies used by a traversal. The top allowance and digit place are $O(B)$. Allowance strictly decreases; digit place can increase only in proportion to that decrease. These differences telescope along the active call chain, while the environment, cursor, shared bit vector and scratch occur once. Theorem 12.1 proves the resulting $O(B)$ bound and termination for every environment.

The median and the other results. The moment bound is integrated over vertices. Conditioning on retention of the designated start loses at most a constant times the stage-size bound $N = 2^{C_N B}$, for fixed C_N. Increasing the statistical budget by a fixed multiple of B pays this loss. Section 13 combines this with the hierarchy and digit errors to obtain the promised majority of accurate dyadic estimates. It enumerates the original environment encodings with their multiplicities, preserving exactly the conditional law. Repeated enumeration finds the median with only logarithmic counting and numerical registers. There are polynomially many encodings, and configuration counting bounds every total logarithmic-space digit query by polynomial time. This proves Theorem 3.1; constant accuracy and threshold 1/2 give Theorem 1.1 and total promise separators in Section 3.

For the unary precision request in Theorem 1.2, increasing digit accuracy alone leaves the statistical error unchanged. Section 3 instead fixes an inverse-linear approximator for a machine that ignores padding. A virtual padded input long enough for that error to meet the request preserves the original acceptance probability. Generating each requested input bit from its address gives the stated $O(\log(n + 2) + q)$ space and $(n + 2)^{O(1)}2^{O(q)}$ time. For accepting computations, Corollary 13.2 repeatedly chooses the successor with largest estimated continuation probability. The total estimation loss is smaller than the promised lower bound on acceptance, forcing an accepting terminal state. A verification pass followed by replay ensures that only accepting computations are output, including outside that promise.

Finally, the compiler in Section 14 fixes one universal randomized interpreter that enforces resource caps on every input. Probability approximation supplies a total deterministic separator for this interpreter even though arbitrary encoded inputs need not satisfy a bounded-error promise. The supplied source bounds determine padding that makes the caps harmless and preserves acceptance probability exactly. Specializing the fixed separator's virtual input operations produces the required machine. Its library and input routines give explicit space bounds; counting complete configurations of that same machine gives explicit time bounds (Section 14.5).

2 The transition model and resource parameters

We first express acceptance probability as a finite matrix inverse. Time labels make the transition matrix nilpotent; keeping the complete configuration at each vertex gives local access in both directions. The construction will use the resulting probability vector at every configuration, as well as its value at the designated start.

A matrix indexed by a finite set V has *transitions only to later times* if there is an integer time attached to every vertex and every nonzero entry goes to a strictly later time. It is *substochastic*

if its entries are nonnegative and each row sums to at most one. An edge is specified by a vertex and a port from a fixed finite set. A *rotation map* returns the other endpoint together with its inverse port; applying the inverse operation restores the first endpoint. Absent ports are explicitly recognized.

Lemma 2.1 (Configuration reduction)**.** *Fix a randomized machine with polynomial worst-case running time and $O(\log(n+2))$ work space. There is a uniform logarithmic-space construction which, on an input of length n, specifies a polynomial-size vertex set V_0, a substochastic matrix S on V_0 with transitions only to later times, a vector $e \in \{0,1\}^{V_0}$, and a start vertex $x_0 \in V_0$ such that*

$$p_0 = (I - S)^{-1} e \in [0,1]^{V_0}, \qquad p_0(x_0) = \Pr[\textit{the machine accepts}].$$

The incoming and outgoing port degrees of S are bounded by constants depending only on the machine. Port validity, edge weights, and the rotation map are computable in logarithmic space. For an RL *computation, $p_0(x_0) = 0$ or $p_0(x_0) \geq 1/2$.*

Proof. Let $T(n)$ be a fixed polynomial upper bound on the running time. A vertex records a time in $\{0, \ldots, T(n)\}$, the finite control, all input and work head positions, and the contents of a sufficiently large box of $O(\log(n+2))$ work cells. Take elementary machine steps with fixed dyadic coin probabilities. A transition advances the time by one. Invalid configurations or moves leaving the box have no corresponding transition. The box is chosen large enough that this never affects a computation from the specified start.

Each elementary step has constantly many successor choices. To find predecessors, guess the old finite control, locally overwritten symbols, head movements, and coin choice, then check that the forward step gives the specified configuration. There are constantly many such guesses. Combine transitions with the same ordered pair of endpoints by adding their probabilities. Order the distinct successors and predecessors by their configuration encodings; their positions give outgoing and incoming port labels for the resulting matrix entries. These lists have constant length, so duplicate removal, summed weights, and the inverse port are computed in logarithmic space by local enumeration. This base computation makes no recursive graph queries. All records use $O(\log(n+2))$ bits, and the read-only input can be consulted during these checks.

All input-head coordinates in the vertex universe are restricted to a fixed polynomial interval containing the positions reachable within $T(n)$ local steps from their specified initial positions. Configurations and predecessor candidates outside that interval are invalid. To obtain a simulated head's scanned symbol, compare its stored coordinate with the input boundaries, supply any required endmarker or off-data blank according to the source tape convention, and otherwise scan the real endmarked input to the designated position using a logarithmic-space counter. A position forbidden by the source tape convention remains invalid. The physical scan stays between the real endmarkers, even when a valid simulated coordinate is outside the data. This routine makes no recursive graph query.

An already halted computation keeps its complete tape and head data while its time advances. In particular, halted configurations are not merged into a single vertex. Let e be one precisely at accepting configurations at time $T(n)$, and zero elsewhere. Final-time rows have no outgoing transitions. Thus S is substochastic and has transitions only to later times, and it is nilpotent. Its inverse expression is the finite sum

$$(I - S)^{-1} e = \sum_{j=0}^{T(n)} S^j e.$$

Starting from any valid configuration, at most one reward can be collected, at final time. The sum is therefore a probability in $[0,1]$. At x_0 it is exactly the machine's acceptance probability. □

The coordinatewise bound on p_0 will be used after weighted projections to larger vertex sets. It holds at every configuration, independently of the acceptance probability at the designated start. Thus the matrix construction and the probability approximation below apply to arbitrary randomized computations satisfying the resource bounds.

Size convention. Fix an integer parameter B, a power of two, comparable with $\log_2(n+2)$ and at least a sufficiently large fixed constant. For example, take the least power of two at least the maximum of that constant and $\lceil \log_2(n+2) \rceil$. This choice is computable in logarithmic space and includes all bounded input lengths in the same construction. All constants throughout the construction are independent of the input; they may depend on the fixed machine and, for fixed inverse-polynomial accuracy, its fixed exponent. The uniform precision parameter of Theorem 1.2 is handled separately by padding. The number of stages is $L = O(B)$. At stage l, a vertex has a base identifier followed by l symbols from a fixed radix D_0. The radix will be selected by the hierarchy. Encode each base vertex by a fixed-width radix-D_0 word and prepend the digit 1; unused base words are invalid. Appending one copy digit at each stage gives injective numerical identifiers even across different stage lengths. Indeed, a base width w places stage-l identifiers in $[D_0^{w+l}, 2D_0^{w+l})$, and these intervals are disjoint. Appending a fixed suffix is an affine map on identifiers.

Every stage has size at most a common bound

$$N = 2^{C_N B}.$$

We may increase the fixed constant C_N as needed. This is a bound on one stage, not a claim that there are only N pairs of vertices. Integrated entry errors below carry a factor $1/N$ while summing over both endpoints.

Fix integers $H \geq 4$ and $2 \leq h_0 < H$, and put

$$b_k = 2^{-Hk} \qquad (k \geq 0).$$

These are the nominal sampling rates. Stage l will have entry bound $f = b_l$. The random tables will use rates from this same grid for their source and terminal retention tests. The error power in the later analysis is always $s = 8$. The opening paragraph of Section 8.4 explains the role of this choice: the coarse- and fine-bin costs have exponents $1/s$ and $(s-1)/s$, which lie on either side of the precision weight $1/2$, with margin $3/8$ when $s = 8$.

Uniform constants and finite ranges. All thresholds needed by the algorithm can be chosen rational, with fixed gaps, and all discrete constants can be chosen integer. We first prove estimates with constants uniform in the ambient finite universe. The sampling construction applies to every finite range of identifiers and rates once its prime field is sufficiently large. The hierarchy uses $O(B)$ copy digits and rates with $k = O(B)$; Section 13.1 will verify this rate bound also for the recursive estimator and give a compatible order of constant choices.

We can then take the prime field to have size $2^{O(B)}$, with its exponent chosen after these constants. Its elements and all identifiers occupy $O(B)$ bits. Increasing that exponent does not change the probabilistic constants proved in the next section.

3 Reduction to fixed-precision approximation

The main theorems follow from an approximation algorithm with a fixed inverse-polynomial error bound. We state that intermediate result here and give the deductions. Its construction, including the median and the simultaneous time and space bounds, is proved in Section 13.

All transducers have read-only input and a one-way write-only output tape. For a fixed randomized machine $\mathcal{M}$, write $p_{\mathcal{M}}(x)$ for its acceptance probability on x.

Theorem 3.1 (Fixed inverse-polynomial accuracy)**.** *Fix a randomized machine $\mathcal{M}$ with polynomial worst-case running time and $O(\log(n+2))$ work space, and fix an integer $d \geq 1$. There is a uniform deterministic transducer that, on every input x of length n, outputs a dyadic rational $z_x \in [0,1]$ with*

$$|z_x - p_{\mathcal{M}}(x)| \leq (n+2)^{-d}.$$

Its work space and output length are $O(\log(n+2))$, and its running time is polynomial in $n+2$. The constants may depend on $\mathcal{M}$ and d, but not on the input.

The proof in Section 13 combines the deterministic reward approximation of Proposition 5.7, the moment bound of Theorem 10.2, and the total space-bounded implementation of Theorem 12.1. The deductions below need only the stated guarantee of Theorem 3.1.

3.1 The class equality and promise separation

Proof of Theorem 1.1. Let $\mathcal{M}$ recognize a language in BPL, with acceptance probability at most $1/3$ on no inputs and at least $2/3$ on yes inputs. Apply Theorem 3.1 with $d = 3$. For every input length, including $n = 0$, its error is at most $(n+2)^{-3} \leq 1/8$. Hence the dyadic answer is at most $1/3 + 1/8 = 11/24$ on a no input and at least $2/3 - 1/8 = 13/24$ on a yes input. Comparing this answer with $1/2$ decides the language uniformly in logarithmic space. This proves $\mathsf{BPL} \subseteq \mathsf{L}$ directly from probability approximation.

A deterministic machine may ignore its coins, giving $\mathsf{L} \subseteq \mathsf{RL}$. Two independent repetitions of an RL machine, accepting if either repetition accepts, have acceptance probability zero on no inputs and at least $3/4$ on yes inputs. They use logarithmic space and polynomial time, so $\mathsf{RL} \subseteq \mathsf{BPL}$. The three inclusions prove $\mathsf{L} = \mathsf{RL} = \mathsf{BPL}$. □

A promise problem is a pair (Y, N) of disjoint sets of inputs. The class prBPL of the introduction consists of the pairs for which one polynomial-time randomized logarithmic-space machine halts on every input, accepts each input in Y with probability at least $2/3$, and accepts each input in N with probability at most $1/3$. The class prL consists of pairs separated by a total deterministic logarithmic-space decider; there is no correctness requirement outside $Y \cup N$.

Corollary 3.2. *Under these conventions,* $\mathsf{prBPL} = \mathsf{prL}$.

Proof. The same $d = 3$ approximation and threshold $1/2$ used in the preceding proof accept every input in Y and reject every input in N. The resulting machine halts also outside the promise because its approximation queries are total. The reverse inclusion follows by ignoring the coins. □

3.2 Accuracy supplied as part of the input

Theorem 3.1 fixes the accuracy exponent before reading the input. To obtain one algorithm for arbitrary input precision q, we instead fix its exponent to one and apply it to a longer input represented implicitly. Padding preserves the source acceptance probability while making the inverse-linear error small enough for the requested accuracy.

Proof of Theorem 1.2. Fix a padded machine $\mathcal{M}_{\mathrm{pad}}$ as follows. On a string beginning with $1^n 0 x$, where $|x| = n$, it simulates $\mathcal{M}(x)$ and ignores all remaining symbols. If the delimiter or the following n symbols are absent, it rejects. This is one fixed randomized machine with polynomial worst-case running time and logarithmic work space in its full input length. On every string with the specified prefix its acceptance probability is $p_{\mathcal{M}}(x)$. Fix once and for all the transducer of Theorem 3.1 for $\mathcal{M}_{\mathrm{pad}}$ and $d = 1$.

On input $(x, 1^q)$, with $n = |x|$ and $q \geq 1$, set

$$m = 2^{q+\lceil \log_2(2n+4) \rceil + 4}.$$

Simulate that fixed transducer on the virtual length-m string

$$y = 1^n 0 x 0^{m-(2n+1)}.$$

An input query is answered by comparing its address with the prefix boundaries and either returning the appropriate bit of x or a fixed bit. The simulated input-head address and all these comparisons use $O(\log m)$ bits; the string y is never stored. The simulated transducer returns a dyadic rational $z \in [0, 1]$ satisfying

$$|z - p_{\mathcal{M}}(x)| \leq (m+2)^{-1} < 2^{-q-4}.$$

Its numerator and exponent have $O(\log m)$ bits and can be retained in work space while its finite output is simulated.

Compute by exact dyadic arithmetic

$$a = \left\lfloor 2^{q+2} z + \tfrac{1}{2} \right\rfloor.$$

Then $0 \leq a \leq 2^{q+2}$, and rounding gives

$$\left| a 2^{-(q+2)} - p_{\mathcal{M}}(x) \right| \leq 2^{-q-3} + 2^{-q-4} < 2^{-q}.$$

Write the binary numerator a to the output tape. It has at most $q + 3$ bits and represents the denominator specified in the theorem. The simulator state, virtual input address, actual input counters, captured dyadic answer, and rounding operands occupy a fixed number of records of total size

$$O(\log m) = O(\log(n+2) + q).$$

The simulation and the synthesis of each input bit take polynomial time in m, so the total time is $m^{O(1)} = (n+2)^{O(1)} 2^{O(q)}$. The padded machine and its exponent-one approximator were fixed before reading q. Thus their constants are independent of the requested accuracy. Throughout their simulation, the original size convention is $B = \Theta(\log(m+2))$, exactly as required by the estimator and controller theorems. □

4 Sampling matrices for retained vertex sets

The hierarchy will use random retained sets to detect columns with many substantial entries. The table construction will use the same sets to represent matrix entries. We define the sampling matrices and prove the marginal, conditional-pair, and count estimates needed for these two constructions. Section 7 later constructs averaging walks that preserve specified retention conditions.

The environment consists of the independent sampling matrices below together with a finite string of independent uniform auxiliary bits. The length and uses of that string will be specified when needed.

We use a fixed number of independent sampling matrices, including sampling matrices denoted X, Y, Z. Extra sampling matrices, when needed, are also a fixed finite number. A sampling matrix is a uniformly random matrix

$$A = (a_0, F) \in \mathbb{F}_P^{H \times h_0}$$

conditioned on the last $r = h_0 - 1$ columns F being linearly independent. The integer P is prime. It is chosen larger than every identifier and sufficiently large for all rate comparisons below. For a numerical identifier y, set

$$v(y) = (1, y, \dots, y^{h_0-1})^{\mathsf{T}}, \qquad a(y) = Av(y) \in \mathbb{F}_P^H.$$

Field elements in a retention test use their representatives in $\{0, \dots, P-1\}$. At nominal rate b_k, let

$$L_k = \lceil P2^{-k} \rceil, \qquad \pi_{b_k} = (L_k/P)^H.$$

The identifier is retained if all coordinates of $a(y)$ lie in $\{0, \dots, L_k - 1\}$. Write $X_a(y)$ for the indicator on sampling matrix X at rate a, and similarly for other sampling matrices. A role sampling matrix may have a fixed radix-D_0 suffix ξ. For a numerical identifier x, write

$$x \| \xi = D_0^{|\xi|} x + [\xi]_{D_0},$$

where $[\xi]_{D_0}$ is the radix value of the suffix and the empty suffix has length and value zero. Appending a fixed suffix is an affine map of the identifier (Section 2). With suffix ξ on sampling matrix X, the notation $X_a(x)$ means the retention test at $x\|\xi$ using that same sampling matrix. The suffix is common to every argument of that role sampling matrix. If $x_i = (x, i)$ is a copy, then

$$x_i \| \xi = x \| (i\xi), \tag{4.1}$$

with numerical copy identifier $D_0 x + i$. Thus a projection to the parent replaces the suffix ξ by $i\xi$ and preserves every retention indicator pointwise, at every rate and in every fixed environment. All padded identifiers are included in the range below P. Retention boxes are nested. All rates in this section belong to the grid $b_k = 2^{-Hk}$, and the admissible range satisfies $P2^{-k} \geq 1$. The choices of H, h_0 and the identifier range are those of Section 2.

Lemma 4.1 (Sampling matrix estimates)**.** *There are positive constants $c_\pi, C_\pi, C_{\text{pair}}, C_{\text{var}}$, depending only on H, h_0, with the following properties in every admissible finite range.*

1. *A specified identifier is retained at rate a with probability π_a, where $c_\pi a \leq \pi_a \leq C_\pi a$.*
2. *For distinct identifiers x, y and any admissible rates u, a,*

$$\Pr\{X_a(y) = 1 \mid X_u(x) = 1\} \leq C_{\text{pair}} a.$$

3. *If A_0 is a deterministic set of q distinct identifiers and $T_a = \sum_{y \in A_0} X_a(y)$, then*

$$\Pr\{T_a < q\pi_a/2\} \leq \frac{C_{\text{var}}}{q\pi_a} \quad (q > 0).$$

4. *Replacing every sampling matrix argument y by the same injective affine map $\alpha y + \beta$ leaves the joint law of all evaluations unchanged, provided $\alpha \neq 0$ in $\mathbb{F}_P$. In particular, a common fixed padding suffix leaves their law unchanged.*

Proof. The probability that an unrestricted H-by-r matrix has independent columns is

$$q_{\mathrm{ind}} = \prod_{j=0}^{r-1} (1 - P^{j-H}) \geq c_{\mathrm{ind}} > 0.$$

For fixed positive-degree coefficients, a_0 is uniform, so any one evaluation is uniform in $\mathbb{F}_P^H$ and independent of the event that the columns are independent. This proves the exact marginal retention probability. In the ranges used, take $P2^{-k} \geq 1$. Then $P2^{-k} \leq L_k \leq 2P2^{-k}$, proving comparability. The prime exponent can in fact make all these sides much larger than one.

Without conditioning on independent columns, two evaluations at distinct field points are independent uniform vectors: the two evaluation functionals have rank two since $h_0 \geq 2$. Consequently

$$\Pr\{X_a(y) = 1, X_u(x) = 1 \mid F \text{ has independent columns}\} \leq \frac{\pi_a \pi_u}{q_{\mathrm{ind}}}.$$

The first marginal after conditioning is still π_u, which proves the second assertion.

For the third assertion, in the unrestricted distribution the hit count has mean $q\pi_a$ and variance at most $q\pi_a$, by pairwise independence. Chebyshev's inequality bounds the stated lower-tail event by $4/(q\pi_a)$. If $\mathcal{E}$ denotes this lower-tail event, then

$$\Pr(\mathcal{E} \mid F \text{ has independent columns}) \leq \frac{\Pr(\mathcal{E})}{q_{\mathrm{ind}}} \leq \frac{4}{q_{\mathrm{ind}} q\pi_a} \leq \frac{4}{c_{\mathrm{ind}} q\pi_a}.$$

Thus one may take $C_{\mathrm{var}} = 4/c_{\mathrm{ind}}$.

Finally, write $r = h_0 - 1$ and let a_j be coefficient column j. After substituting $\alpha y + \beta$, the new columns are

$$a'_0 = \sum_{j=0}^{r} \beta^j a_j, \qquad a'_k = \alpha^k \sum_{j=k}^{r} \binom{j}{k} \beta^{j-k} a_j \quad (1 \leq k \leq r).$$

Thus $F' = FT$ for an invertible triangular r-by-r matrix T with diagonal $\alpha, \ldots, \alpha^r$. In particular, $\operatorname{rank} F' = \operatorname{rank} F$, and the constant column is translated by a function of F. The full coefficient change is bijective, with inverse given by substitution $y \mapsto \alpha^{-1}(y - \beta)$. It preserves the uniform measure on arrays conditioned on independent nonconstant columns and sends $A'v(y)$ to $Av(\alpha y + \beta)$ for every y simultaneously. This proves invariance of the joint evaluations, including retention tests at different rates. Appending digits in radix D_0 has α a power of D_0; taking $P > D_0$ ensures the required invertibility. □

5 The exact correction and copy hierarchy

At each stage we reduce the transition entries and transfer their contribution to a reward. The algebra behind this transfer is simple. Given a transition matrix C and a nonnegative correction E, put

$$D = C - E + EC.$$

Then $I - D = (I + E)(I - C)$. Thus a relation $W = (I - C)p$ becomes $(I + E)W = (I - D)p$: the same vector p is represented by the new transition and the updated reward. The correction must make D nonnegative and keep its row sums at most one. It must also reduce entries sufficiently fast that, after a controlled increase in the number of active vertices, the remaining transition contribution becomes small. The construction uses only the model reduction.

Proposition 5.7 gives the output of this construction: after l stages, the reward at a copy with every appended digit zero approximates the base acceptance probability from below with error at most $|V_0|2^{-(H-1)l}$. Thus $O(B)$ stages suffice for fixed inverse-polynomial accuracy. Its proof combines the one-stage entry bound and active-count bounds of Proposition 5.5 with the reward update in Equation (5.28).

At one stage the input is a finite ordered set V, a rate $0 < f \le 1$ on the grid of Section 4, and a nonnegative matrix C with

$$C(x,y) = 0 \quad \text{unless } x < y, \qquad C\mathbf{1} \le \mathbf{1}, \qquad C(x,y) \le f. \tag{5.1}$$

All matrix and vector inequalities are entrywise. Write

$$\mathcal{R}(C) = \{x : \text{some } C(x,y) > 0\}, \quad \mathcal{T}(C) = \{y : \text{some } C(x,y) > 0\}, \quad \mathcal{A}(C) = \mathcal{R}(C) \cup \mathcal{T}(C).$$

The stage will reduce the entry bound from f to $f/2^H$ and increase $|\mathcal{A}(C)|$ by at most a factor of two. We first analyze the correction with an arbitrary fixed column multiplier. We then choose that multiplier by a finite detector calculation, and use copies to divide the entries in columns where correction is not fully enabled.

5.1 Correction with a fixed column multiplier

Put $q = 2^H$. Fix rational numbers $0 < \theta_1 < \theta_2 < q^{-1}/32$ and a nondecreasing Lipschitz function $Q : [0,\infty) \to [0,\infty)$ satisfying

$$(t - \theta_2)_+ \le Q(t) \le (t - \theta_1)_+. \tag{5.2}$$

Its eighth root is required to be Lipschitz on bounded intervals. From Section 6 on, the errors of bounded nonnegative tables are measured through their sth roots with $s = 8$ (Equation (6.19)). This extra regularity lets input perturbations control those root discrepancies even where a cutoff vanishes; the other cutoffs below will satisfy the same requirement. Here and below the scalar cutoffs can be chosen piecewise polynomial with rational coefficients. To see this explicitly, let β be the normalized integral of $t^7(1-t)^7$ on $[0,1]$, extended by zero to the left and one to the right. Both $\beta^{1/8}$ and $(1-\beta)^{1/8}$ are Lipschitz: near their vanishing endpoints the corresponding functions vanish to order eight, and away from those endpoints the assertion follows by differentiation. For example,

$$Q(t) = (t - \theta_1)_+ \beta\left(\frac{t - \theta_1}{\theta_2 - \theta_1}\right)$$

has all the stated properties. Near θ_1 its eighth root is of order $(t-\theta_1)_+^{9/8}$; elsewhere its root has bounded derivative on every bounded interval. Let L_Q be a fixed Lipschitz constant for Q. For real arguments, extend Q by $Q(t) = 0$ when $t < 0$. This extension preserves Lipschitz continuity of the function and its eighth root and permits evaluation on real approximations. The exact hierarchy itself uses only nonnegative arguments.

Fix a column multiplier $g : V \to [0,1]$. Starting from zero, define

$$E_0 = 0, \qquad G_i' = C + E_{i-1}C, \qquad E_i(x,y) = fQ(G_i'(x,y)/f)g(y) \quad (i \ge 1). \tag{5.3}$$

The cutoff leaves a positive residual at each corrected entry. Keeping g fixed makes the update map monotone. We next show that a fixed number of iterations suffices for every such multiplier, independently of the size of V. At a nonnegative fixed point E_* of this recurrence, put $T = C - E_* + E_*C$. In a column with $g(y) = 1$, the lower bound on Q gives $T(x,y) \le \theta_2 f$. The truncation estimate below will transfer this entry bound to a finite correction, with a small additional error. The fixed point is only a comparison object; the hierarchy will use a finite iteration.

Lemma 5.1 (Fixed point and factorial truncation)**.** *For every fixed column multiplier $0 \le g \le 1$, there is a unique nonnegative solution with transitions only to later times of*

$$E_*(x,y) = fQ((C + E_*C)(x,y)/f)g(y). \tag{5.4}$$

For the recurrence in Equation (5.3)*, $0 \le E_i \le E_*$, and for every $k \ge 0$,*

$$\max_{x,y} \frac{E_*(x,y) - E_k(x,y)}{f} \le e^{1/\theta_1} \frac{(L_Q/\theta_1)^k}{k!}. \tag{5.5}$$

Fix an integer k_* for which the right side of Equation (5.5) is at most $q^{-1}/32$. This choice is uniform in C, f, V, and g.

Proof. Order the vertices compatibly with the time order. For a fixed row x, the entry $(E_*C)(x,y)$ involves only $E_*(x,z)$ with $z < y$. Equation (5.4) therefore defines the row successively, in increasing column order. Columns at or before x receive zero. This proves existence and uniqueness in the stated class. Monotonicity of the update map proves $E_i \le E_*$ and the nonnegativity of the differences.

Set $T = C - E_* + E_*C$. If $a = (C + E_*C)(x,y)$, then Equation (5.2) implies

$$0 \le E_*(x,y) \le (a - \theta_1 f)_+.$$

Thus $T \ge 0$, and at every positive position of E_* one has $T(x,y) \ge \theta_1 f$. Also

$$T\mathbf{1} = C\mathbf{1} - E_*(\mathbf{1} - C\mathbf{1}) \le C\mathbf{1} \le \mathbf{1}. \tag{5.6}$$

Consequently, for each fixed row x, its set $A_x = \{y : E_*(x,y) > 0\}$ has cardinality $m_x \le 1/(\theta_1 f)$.

Unroll the inequality $E_* \le C + E_*C$ along this row's support. Each resulting term for $E_*(x,y)$ is a product of entries of C along a sequence

$$x < z_1 < \cdots < z_j < y, \qquad z_1, \ldots, z_j \in A_x.$$

Every substitution keeps the source coordinate equal to x: the recursive factor is $E_*(x,z)$, while only its terminal coordinate changes. All intermediate vertices therefore belong to the same set A_x. Strict ordering makes a choice of intermediate vertices specify at most one such sequence. There are at most $\binom{m_x}{j}$ such sequences, and each product is at most f^{j+1}. The expansion terminates because every sequence is strictly increasing. Therefore

$$E_*(x,y) \le f \sum_{j=0}^{m_x} \binom{m_x}{j} f^j = f(1+f)^{m_x} \le f e^{1/\theta_1}. \tag{5.7}$$

Only the support of the single fixed row x was used here; the number of vertices in the whole graph does not enter the exponent.

Let $\Delta_i = E_* - E_i$. These matrices are nonnegative, and their row-x supports are contained in A_x. The Lipschitz bound gives

$$\Delta_i(x,y) \le L_Q \sum_{z<y} \Delta_{i-1}(x,z) C(z,y).$$

Outside A_x the left side is zero, so when this inequality is unrolled it suffices to use intermediate vertices from A_x at every step. After k steps there are at most $\binom{m_x}{k}$ strictly increasing sequences of

intermediate vertices. Their initial differences are at most fe^{1/θ_1} by Equation (5.7), and their k transition factors are at most f^k. Thus

$$\Delta_k(x,y) \le fe^{1/\theta_1} L_Q^k f^k \binom{m_x}{k} \le fe^{1/\theta_1} \frac{(L_Q/\theta_1)^k}{k!}.$$

For $k > m_x$ the first upper bound is zero. This proves Equation (5.5). The factorial eventually dominates the fixed exponential factor, giving the stated fixed choice of k_*. □

The iteration count k_* is now fixed independently of the detectors. We next choose g so that correction is supported on columns with only boundedly many substantial entries. A pilot selects this multiplier; the actual recurrence then uses that one multiplier for all k_* steps.

5.2 A finite pilot chooses the column multiplier

All expectations defining the detectors are true expectations; an estimate is never used to define its own target.

For the detectors we use independent sampling matrices X, Y from Section 4. Denote their common retention probability at rate f by π_f. The consequences of Lemma 4.1 needed here can be written, with fixed positive constants, as

$$c_\pi f \le \pi_f \le C_\pi f, \qquad \Pr\{Y_f(w) = 1 \mid Y_f(y) = 1\} \le C_{\mathrm{pair}} f \quad (w \ne y), \tag{5.8}$$

$$\Pr\left\{\sum_{x\in A} X_f(x) < \tfrac{1}{2}|A|\pi_f\right\} \le \frac{C_{\mathrm{var}}}{|A|\pi_f} \quad (\varnothing \ne A \subseteq V \text{ deterministic}). \tag{5.9}$$

These constants are uniform in the stage, the admissible moduli, and common fixed padding suffixes.

All threshold choices and ranks below are fixed constants. The pilot uses the iteration count from Lemma 5.1. Choose

$$0 < \delta_0 < \delta_1 < \theta_1/100, \qquad 0 < \eta_- < \eta_+ < \delta_0/3.$$

If other row scores require still smaller detector thresholds, make those choices before fixing the ranks. Let ϕ be a nondecreasing cutoff which is zero through δ_0 and one from δ_1 on. Let ρ be a nonincreasing cutoff which is one through η_- and zero from η_+ on. Finally, let $\gamma : [0,1] \to [0,1]$ be nonincreasing, one on $[0, 1/2]$, and zero on $[4/5, 1]$. These functions and their complements, when used, have the root regularity just described.

For a finite multiset of nonnegative numbers, $\mathrm{largest}_j$ means its j-th largest member, with enough additional zeros that this is always defined. Starting from $\widehat{E}_0 = 0$, perform k_* pilot steps. At step i, first set

$$G_i = C + \widehat{E}_{i-1} C. \tag{5.10}$$

For a draw of the terminal sampling matrix, set

$$R_i(x) = \rho\left(\mathrm{largest}_K\{G_i(x,w)/f : Y_f(w) = 1\}\right). \tag{5.11}$$

Define the deterministic column number

$$U_i(y) = \mathbb{E}\left[\mathrm{largest}_J\{\phi(G_i(x,y)/f) R_i(x) : X_f(x) = 1\} \mid Y_f(y) = 1\right]. \tag{5.12}$$

Then put

$$\widehat{g}_i(y) = \gamma\left(\max_{1\le r\le i} U_r(y)\right), \qquad \widehat{E}_i(x,y) = fQ(G_i(x,y)/f)\widehat{g}_i(y). \tag{5.13}$$

The order of these definitions is essential: G_i is already deterministic when its detector is formed, and U_i is defined before $\widehat{E}_i$. The running maximum makes $\widehat{g}_i$ nonincreasing. Its final value is therefore no larger than any earlier pilot multiplier; monotonicity of Q will let us bound the actual corrections by the pilot corrections.

After the pilot, set $g = \widehat{g}_{k_*}$ and specialize the fixed-multiplier recurrence of Equation (5.3) to the actual correction

$$E_0 = 0, \qquad G_i' = C + E_{i-1}C, \qquad E_i(x,y) = fQ(G_i'(x,y)/f)g(y) \quad (1 \le i \le k_*). \tag{5.14}$$

Write

$$E = E_{k_*}, \qquad D = C - E + EC. \tag{5.15}$$

Lemma 5.2 (Elementary pilot bounds)**.** *All pilot and actual inputs and corrections are nonnegative with transitions only to later times. For* $1 \le i \le k_*$*,*

$$G_i\mathbf{1},\ \widehat{E}_i\mathbf{1},\ G_i'\mathbf{1},\ E_i\mathbf{1} \le i\mathbf{1},$$
$$G_i(x,y),\ \widehat{E}_i(x,y),\ G_i'(x,y),\ E_i(x,y) \le if.$$

Moreover,

$$E_{i-1} \le E_i \le \widehat{E}_i, \qquad G_i' \le G_i.$$

The row and column supports of every matrix in this statement are contained in $\mathcal{R}(C)$ *and* $\mathcal{T}(C)$*, respectively.*

Proof. Equation (5.2) gives $Q(0) = 0$ and $0 \le Q(t) \le t$ for $t \ge 0$. Consequently $\widehat{E}_i \le G_i$. If $\widehat{E}_{i-1}\mathbf{1} \le (i-1)\mathbf{1}$, then

$$G_i\mathbf{1} = C\mathbf{1} + \widehat{E}_{i-1}C\mathbf{1} \le \mathbf{1} + \widehat{E}_{i-1}\mathbf{1} \le i\mathbf{1}.$$

For an individual entry,

$$G_i(x,y) \le f + f\sum_z \widehat{E}_{i-1}(x,z) \le if.$$

This proves the pilot bounds by induction.

The sequence $\widehat{g}_i$ is nonincreasing, so $g \le \widehat{g}_i$ for every i. Induction in Equations (5.10)–(5.14), using monotonicity of Q, proves $E_i \le \widehat{E}_i$ and $G_i' \le G_i$. The map $F \mapsto fQ((C + FC)/f)g$, applied entrywise after forming the matrix product, is monotone. Its iteration from zero is therefore nondecreasing. The actual bounds follow from the pilot bounds.

A product of matrices with transitions only to later times also has transitions only to later times. A zero row of C remains a zero row in all these formulas, by induction from the zero correction. A zero column of C is a zero column of every product of the form FC and hence of every input and correction. This proves the remaining assertions. □

For the column test, take an integer $K \ge 160(1 + C_{\mathrm{pair}}k_*/\eta_-)$, large enough also for the other finitely many row-cutoff estimates. Fix any positive integer J and take

$$R_* \ge c_\pi^{-1} \max(4J, 40C_{\mathrm{var}}). \tag{5.16}$$

Lemma 5.3 (The detector bounds overloaded columns)**.** *With these choices, for every pilot step* $1 \le i \le k_*$ *and column* y*:*

1. *If* $\#\{x : G_i(x,y) \ge \delta_1 f\} > R_*/f$*, then* $U_i(y) \ge 19/20 > 9/10$*.*
2. *If* $U_i(y) > 1/4$*, then*
$$\#\{x : G_i(x,y) > \delta_0 f\} > \frac{J}{4C_\pi f}.$$
3. *For any pilot correction with multiplier* $\widehat{g}_i(y) > 0$*,*
$$\#\{x : G_i(x,y) \ge \delta_1 f\} \le R_*/f.$$
For every actual correction, $g(y) > 0$ *implies*
$$\#\{x : G'_i(x,y) \ge \delta_1 f\} \le R_*/f.$$

Proof. Put $D_{\mathrm{row}} = 1 + C_{\mathrm{pair}} k_*/\eta_-$. Fix i, y and condition on $Y_f(y) = 1$. In a fixed row, the set $\{w : G_i(x,w) > \eta_- f\}$ has cardinality at most $k_*/(\eta_- f)$ by Lemma 5.2. Its expected number of retained members is at most D_{row}: the specified terminal y, if in this set, contributes at most one, and Equation (5.8) applies to every other member. If the row cutoff is not full, at least K such members have been retained. Markov's inequality therefore gives

$$\Pr\{R_i(x) < 1 \mid Y_f(y) = 1\} \le D_{\mathrm{row}}/K. \tag{5.17}$$

Let $A = \{x : G_i(x,y) \ge \delta_1 f\}$ and $n_A = |A|$. Let N_A count its retained starts, and let L_A count those retained starts whose row cutoff is not full. Independence of the two sampling matrices and Equation (5.17) imply

$$\mathbb{E}[L_A \mid Y_f(y) = 1] \le n_A \pi_f D_{\mathrm{row}}/K.$$

Thus

$$\Pr\{L_A > n_A \pi_f/4 \mid Y_f(y) = 1\} \le 1/40.$$

The law of the start sampling matrix is unchanged by the conditioning, so Equation (5.9) gives

$$\Pr\{N_A < n_A \pi_f/2 \mid Y_f(y) = 1\} \le C_{\mathrm{var}}/(n_A \pi_f).$$

If $n_A > R_*/f$, Equation (5.16) ensures $n_A \pi_f \ge \max(4J, 40C_{\mathrm{var}})$. Outside an event of probability at most $1/20$, there are at least J retained high rows with full cutoffs. Each has score exactly one, so the trial's J-th largest score is one. All trial scores lie in $[0,1]$, proving the first assertion.

For the second assertion let $A_0 = \{x : G_i(x,y) > \delta_0 f\}$, and let N_0 be its number of retained starts. A positive score must come from A_0. Pointwise the trial is at most $\mathbf{1}_{\{N_0 \ge J\}}$, and hence at most N_0/J. Its conditional expectation is at most $|A_0|\pi_f/J \le |A_0| C_\pi f/J$. The asserted lower bound on $|A_0|$ follows.

Finally, a positive $\widehat{g}_i(y)$ implies $U_i(y) < 4/5$, so the first assertion excludes a high-entry count greater than R_*/f. If $g(y) > 0$, all pilot multipliers at y are positive. Apply the same bound to every G_i and then use $G'_i \le G_i$ from Lemma 5.2. □

5.3 The finite residual

The factorial estimate gives a fixed iteration count. We now return to the finite correction $E = E_{k_*}$ and show that its residual is substochastic and has small entries on columns where correction is fully enabled. We also bound the unsigned constituents of its defining expression by the residual itself, as needed when that expression is estimated term by term.

Lemma 5.4 (Residual bounds and unsigned-term domination)**.** *For the k_* chosen in Lemma 5.1, the matrix D in Equation* (5.15) *is nonnegative and substochastic, with transitions only to later times. Its entries satisfy*

$$D(x,y) \le (k_*+1)f, \qquad g(y)=1 \implies D(x,y) \le (\theta_2 + q^{-1}/32)f < q^{-1}f/16. \tag{5.18}$$

Furthermore,

$$D \ge \frac{\theta_1}{2k_*}(C + E + EC). \tag{5.19}$$

Moreover, $\mathcal{R}(D) \subseteq \mathcal{R}(C)$ and $\mathcal{T}(D) \subseteq \mathcal{T}(C)$.

Proof. Set $A = G'_{k_*}$ and $\delta = (E - E_{k_*-1})C \ge 0$. Then

$$D = A - E + \delta, \qquad A - E \ge \min(A, \theta_1 f). \tag{5.20}$$

Here and in the minimum the scalar $\theta_1 f$ denotes the constant entrywise bound. The second inequality follows from $E \le (A - \theta_1 f)_+$. It proves $D \ge 0$. Since $C\mathbf{1} \le \mathbf{1}$ and $E \ge 0$,

$$D\mathbf{1} = C\mathbf{1} - E(\mathbf{1} - C\mathbf{1}) \le C\mathbf{1} \le \mathbf{1}.$$

Also $D \le C + EC$, whose entries are at most $f + f\sum_z E(x,z) \le (k_*+1)f$ by Lemma 5.2. The increase in time along transitions and the support claims follow from the same formulas.

Because $A(x,y) \le k_* f$, Equation (5.20) gives $A - E \ge (\theta_1/k_*)A$. On the other hand,

$$C + E + EC = A + E + \delta \le 2A + \delta.$$

Since $\theta_1/k_* < 1$, these two inequalities prove Equation (5.19).

For the good-column bound use the fixed point with this same multiplier g, and set $\Delta = E_* - E$. With $T = C - E_* + E_* C$,

$$D = T + \Delta - \Delta C \le T + \Delta.$$

If $g(y) = 1$, the lower bound in Equation (5.2) gives $T(x,y) \le \theta_2 f$. Equation (5.5) gives $\Delta(x,y) \le q^{-1}f/32$. This proves Equation (5.18). □

Every unsigned term in $C - E + EC$ is bounded by a fixed multiple of D. In $G_i = C + \widehat{E}_{i-1}C$ and $G'_i = C + E_{i-1}C$, nonnegativity bounds each summand by its corresponding input. These bounds let us compare the three unsigned contributions C, E, and EC separately against their estimates, measuring each error relative to the same nonnegative target D. Their signs are then combined by a triangle inequality in Lemma 8.3. The estimated raw arrays may have negative entries; positivity here is a property of the exact target.

5.4 Copies reduce entry bound while controlling active vertices

The residual already has small entries in columns with $g(y) = 1$. We distribute the remaining large entries among copies of their destinations. The detector incidence bound will limit the number of parents with active extra copies, and a weighted projection will relate the new transition to the residual.

Choose an integer $D_0 > (k_* + 1)q$, and let $\tau_0 = q^{-1}/8$, $\tau_1 = q^{-1}/4$. Choose a rising cutoff χ which is zero through τ_0 and one from τ_1 on. For $0 \le j < D_0$ set

$$s_0(t) = 1 - (1 - D_0^{-1})\chi(t), \qquad s_j(t) = D_0^{-1}\chi(t) \quad (j > 0). \tag{5.21}$$

They are nonnegative, sum to one, and have Lipschitz eighth roots on bounded ranges. Define $U^{\max}(x) = \max_{1 \le i \le k_*} U_i(x)$. Let κ be a rising cutoff, zero through $1/4$ and one from $1/2$ on, and put

$$r_0(x) = 1, \qquad r_j(x) = \kappa(U^{\max}(x)) \quad (j > 0). \tag{5.22}$$

On $V^+ = V \times \{0, \dots, D_0 - 1\}$ write $x_i = (x, i)$ and set

$$C^+(x_i, y_j) = r_i(x)D(x, y)s_j(D(x, y)/f). \tag{5.23}$$

Order the copies first by their parent and then by their copy symbol.

Let P^r be the matrix from V^+ to V given by

$$P^r(x_i, x) = r_i(x), \qquad P^r(x_i, z) = 0 \quad (z \ne x). \tag{5.24}$$

Proposition 5.5 (One-stage entry bound and active-count reduction)**.** *Suppose the detector rank satisfies*

$$J \ge \frac{2C_\pi(D_0 - 1)k_*(k_* + 1)}{\delta_0}, \tag{5.25}$$

and R_* satisfies Equation (5.16)*. The matrix C^+ is substochastic with transitions only to later times, and*

$$C^+(x_i, y_j) \le f/q, \qquad |\mathcal{A}(C^+)| \le 2|\mathcal{A}(C)|.$$

The weighted projection satisfies

$$C^+P^r = P^rD. \tag{5.26}$$

Proof. The row sum at x_i is $r_i(x)\sum_y D(x, y) \le 1$. The increase in time along transitions follows from that of D. Put $t = D(x, y)/f$. If $t < \tau_1$, then every routed entry is at most $ft < f/q$. If $t \ge \tau_1$, all fractions equal $1/D_0$, so every entry is at most $(k_* + 1)f/D_0 < f/q$. This proves the entry bound bound.

If a fraction for a nonzero copy label $s_j(D(x, y)/f)$ is positive on a positive entry, then $D(x, y)/f > \tau_0 > \theta_2 + q^{-1}/32$. Lemma 5.4 therefore implies $g(y) < 1$. Since γ equals one through $1/2$, this gives $U^{\max}(y) > 1/2$, and hence $r_j(y) = 1$. The factor for copy label zero is always one. Thus for every positive entry of D,

$$\sum_{j=0}^{D_0-1} s_j(D(x, y)/f)r_j(y) = 1. \tag{5.27}$$

For a zero entry no identity of its fractions is needed. For each parent y the only nonzero entries of the y-column of P^r are at its own copies. Hence, for every source copy x_i,

$$(C^+P^r)(x_i, y) = r_i(x)D(x, y)\sum_j s_j(D(x, y)/f)r_j(y) = r_i(x)D(x, y) = (P^rD)(x_i, y).$$

This gives Equation (5.26) with the destination row factors included in the projection.

It remains to count active vertices. Put

$$\mathcal{O} = \{y : U^{\max}(y) > 1/4\}, \qquad n' = |\mathcal{A}(C)|.$$

Every $y \in \mathcal{O}$ has, for some pilot step i, more than $J/(4C_\pi f)$ entries with $G_i(x,y) > \delta_0 f$, by Lemma 5.3. Counting these incidences over all steps and all active rows, and using Lemma 5.2, gives

$$\begin{aligned} |\mathcal{O}|\frac{J}{4C_\pi f} &\le \sum_{i=1}^{k_*} \#\{(x,y) : G_i(x,y) > \delta_0 f\} \\ &\le \frac{n'}{\delta_0 f}\sum_{i=1}^{k_*} i = \frac{n' k_*(k_*+1)}{2\delta_0 f}. \end{aligned}$$

The assertion is also valid when $n' = 0$, since then all matrices and detectors vanish. Thus

$$|\mathcal{O}| \le \frac{2C_\pi n' k_*(k_*+1)}{\delta_0 J}.$$

Every active child has an active parent, by the support conclusion of Lemma 5.4. A child with a nonzero copy label active as a row has $r_i(x) > 0$, so its parent belongs to $\mathcal{O}$. A nonprimary child active as a column has a positive extra routing fraction and, as shown above, its parent has $U^{\max} > 1/2$, so again belongs to $\mathcal{O}$. Therefore

$$|\mathcal{A}(C^+)| \le n' + (D_0 - 1)|\mathcal{O}| \le 2n',$$

where the last inequality is Equation (5.25). □

Remark 5.6 (Order of the one-stage constants). The bounds used to select k_* in Lemma 5.1 hold for every multiplier in $[0,1]$. Thus $\theta_1, \theta_2, Q, k_*$ and D_0 can be fixed before any detector rank. Next fix all finitely many correction, detector, and access-score threshold ladders (the threshold templates and access classes of Section 6), taking δ_0, δ_1 below the correction support thresholds as required. Choose K large enough for the corresponding row estimates. Choose J by Equation (5.25), and then R_* by Equation (5.16). An incoming rank used later can be chosen after R_*: its full-cutoff probability is controlled by the column bound in Lemma 5.3, independently of the auxiliary row cutoffs. There is therefore no requirement to enlarge K again in response to that incoming rank. Subsequent estimator constants may depend on these already fixed ranks and thresholds.

5.5 Reward transport and the final deterministic target

The copy step controls transition entries and active vertices. We now iterate it and transport the terminal reward through the same weighted projections. The resulting reward will approximate the original acceptance probability, including probabilities strictly between zero and one.

Starting with $C_0 = S$ on V_0, repeat the one-stage construction with

$$f_l = q^{-l}, \qquad V_{l+1} = V_l \times \{0, \dots, D_0 - 1\}, \qquad C_{l+1} = C_l^+.$$

Let E_l, D_l, P_l^r denote the specified correction, residual, and weighted projection at stage l. At each stage, $r_i(x)$ denotes the row factor in Equation (5.22) for that stage. Let e be the base final-success reward and $p_0 = (I - S)^{-1}e$. Define the rewards on V_l by $W_0 = e$ and

$$W_{l+1}(x_i) = r_i(x)\big(W_l(x) + (E_l W_l)(x)\big). \tag{5.28}$$

For a base vertex x, let $\bar{x}_l$ be its copy with every appended digit zero in V_l.

Proposition 5.7 (Reward identity and a shallow target)**.** *For every stage l, $0 \le W_l \le 1$. For every base vertex x,*

$$0 \le p_0(x) - W_l(\bar{x}_l) \le |V_0|\, 2^{-(H-1)l}. \tag{5.29}$$

There is a choice $L = O(B)$ for which this upper bound is strictly less than $1/16$ for every x. For every $0 \le l \le L$, the full stage domain has size $2^{O(B)}$ and admits $O(B)$-bit addresses.

In particular, $p_0(x) = 0$ implies $W_L(\bar{x}_L) = 0$, and $p_0(x) \ge 1/2$ implies $W_L(\bar{x}_L) > 7/16$.

Proof. Define matrices from V_l to V_0 by

$$P_0^c = I, \qquad P_{l+1}^c = P_l^r P_l^c, \qquad B_0 = I, \qquad B_{l+1} = P_l^r (I + E_l) B_l. \tag{5.30}$$

The superscript c denotes the cumulative projection, not a transpose. The reward recurrence gives $W_l = B_l e$ by induction. We will prove

$$C_l P_l^c = P_l^c - B_l(I - S). \tag{5.31}$$

The assertion at $l = 0$ is $S = I - (I - S)$. The residual identity can be written as

$$D_l = I - (I + E_l)(I - C_l).$$

Using Equation (5.26) and then the inductive hypothesis,

$$\begin{aligned} C_{l+1} P_{l+1}^c &= P_l^r D_l P_l^c \\ &= P_l^r \left(P_l^c - (I + E_l)(P_l^c - C_l P_l^c)\right) \\ &= P_{l+1}^c - P_l^r (I + E_l) B_l (I - S) \\ &= P_{l+1}^c - B_{l+1}(I - S). \end{aligned}$$

This proves Equation (5.31).

Every B_l is nonnegative. Every P_l^r has at most one nonzero entry per row, of value in $[0, 1]$, and thus P_l^c is substochastic. Apply Equation (5.31) to p_0, using $(I - S)p_0 = e$. This gives

$$0 \le W_l \le 1, \qquad W_l = P_l^c p_0 - C_l P_l^c p_0. \tag{5.32}$$

The lower bound follows from $B_l e \ge 0$; the upper bound follows from $W_l \le P_l^c p_0 \le 1$. Equation (5.28) is Equation (5.30) applied to e.

By Proposition 5.5, induction gives

$$|\mathcal{A}(C_l)| \le 2^l |\mathcal{A}(S)| \le 2^l |V_0|, \qquad C_l(x, y) \le q^{-l}.$$

There are at most $|\mathcal{A}(C_l)|$ columns containing any positive entry, so each row sum of C_l is at most $|V_0| 2^l q^{-l} = |V_0| 2^{-(H-1)l}$. The projection along copies labelled zero has every row factor equal to one and hence $(P_l^c p_0)(\bar{x}_l) = p_0(x)$. The identity in Equation (5.32), together with $0 \le P_l^c p_0 \le 1$, now proves Equation (5.29).

Choose an integer L such that $(H - 1)L > \log_2(16|V_0|)$. Since $\log_2 |V_0| = O(B)$, this is $O(B)$. If $p_0(x) = 0$, nonnegativity forces $W_L(\bar{x}_L) = 0$. If $p_0(x) \ge 1/2$, the strict error bound gives $W_L(\bar{x}_L) > 7/16$. Finally,

$$|V_l| = |V_0| D_0^l = 2^{O(B)} \qquad (0 \le l \le L),$$

because D_0 is fixed. Parent projection deletes one radix symbol, and appending a specified symbol is affine in the numerical ID. Thus these domain changes preserve the address convention required by the sampling matrices and by the later implementation. □

The remainder of the proof will estimate the finite targets just constructed. In particular, it need only implement the k_* pilot and k_* actual updates at a stage and the reward update in Equation (5.28). The same error bound in Equation (5.29) gives any fixed inverse-polynomial accuracy by increasing the constant in $L = O(B)$. Section 13 makes that choice together with the statistical and numerical precisions.

6 Random sparse tables and their interfaces

The exact hierarchy uses matrix entries, correction products, and column detector expectations. We represent each of these by sample-dependent tables: bins represent entries of the transitions and residuals, correction tables represent the pilot and actual corrections, and score tables represent the detector trials. Each normalized table has an earlier preparatory table that supplies its denominator.

We first prove the exact conditional-mean identities. We then extend the same row formulas to computed raw arrays and prove the support and access bounds that hold even when those arrays are inaccurate. Finally we bound the effect of errors in the incoming cutoffs and column multipliers. The later implementation supplies enumeration and pairing of the resulting ports.

Exact conditional-mean identities refer only to the true tables defined here. Applying the same formulas to approximate raw signals gives approximate tables; their normalization need not be unbiased.

Weighted pseudorandomness provides a related way to represent expectations with few random bits; see Braverman, Cohen and Garg [8] and the error-reduction work of Cohen, Doron, Renard, Sberlo and Ta-Shma [15]. The tables below have exact conditional-mean identities. Their estimates use a common environment, so products and subsequent normalization are analyzed through joint error bounds rather than independent sampling.

6.1 Table domains and the normalization principle

Throughout this section $s = 8$. Fix a stage, write $f = b_l$, and let G be one of its nonnegative raw matrix inputs. The hierarchy gives absolute constants M_G, K_G such that

$$\sum_y G(x,y) \le M_G, \qquad G(x,y) \le K_G f. \tag{6.1}$$

These bounds include pilot and actual inputs and the residual D; see Lemmas 5.2 and 5.4. For a correction $F = fQ(G/f)g_0$, the additional input is

$$g_0(y) > 0 \quad \Longrightarrow \quad \#\{x : G(x,y) \ge \delta_1 f\} \le R_*/f, \tag{6.2}$$

from Lemma 5.3.

A row task has start rate $u \le f$, end scale a, and, when needed, a usage rate $t \le a$, all on the rate grid. Its domain is $X_u(x) = Y_a(y) = 1$, with distinct start and terminal sampling matrices. We use $a \le K_f f$ for an absolute K_f, and use $a = f$ for corrections and detector scores. Here and in the estimation and compression sections, t denotes a sampling rate; the numerical digit position ν is introduced separately in Section 11. The true first raw signal is

$$\mathsf{d}_y = G(x,y)/a. \tag{6.3}$$

An optional second raw signal is $\zeta \mathbf{1}_{\{Y_t(y)=1\}} q_y$, where $q_y = q(x,y) \in [0,1]$ is a deterministic conditional mean and $\zeta > 0$ is a fixed sufficiently small constant. That mean will be specified separately for each normalized task.

All parameters of a table are common to the whole table: its stage and iteration, task type, sampling matrix roles, sampling matrix padding suffixes, rates u, a, t, and subsequently its budget and numerical convention. They do not depend on the row or column at which the table is queried.

As in Section 5, $\mathrm{largest}_k$ denotes the kth largest value, with unlimited zero padding. For two arrays on the same padded index set,

$$|\,\mathrm{largest}_k(v) - \mathrm{largest}_k(w)| \le \sup_z |v_z - w_z|. \tag{6.4}$$

Indeed, if every coordinate changes by at most ϵ, at least k coordinates at or above the old kth value remain at or above that value minus ϵ; apply this in both directions.

The normalization can be seen before choosing the row functions. Fix a pair (x, y) on its conditioning slice, a deterministic nonnegative value $k(\mathsf{d}_y)$, and a random row cutoff $R \in [0, 1]$. Choose a preparatory factor h that is one wherever k is positive. If

$$q(x, y) = \mathbb{E}[h(\mathsf{d}_y)R \mid X_u(x) = Y_t(y) = 1] \ge \tfrac{1}{2}$$

on that support, then $k(\mathsf{d}_y)R/q(x, y)$ has conditional mean $k(\mathsf{d}_y)$ there. A clipped denominator keeps the formula bounded at every other position and for computed inputs. The constructions below arrange a uniform positive lower bound on this mean while the row cutoff limits the number of nonzero retained entries. Corrections use an additional incoming cutoff, with the same normalization principle.

For real $\alpha < \beta$, let $\chi_{\alpha,\beta}$ be a fixed increasing switch, zero on $(-\infty, \alpha]$ and one on $[\beta, \infty)$. Choose it by rescaling the normalized integral of $z^{s-1}(1 - z)^{s-1}$ on $[0, 1]$. Both $\chi_{\alpha,\beta}^{1/s}$ and $(1 - \chi_{\alpha,\beta})^{1/s}$ are Lipschitz: at either endpoint the vanishing function has a zero of order s, and on the interior ordinary differentiation applies. Products of bounded functions with Lipschitz sth roots again have Lipschitz sth roots.

Every scalar saturation below uses fixed min/max clipping,

$$\mathrm{clip}_{[\alpha,\beta]}(v) = \min\{\beta, \max\{\alpha, v\}\},$$

which is 1-Lipschitz. We specify its bounds when defining the function. Functions initially defined for nonnegative first entries are extended by zero below zero; each already vanishes below a positive support threshold. These conventions apply only to evaluation of the row functions. Order statistics and access tests always use the full prescribed raw arrays before scalar clipping.

Here is one explicit set of margins used below. Let $\vartheta > 0$ be a fixed lower support threshold of a main row function, and put $\rho = \mathrm{largest}_K(v)$ for its first raw row v. Define

$$\begin{aligned} h_\vartheta(v) &= \chi_{\vartheta/2,\,3\vartheta/4}(v), \\ R_\vartheta(\rho) &= 1 - \chi_{\vartheta/1000,\,\vartheta/500}(\rho), \\ I_\vartheta(v, \rho) &= \chi_{\vartheta/8,\,\vartheta/4}(v)\big(1 - \chi_{\vartheta/100,\,\vartheta/50}(\rho)\big). \end{aligned} \tag{6.5}$$

We take $\vartheta = \theta_1$ for corrections, $\vartheta = 1/2$ for bins, and $\vartheta = \delta_0$ for the detector score. For that detector we make the allowed hierarchy choices

$$\eta_- = \delta_0/1000, \qquad \eta_+ = \delta_0/500, \qquad \rho(t) = 1 - \chi_{\delta_0/1000,\,\delta_0/500}(t) = R_{\delta_0}(t).$$

Here ρ denotes the cutoff in the hierarchy, and the same switch χ is used in both occurrences. These choices satisfy $0 < \eta_- < \eta_+ < \delta_0/3$. Thus the detector row cutoff here is exactly R_i in Equation (5.11). The hierarchy's choices may and do satisfy $\delta_1 < \theta_1/100$ and $0 < \delta_0 < \delta_1$. Thus the correction auxiliary score I_{θ_1} can be positive only when $G(x, y) > \delta_1 f$.

6.2 Bins and preparatory normalization

We first split a raw entry into contributions at the rate-grid scales. At each scale a row cutoff bounds the number of retained entries, and an exact preparatory conditional mean corrects the loss caused by that cutoff.

Let $\chi = \chi_{1/2,1}$. For $w \geq 0$ define

$$\begin{aligned} \omega_0(w) &= \chi(w), \\ \omega_k(w) &= \chi(w/b_k) - \chi(w/b_{k-1}) \qquad (k \geq 1). \end{aligned} \tag{6.6}$$

The top bin has no preceding switch. The weights are nonnegative and sum to one for $w > 0$, since their partial sums telescope to $\chi(w/b_k) \to 1$. Hence

$$w = \sum_{k \geq 0} w\omega_k(w). \tag{6.7}$$

At scale $a = b_k$, the bin in units of the sampling scale is

$$k_a(d) = d\omega_k(ad). \tag{6.8}$$

For $k \geq 1$ its support is contained in $1/2 < d < 2^H$. Its two switch transition intervals are disjoint, and

$$\chi(d) - \chi(d/2^H) = \chi(d)(1 - \chi(d/2^H)),$$

which proves directly that its sth root is Lipschitz. For $k = 0$, fix $B_{\text{top}} > \max\{K_G, 1\}$ and evaluate the defining expression at $\text{clip}_{[0,B_{\text{top}}]}(d)$. This makes the top-bin function bounded with Lipschitz root. It preserves every true value, since $a = b_0 = 1$ and $G(x, y) \leq K_G f \leq K_G$. For $k \geq 1$, retain the zero extension outside the support $(1/2, 2^H)$; no upper clipping changes a zero bin value. Bins with $a > 2K_G f$ vanish on the true input, so a fixed $K_f \geq 2K_G$ suffices for the permitted scale range.

For the routed residual bin use instead

$$k_a^{(j)}(d) = k_a(d)s_j(ad/f). \tag{6.9}$$

The routing fractions have bounded Lipschitz roots. Since $a/f \leq K_f$ and the bin is zero above a fixed argument range (or is top-bin clipped), these functions have uniformly bounded Lipschitz roots as well. The same zero-extension and clipping conventions apply to routed bins. Summing Equation (6.9) over scales gives the routed residual contribution exactly on every true input.

Choose $R_{1/2}$ and $h = h_{1/2}$ from Equation (6.5). For the true row set

$$\rho_x = \text{largest}_K\{G(x, z)/a : Y_a(z) = 1\}, \qquad R = R_{1/2}(\rho_x).$$

Write

$$q_y = \mathbb{E}[h(\mathsf{d}_y)R \mid X_u(x) = Y_t(y) = 1], \qquad c(q) = \min\{1, \max\{1/2, q\}\}. \tag{6.10}$$

The number q_y is the deterministic conditional mean at the indicated pair. The second raw signal carries the Y_t membership mask. In addition, $q_y = 0$ if $\mathsf{d}_y \leq 1/4$. The true preparatory table on the broad terminal domain Y_a is the unnormalized value $h(\mathsf{d}_y)R$. It uses only the first raw signal. Its conditional mean on the narrower slice is exactly the q_y in Equation (6.10). The usage rate t is part of the fixed task tuple: changing the slice on which this mean is taken would in general change the denominator. The broad preparatory domain and the narrower normalization slice must therefore be retained as separate parts of the interface.

Lemma 6.1 (Bin table identity)**.** *Fix an admissible tuple (u, a, t) with $a = b_k$ as above. For sufficiently large fixed K, the true table*

$$\mathcal{B}_a(x, y) = \mathbf{1}_{\{Y_t(y)=1\}} \frac{k_a(\mathsf{d}_y)R}{c(q_y)} \tag{6.11}$$

has uniformly bounded values and row support, and is zero outside $X_u(x) = Y_t(y) = 1$. On this slice,

$$a\,\mathbb{E}[\mathcal{B}_a(x, y) \mid X_u(x) = Y_t(y) = 1] = G(x, y)\omega_k(G(x, y)).$$

For $\mathcal{B}_a^{(j)}$, defined by replacing k_a with $k_a^{(j)}$, the right side is multiplied by $s_j(G(x, y)/f)$. Apart from its start membership, its true value uses only the terminal sampling matrix.

Proof. The row cutoff can fail to be full only if at least K sampled columns have $G(x, z)/a > r$, where $r = (1/2)/1000$ is fixed. There are at most $M_G/(ar)$ such deterministic columns. Conditional on $Y_t(y) = 1$, the specified column contributes at most one hit and each other column has Y_a-hit probability at most Ca, by Lemma 4.1. Thus the expected count is at most $1+CM_G/r$. Conditioning also on the independent source sampling matrix does not alter this bound. Markov's inequality makes the failure probability at most $1/8$ by a sufficiently large K.

Where $k_a(\mathsf{d}_y) > 0$, we have $h(\mathsf{d}_y) = 1$. Therefore $q_y = \mathbb{E}[R \mid Y_t(y) = 1] \in [7/8, 1]$ there, and the clipping does nothing. The conditional mean of Equation (6.11) is consequently $k_a(\mathsf{d}_y)$. Outside that support both sides vanish. If a row output is positive, its first entry exceeds $1/2$ and its Kth largest first-coordinate value is below $1/1000$. Hence there are at most $K - 1$ positive entries. Boundedness follows from the clipped denominator. The routed fraction is deterministic at truth and obeys the same argument. □

For the exact all-bin identity, the same true-table formulas also have a meaning at scales with $P2^{-k} < 1$. Here $L_k = 1$, and every narrower usage box is the same singleton $\{0\}^H$. Distinct identifiers have distinct sampling matrix evaluations: writing $r = h_0 - 1$ and $\bar{v}(y) = (y, \dots, y^r)^{\mathsf{T}}$, we have

$$a(y) - a(z) = F(\bar{v}(y) - \bar{v}(z)) \neq 0 \qquad (y \neq z),$$

because F has independent columns and the first coordinate of $\bar{v}(y) - \bar{v}(z)$ is $y - z \neq 0$ in $\mathbb{F}_P$. Thus at most one entry in a row is retained. Since $K > 1$, its row cutoff is identically one. On the support of $k_a(\mathsf{d}_y)$ we also have $h(\mathsf{d}_y) = 1$, so $q_y = c(q_y) = 1$ and the bin table identity holds at these finer scales as well, including its routed form. The comparisons $\pi_a \asymp a$ are needed only on the admissible scales. For each fixed positive entry, only finitely many ω_k are nonzero; summing these true identities therefore recovers Equation (6.7) exactly. Section 8 will estimate the omitted deterministic tail without sampling it.

6.3 Corrections with an incoming cutoff

Corrections also need a bound on incoming degree. Their deterministic column-support bound permits an additional incoming cutoff while keeping the probability that both cutoffs are full bounded away from zero.

Consider $F = fQ(G/f)g_0(y)$ and put $a = f$, $\vartheta = \theta_1$. Fix $B_Q > \max\{K_G, \theta_2\}$. In the table formula, evaluate Q at $\mathrm{clip}_{[0,B_Q]}(d)$. This bounded function has a Lipschitz sth root on the real line, is zero for negative arguments, and agrees with $Q(d)$ for every true argument $d = G(x, y)/f$. The exact hierarchy continues to use the uncapped Q. Let $R = R_{\theta_1}(\rho_x)$, $h = h_{\theta_1}$, and $I = I_{\theta_1}$ be the functions in Equation (6.5), where the true row order statistic is $\rho_x = \mathrm{largest}_K\{G(x, z)/f : Y_f(z) = 1\}$. On a

true or computed raw table, form $I(x,y)$ from that table's own row entry and own order statistic. With $\chi_{1/4,1/2}$ as above, define

$$T_y = 1 - \chi_{1/4,1/2}\left(\text{largest}_{K_c}\{I(x,y) : X_u(x) = 1\}\right). \tag{6.12}$$

Thus $T_y = 1$ through order statistic $1/4$ and $T_y = 0$ from order statistic $1/2$. This order statistic is taken over the fixed retained source set before any incoming port filtering; there is no cutoff-dependent change of its index set. The true correction preparation is $h(\mathsf{d}_y)RT_y$ on Y_a. Define its conditional mean by

$$q_y = \mathbb{E}\big[h(\mathsf{d}_y)RT_y \mid X_u(x) = Y_t(y) = 1\big]. \tag{6.13}$$

The number q_y is deterministic; as before, its second raw signal carries the Y_t membership mask. Define the normalized true correction using the clipped expression everywhere:

$$\mathcal{F}(x,y) = \mathbf{1}_{\{Y_t(y)=1\}} \frac{Q(\text{clip}_{[0,B_Q]}(\mathsf{d}_y))RT_y}{c(q_y)}\, g_0(y). \tag{6.14}$$

Here $q_y = 0$ whenever $\mathsf{d}_y \le \theta_1/2$. Thus both kinds of second signal are supported above a fixed positive first-signal threshold. Choose $\zeta \le 10^{-6}\min\{\theta_1, \delta_0, 1/2\}$, decreasing it later if a fixed compression margin requires this. The second signals are then uniformly tiny on the first-signal threshold scale.

Lemma 6.2 (Correction table identity, including zero multipliers)**.** *For sufficiently large fixed K and then K_c, the true correction table satisfies*

$$f\,\mathbb{E}[\mathcal{F}(x,y) \mid X_u(x) = Y_t(y) = 1] = F(x,y).$$

This holds also when $g_0(y) = 0$, without a positive lower bound on the denominator's unclipped mean. The cutoffs and normalization use only the two role sampling matrices X, Y; g_0 is deterministic.

Proof. Write $Q_{\text{at}}(d) = Q(\text{clip}_{[0,B_Q]}(d))$ within this proof. The row count argument in Lemma 6.1, with the correction thresholds, gives $\Pr(R \ne 1 \mid \text{the two hits}) \le 1/8$. If $g_0(y) > 0$, every source with $I(x',y) > 0$ belongs to the deterministic set in Equation (6.2). Conditional on $X_u(x) = 1$, the number of sampled members of that set has expectation at most

$$1 + CuR_*/f \le 1 + CR_*.$$

The bound remains valid after conditioning on the independent terminal hit; it ignores the auxiliary row cutoff and therefore needs no additional independence between rows. If fewer than K_c such sources are sampled, the K_cth largest auxiliary score is zero and $T_y = 1$. Choose $K_c \ge 8(1 + CR_*)$ to make $\Pr(T_y \ne 1 \mid \text{the two hits}) \le 1/8$. The union bound gives $\mathbb{E}[RT_y \mid \text{the two hits}] \ge 3/4$. Since $h(\mathsf{d}_y) = 1$ on Q's support, at every position with $Q(\mathsf{d}_y) > 0$ and $g_0(y) > 0$ we have

$$q_y = \alpha(x,y) := \mathbb{E}[RT_y \mid X_u(x) = Y_t(y) = 1] \ge 3/4. \tag{6.15}$$

Thus $c(q_y) = q_y$ there. No lower bound on q_y or $\alpha(x,y)$ is required when $g_0(y) = 0$. The claimed conditional identity follows by division by this exact conditional mean. If $Q(\mathsf{d}_y) = 0$ or $g_0(y) = 0$, both sides of the identity are zero. In the latter case the pre-incoming quantity $Q_{\text{at}}(\mathsf{d}_y)R/c(q_y)$ is still a well-defined bounded comparison target, even if the unclipped mean vanishes. □

6.4 Detector scores

The unnormalized detector table is

$$\mathcal{S}_i(x,y) = \phi(G_i(x,y)/f)R_{\delta_0}(\text{largest}_K\{G_i(x,w)/f : Y_f(w) = 1\}) \quad \text{on } X_f(x) = Y_f(y) = 1. \quad (6.16)$$

Its column trial is the Jth largest such score over retained starts. By the identical cutoff choice, its conditional mean given $Y_f(y) = 1$ is exactly $U_i(y)$ from Equation (5.12). The trial is an inverse order statistic access to a row-bounded table and does not require an incoming degree bound.

6.5 Actual ports and estimated assembly

The conditional-mean identities now give the true contributions. A product estimate will query a correction at (x, z) and then a bin at (z, y). Bounded outgoing degree limits the possible midpoints; bounded incoming degree is needed when errors at the new source z are summed over old sources x. We therefore specify access relations as well as nonzero table supports. The access relations may contain entries of weight zero.

Passing through one copy projection is partitioned by its fixed source and destination symbols i, j. A task's *inherited suffix* on a sampling matrix is the string that the projections through which it was called have appended there. If the stage-$(l+1)$ copy sampling matrices have suffixes ξ_X, ξ_Y, the stage-l parent sampling matrices use $i\xi_X, j\xi_Y$. Equation (4.1) identifies their evaluations pointwise with the copy evaluations, including the source, broad terminal and narrower usage tests. Separately, Lemma 4.1(4) gives the unchanged joint law under each common suffix, so the stage-l identities and sampling estimates apply to the parent table. The task tuple records these fixed symbols and suffixes. Neither a right factor's task parameters nor a column vector's task parameters retain an earlier source address. The environment supplied to the table is common to all its entries. The vertices x, y are arguments of that table, rather than additional parameters. Thus two paths that discover the same entry in that environment query the same value; the path used to discover it is not part of its mathematical definition.

We will apply these row functions to arrays represented by a finite list of exceptional values and one common value at every omitted endpoint. Call that common value the *default*. Adjoin arbitrarily many dummy coordinates with the same default, and include them in every order statistic; they are never matrix entries or ports. For the true row arrays, dummy values are zero. Section 9 constructs the finite-list representations. The elementary order statistic convention below lets us prove support and access bounds before choosing that construction.

Remark 6.3 (Order statistics of arrays with a common default). Suppose a row array equals a common default outside a finite list, and arbitrarily many dummy coordinates have that same default. Its kth largest value can be computed from the listed coordinates and k copies of the default, with the stipulated zero padding. Indeed, every omitted coordinate merely supplies another copy of a value already available with multiplicity at least k. Additional listed real coordinates equal to the default therefore do not change the order statistic. The same observation applies to an order statistic of entry sizes when each omitted entry has the same default vector.

This is an equality of order statistics for one prescribed full array. The dummy coordinates are never matrix entries or ports. A common componentwise clamp of the entries and their default preserves this representation: an omitted position still equals the clamped default.

For the following access classes, $v = (v_y)$ is the full first raw row, including its dummy coordinates, and $\rho = \text{largest}_K(v)$. For each fixed $\vartheta > 0$, use repeatable approximations $\widetilde{v}_y, \widetilde{\rho}$ with absolute error at most $\eta = \vartheta/10000$. Define the classes of real retained endpoints by

$$A = \{y : Y_a(y) = 1, \quad \widetilde{v}_y > 3\vartheta/8, \quad \widetilde{\rho} < \vartheta/200\}. \quad (6.17)$$

For access to the auxiliary score, define

$$A_I = \{y : Y_a(y) = 1, \quad \widetilde{v}_y > \vartheta/10, \quad \widetilde{\rho} < \vartheta/40\},$$

and use the smooth function

$$J_\vartheta(v, \rho) = \chi_{82\vartheta/1000,\, 90\vartheta/1000}(v)\big(1 - \chi_{26\vartheta/1000,\, 27\vartheta/1000}(\rho)\big).$$

The scalar functions in these formulas are evaluated entrywise.

Lemma 6.4 (Support margins and exposed row classes)**.** *Suppose a bounded nonnegative main function $k(v)$ vanishes for $v \le \vartheta$. Its product $k(v)R_\vartheta(\rho)$ and the preparatory product $h_\vartheta(v)R_\vartheta(\rho)$ have at most $K-1$ nonzero real entries in a row. Their common exposed class A has at most $K-1$ real entries, contains their supports, and satisfies $I_\vartheta = 1$ throughout A. These assertions hold for arbitrary real raw estimates, without a sign or accuracy hypothesis.*

The auxiliary score I_ϑ also has at most $K-1$ nonzero real entries. Its support is contained in A_I, which has at most $K-1$ entries, and J_ϑ is a bounded-support smooth witness equal to one throughout A_I.

Proof. For the main and preparatory products a nonzero entry has $v > \vartheta/2$ and $\rho < \vartheta/500$. There cannot be K such entries, since then the Kth largest value would exceed $\vartheta/2$.

Every positive main or preparatory entry belongs to A. An entry in A satisfies $v_y > (3/8 - 1/10000)\vartheta > \vartheta/4$ and $\rho < (1/200 + 1/10000)\vartheta < \vartheta/100$, so its witness I_ϑ equals one. Its entry threshold exceeds its order statistic threshold, proving $|A| \le K-1$.

The support of I_ϑ requires $v > \vartheta/8$ and $\rho < \vartheta/50$, again giving the same row bound. The same strict threshold separation shows that A_I contains this support and has at most $K-1$ entries. The approximation margins imply $A_I \subseteq \{J_\vartheta = 1\}$. Moreover $82/1000 > 3(27/1000)$, so this witness also has bounded row support and the required entry versus order statistic separation. The witness is an analytical output.

These arguments use the order statistic of the prescribed entire raw array. In particular, if at least K dummy positions have a common virtual default β, then $\rho \ge \beta$. A real position still equal to that default cannot meet either exposed class: the required lower bound on $v = \beta$ exceeds the required upper bound on ρ. Thus omitted default positions do not create unlisted ports. □

The fixed-accuracy approximations in this lemma are part of the access interface, not tests for equality of exact reals. The implementation must use the same approximation and tie conventions whenever it queries the same table. The strict margins prove correctness for every such choice. Auxiliary-only access classes require a row bound but no incoming bound. In particular, different computed versions may expose different zero-weight entries. The assertions are applied to each version's own final raw array and repeatable approximation convention. Accuracy of the raw array is used later in the error estimates, not in this support argument.

An estimated table applies the preceding bounded row functions to its *final* raw row arrays. In a normalized task its second input is a computed value approximating $\zeta \mathbf{1}_{\{Y_t(y)=1\}} q_y$: the calculator divides that input by ζ and applies c. It does not launch another averaging operation after selecting an endpoint. Its auxiliary incoming scores are formed from its own final first-signal rows, and its incoming cutoff is Equation (6.12) for those scores. A preparatory task is an earlier, separate task; its raw arrays, scores, and cutoffs are its own, and its target is the unnormalized preparatory table specified above. Corrections additionally multiply by a clipped vector estimate of the prescribed U-cutoff $g_0(y)$, queried with a common broad Y_f-domain tuple. The unnormalized preparations hR and hRT remain on their broad Y_a domain; their usage parameter t specifies the conditional slice

on which they are averaged. The resulting second raw signal and the normalized main output carry the Y_t mask. Bins and their preparations and detector tables have no additional incoming multiplier. For example, the root of a normalized pre-incoming function is, on its fixed usage domain,

$$k(v)^{1/s}R(\rho)^{1/s}c(w/\zeta)^{-1/s}.$$

The first two factors have the root regularity already proved, and $q \mapsto q^{-1/s}$ has bounded derivative on $[1/2, 1]$. Since c is 1-Lipschitz, changing w changes the last factor by at most $C\zeta^{-1}|w - w'|$. Splitting a difference of the three-factor product one factor at a time proves a global Lipschitz bound on the saturated inputs. The same argument applies to the unnormalized preparations and the score and witness products. The constants may depend on the fixed ζ, but not on the stage, rates, or budget. Usage masks are the same in the compared tuple and multiply both sides of these inequalities.

For clarity, Table 1 records the thresholds needed by the compression argument. For a pre-incoming row function H, its entry factor vanishes through e_H, and its row cutoff vanishes from order statistic g_H onward. Every listed pair satisfies $3g_H < e_H$, including the broadest witness, for which $82\vartheta/1000 - 3(27\vartheta/1000) = \vartheta/1000 > 0$. All cutoffs are full in a neighborhood of order statistic zero. The correction preparation in this table is its pre-incoming factor hR; its incoming cutoff is assembled afterward.

Pre-incoming row function	e_H	g_H
Bin, including routed bins	$1/2$	$1/1000$
Bin preparation hR	$1/4$	$1/1000$
Correction clipped-ratio function	θ_1	$\theta_1/500$
Correction preparation hR	$\theta_1/2$	$\theta_1/500$
Detector score	δ_0	$\delta_0/500$
Auxiliary score I_ϑ	$\vartheta/8$	$\vartheta/50$
Witness J_ϑ	$82\vartheta/1000$	$27\vartheta/1000$

Table 1: Entry and row-cutoff thresholds. The entry factor vanishes through e_H, and the row cutoff vanishes from order statistic g_H onward. Here $\vartheta \in \{1/2, \theta_1, \delta_0\}$ as appropriate.

The bounded first-entry ranges and cutoff transition intervals already specified give fixed sensitivity ranges for these functions. If w is a computed second raw entry, its only use is through $c(w/\zeta)$; clipping w to $[\zeta/2, \zeta]$ leaves this quantity unchanged. Its denominator stays in $[1/2, 1]$ even for negative or inaccurate inputs. Usage masks only remove entries and do not change any of these support or continuity bounds.

For a correction or its preparation, let A be the specified pre-incoming class in Equation (6.17). For a correction, the capped relation below is selected from this uncapped row access class. Define its capped relation by retaining $(x, y) \in A$ precisely when

$$\#\{x' : X_u(x') = 1,\ (x', y) \in A\} \le K_c. \tag{6.18}$$

This condition removes every entry at an overfull column, including entries of weight zero.

Lemma 6.5 (Port contract)**.** *All main and preparatory row tables, auxiliary scores, and the smooth witnesses just constructed have bounded values and bounded row support. Every row access class used for a main or preparatory output, or for access to its auxiliary score, has bounded size and a smooth witness equal to one on every exposed entry, including exposed entries of weight zero. A witness itself is needed as an analytical row function. The capped relation in Equation* (6.18) *preserves every correction or preparatory table value and has at most* $K - 1$ *outgoing and* K_c *incoming exposed entries.*

These assertions hold separately for every computed version, provided that its values, order statistics, scores, and access classes use the same final raw arrays and common task tuple. Access ports denote distinct entries: the implementation must expose each pair at most once and pair its incoming and outgoing ports consistently. Access to auxiliary scores alone has only the asserted row bound.

Proof. The row and witness assertions are Lemma 6.4; clipping and multiplication by bounded factors do not enlarge supports. If a column has more than K_c entries in A, its auxiliary score is one at all those entries, so its K_cth largest auxiliary score is one. Equation (6.12) makes $T_y = 0$ there. Consequently every value deleted by Equation (6.18) was already zero. The surviving relation has the stated two degree bounds even when zero weights are retained. The argument applies to arbitrary raw arrays, so it applies equally to the estimated and true versions.

The cap removes every uncapped row access port at an overfull column; it does not choose K_c of them. It is applied even when the particular entry has weight zero. Every surviving port still lies in A, so its pre-incoming witness I equals one regardless of a zero usage mask, incoming cutoff, or column multiplier. The broader auxiliary class A_I is used to enumerate scores for an incoming order statistic. It is not the capped correction relation used as a left factor with a rotation map in a product.

The classes are relations on row and column IDs, rather than multisets of paths. Choosing one port per relation element gives the stated combinatorial interface. Establishing its single-address enumeration and port pairing within the space bound is an implementation obligation; the degree argument does not itself grant random access to both IDs. □

For later reference, the full interface supplied here is the following. Each task has fixed common parameters; first-coordinate order statistics include the prescribed dummy coordinates; exposed ports are real distinct entries and lie in a unit witness; sparse main and preparatory ports obey the actual incoming cap; and values, auxiliary scores, incoming order statistics, and column multipliers are all queried at the same supplied environment and tuple. The path by which an entry is discovered must not change that environment or tuple for its assembly. Only completed earlier output types, or stated fixed local functions and aggregations of those types, may be used as raw inputs of a later task. No prefix still waiting for assembly is such an output. These requirements distinguish finite-support analytical bounds from the access properties used in the subsequent implementation. Completing a raw array here means defining its entries from the allowed earlier outputs, rather than storing the whole array at once. Its auxiliary scores and uncapped row access classes are defined before its incoming cutoff and cap, so their definitions do not call that same cutoff or cap.

These contracts apply directly to each computed array after compression and clamping. The later comparison in Lemma 9.6 preserves relevant row-function values under its stated common-increment hypothesis. Each calculation uses its own consistent classes, and Lemma 6.5 gives their support and degree bounds separately.

6.6 Integrated losses and assembly bounds

It remains to control how errors in the row functions, incoming scores, and detector vectors affect an assembled table. The incoming degree bound lets us sum these errors by columns without multiplying by the number of possible source vertices.

For a real row array Z define

$$\|Z\|_{u,a;s}^{s} = \frac{1}{N}\mathbb{E}_{\sigma}\sum_{x}\frac{X_u(x)}{u}\sum_{y:Y_a(y)=1}|Z(x,y)|^{s}, \qquad \mathfrak{d}_{u,a}(H,H') = \|H^{1/s}-(H')^{1/s}\|_{u,a;s}. \tag{6.19}$$

Here σ has the product law of the sampling matrices and independent auxiliary bits specified in Section 4. All table values are extended by zero outside their membership domain. The denominator u is the nominal rate, and actual-rate comparability in Lemma 4.1 absorbs the resulting absolute constants. For a vertex array on a sampling matrix with rate u, write

$$\|v\|_{u;s}^{s} = \frac{1}{N}\mathbb{E}_{\sigma}\sum_{x}\frac{X_u(x)}{u}|v(x)|^{s}; \tag{6.20}$$

the indicated role sampling matrix replaces X when necessary. These are also the comparison norms for two sample-dependent versions. All required pre-incoming functions and smooth support witnesses are included among the row outputs whose discrepancies are to be bounded.

Bounded row support and range imply a uniform bound on these row-output norms, since $|V_l| \le N$ and $\mathbb{E}X_u/u = O(1)$. Bounded vertex arrays have the analogous uniform bound. Ordinary error is controlled by root error on bounded nonnegative outputs, since

$$|z-w| \le sC^{(s-1)/s}|z^{1/s}-w^{1/s}| \quad (0 \le z,w \le C). \tag{6.21}$$

Lemma 6.6 (Assembly comparisons)**.** *Compare two versions with the same environment, rates, and task tuple. For a sparse normalized output write $H = PTg$ and $H' = P'T'g'$, where P, P' are the pre-incoming clipped-ratio row functions (including their usage masks), I, I' are their auxiliary scores, T, T' are the incoming cutoffs of Equation* (6.12) *formed from I, I', respectively, and g, g' are their column multipliers. If g, g' are the prescribed smooth cutoffs of bounded vertex arrays U_i, U_i', then*

$$\mathfrak{d}_{u,f}(H,H')^{s} \le C\mathfrak{d}_{u,f}(P,P')^{s} + C\mathfrak{d}_{u,f}(I,I')^{s} + C\frac{f}{u}\sum_{i}\|U_i-U_i'\|_{f;s}^{s}. \tag{6.22}$$

For a sparse preparation omit the last term and use its unnormalized pre-incoming row function. The corresponding bin and detector row functions have no incoming or column-multiplier assembly cost.

If V_y, V_y' are the detector trials formed from $\mathcal{S}_i, \mathcal{S}_i'$ by their column order statistic, then

$$\|V-V'\|_{f;s}^{s} \le C\mathfrak{d}_{f,f}(\mathcal{S}_i,\mathcal{S}_i')^{s}. \tag{6.23}$$

Proof. All factors are bounded. On the union E of the two final nonzero supports, the triangle inequality for the products of sth roots gives

$$|H^{1/s}-(H')^{1/s}| \le C|P^{1/s}-(P')^{1/s}| + C|T^{1/s}-(T')^{1/s}| + C|g^{1/s}-(g')^{1/s}|.$$

Outside E the left side is zero. The pre-incoming discrepancy may be charged on its entire row domain. Each column has at most $2K_c$ edges in E, by Lemma 6.5. By Equation (6.4), Lipschitz continuity of the incoming cutoff's sth root, and Equation (6.21),

$$|T_y^{1/s}-(T_y')^{1/s}|^{s} \le C\sup_{x:X_u(x)=1}|I(x,y)^{1/s}-I'(x,y)^{1/s}|^{s}.$$

Consequently its integrated contribution is at most

$$\frac{C}{Nu}\mathbb{E}\sum_{y:Y_f(y)=1}\sum_{x:X_u(x)=1}|I(x,y)^{1/s}-I'(x,y)^{1/s}|^s,$$

which is $C\mathfrak{d}_{u,f}(I,I')^s$.

The maximum over a fixed number of U_i values is Lipschitz, and the multiplier cutoff has Lipschitz root. Thus the last discrepancy is bounded in sth power by $C\sum_i|U_i(y)-U_i'(y)|^s$. Summing over E costs at most $2K_c/u$ per retained column. The vertex metric has weight $1/f$ instead, giving precisely the factor f/u in Equation (6.22). All columns in this comparison are in the common Y_f domain, including those whose final values carry a narrower usage mask. The common-tuple condition ensures that the vertex discrepancy at y is the same quantity for every old start. Explicitly, if $m_E(y)$ counts the edges of E entering y, then for each i its contribution obeys

$$\frac{1}{Nu}\mathbb{E}\sum_{y:Y_f(y)=1}m_E(y)|U_i(y)-U_i'(y)|^s\le 2K_c\frac{f}{u}\|U_i-U_i'\|_{f;s}^s.$$

This is the column-counting step.

For a detector trial, Equation (6.4) bounds the trial discrepancy by the supremum of the input score discrepancies over retained starts. Its sth power is at most the sum of their sth powers. The factor $Y_f(y)/f$ in the vertex metric is exactly the factor obtained by reversing the two sums in the row metric with $u=a=f$. Equation (6.21) gives Equation (6.23). □

The required task types are therefore the bins $\mathcal{B}_b$ of C_l, the routed bins $\mathcal{B}_b^{(j)}$ of D_l, detector scores $\mathcal{S}_i$, and the pilot and actual correction tables $\mathcal{F}$, together with one earlier unnormalized preparation for each normalized type. Their true conditional-mean identities are Lemmas 6.1 and 6.2. Their computed versions use the same bounded row functions and the assembly contract above. The next sections bound their discrepancies through the additive schedule and implement their accesses without storing two unrestricted vertex addresses.

7 Conditional averaging in the shared environment

The table identities use conditional expectations on slices that retain specified source and terminal vertices. To approximate these expectations, we need a walk that mixes on each slice. We construct such a walk using the sampling matrices of Section 4, then express its steps by transformations that can be specified before the retained identifiers are known. Validation tests those identifiers when their vertices are visited.

The identifiers are fixed when the conditional space is defined. We condition on one retained identifier per conditioned sampling matrix; additional independent bit coordinates may be varied or frozen as specified below.

The *full environment law* already includes the independent-column restriction on each sampling matrix. A conditional slice additionally imposes the indicated retention events and fixes any designated frozen bits. Its averaging operator and L_s norms use the normalized probability law on that slice.

The only external group-theoretic inputs are property (T) of $\mathrm{SL}_H(\mathbb{Z})$ for $H\ge 3$ and relative property (T) of

$$(\mathrm{SL}_2(\mathbb{Z})\ltimes\mathbb{Z}^2,\mathbb{Z}^2).$$

We use Shalom's quantitative statements: [33, Theorem 2.6, p. 156] for the linear group and [33, Theorem 2.1, p. 152] for the relative property. We derive the affine finite-action gap and the conditional averaging walk from these inputs below.

Lemma 7.1 (A uniform affine spectral gap)**.** *Fix $H \geq 3$ and write $G_H = \mathrm{SL}_H(\mathbb{Z}) \ltimes \mathbb{Z}^H$. Let $\mathcal{S}$ consist of the elementary matrices with a single ± 1 off the diagonal, acting linearly, and the translations by $\pm e_i$, where e_i are the standard basis vectors of $\mathbb{Z}^H$. For the permutation representation ρ of any finite transitive action of G_H, with the uniform probability measure, the lazy rational walk*

$$K_H = \frac{1}{2} I + \frac{1}{2|\mathcal{S}|} \sum_{g \in \mathcal{S}} \rho(g)$$

has a spectral gap bounded below by a positive constant γ_H depending only on H.

Proof. The set $\mathcal{S}$ is a fixed finite symmetric generating set of G_H. For a unitary representation ρ and a vector ξ, put

$$\delta = \max_{g \in \mathcal{S}} \|\rho(g)\xi - \xi\|.$$

We first bound the distance from ξ to the invariant subspace.

For each coordinate pair $\{i, j\}$, embed the two-dimensional affine group on those coordinates, and let P_{ij} be the orthogonal projection onto the subspace fixed by its translations. This subspace and its orthogonal complement are invariant under that pair group, because its translation subgroup is normal. The eight generators in [33, Theorem 2.1, p. 152] belong to $\mathcal{S}$. Applying their relative Kazhdan constant $1/10$ on the orthogonal complement gives

$$\frac{1}{10} \|\xi - P_{ij}\xi\| \leq \max_{g \in \mathcal{S}_{ij}} \|\rho(g)(I - P_{ij})\xi - (I - P_{ij})\xi\| \leq \delta,$$

where $\mathcal{S}_{ij}$ is that eight-element subset. The last inequality uses the commutation of P_{ij} with the pair group and the norm-one bound for $I - P_{ij}$. The first follows by normalizing a nonzero vector in the complement: a smaller maximum would give a translation-invariant vector in that complement, contrary to its definition. This is the projection argument in the proof of [33, Corollary 2.3, p. 154].

To check commutation, let $U_i = \rho(e_i)$ and let P_i project onto $\ker(U_i - I)$. Each U_j and its inverse preserve this subspace because the translations commute; therefore P_i commutes with U_j. It then preserves both the fixed subspace of U_j and its orthogonal complement, so $P_i P_j = P_j P_i$. The product $P_i P_j$ is the projection onto the vectors fixed by both coordinate translations, hence equals P_{ij}. It follows that all P_{ij} commute. Every coordinate occurs in some pair, so their product P_{tr} projects onto the subspace fixed by all translations. With $q = \binom{H}{2}$, telescoping the product gives

$$\|\xi - P_{\mathrm{tr}}\xi\| \leq \sum_{i<j} \|\xi - P_{ij}\xi\| \leq 10q\delta.$$

Since the full translation subgroup is normal, P_{tr} commutes with every $\rho(g)$. Its range carries a representation of $G_H/\mathbb{Z}^H = \mathrm{SL}_H(\mathbb{Z})$. Let P_{inv} project onto the full G_H-invariant subspace, and let $\kappa_H > 0$ be the Kazhdan constant for the elementary generators from [33, Theorem 2.6, p. 156]; adjoining inverse generators preserves the bound. Applying that constant on the range of P_{tr} gives

$$\|P_{\mathrm{tr}}\xi - P_{\mathrm{inv}}\xi\| \leq \kappa_H^{-1}\delta, \qquad \|\xi - P_{\mathrm{inv}}\xi\| \leq C_H\delta, \quad C_H = 10q + \kappa_H^{-1}.$$

Thus the same generating set controls distance to invariants in every unitary representation, with a constant depending only on H.

Now let ρ be the permutation representation of a finite transitive action, with the uniform probability measure, and put $m = |\mathcal{S}|$. The operator K_H of the statement is self-adjoint, and its spectrum lies in $[0, 1]$. The invariant functions are exactly the constants. For f orthogonal to constants, the preceding inequality and the Dirichlet identity give

$$\langle f, (I - K_H)f\rangle = \frac{1}{4m}\sum_{g\in\mathcal{S}} \|\rho(g)f - f\|_2^2 \geq \frac{1}{4mC_H^2}\|f\|_2^2.$$

This proves the uniform spectral gap $\gamma_H = 1/(4mC_H^2)$: every eigenvalue of K_H on functions orthogonal to constants is at most $1 - \gamma_H$. □

Coordinates and actions on a conditional slice. On a sampling matrix conditioned on retention of an identifier y in a box of side L, we use the coordinates (F, w), where F is the tuple of independent nonconstant columns of the sampling matrix A and $w = Av(y) \in \{0, \dots, L-1\}^H$, identified with an element of $(\mathbb{Z}/L\mathbb{Z})^H$. The constant column of A is uniquely determined by F, w and y. As observed in the proof of Lemma 4.1, F is uniform over independent column tuples and w is independent uniform in its box. An element $(M, c) \in G_H$ acts by

$$(F, w) \longmapsto (MF,\ Mw + c), \tag{7.1}$$

reducing modulo P in the first coordinate and modulo L in the second. On a sampling matrix without a conditioning identifier we use the same description with a fixed public identifier and side $L = P$. Concatenate the designated free bits into one block of length $H\nu$, and identify it with $(\mathbb{Z}/2^\nu\mathbb{Z})^H$, on which (M, c) acts by $v \mapsto Mv + c$ modulo 2^ν; frozen bits are not transformed. The proof of Lemma 7.2 shows that each of these actions is transitive.

Lemma 7.2 (Conditional averaging)**.** *Consider the fixed product of sampling matrices, conditioned on retention of one specified identifier per conditioned sampling matrix. Include any designated set of independent uniform bits, with length padded to a multiple of H, and freeze any remaining independent bits. Let K be the operator on this conditional space that chooses one of its boundedly many components uniformly, namely a sampling matrix or the block of designated bits, applies one step of the walk K_H of Lemma 7.1 to that component through the action of G_H in Equation* (7.1)*, or the affine action on the free-bit block defined with it, and leaves the other components unchanged. Then K is a rational Markov averaging operator, its stationary distribution is the indicated uniform conditional law and, for each fixed $2 \leq s < \infty$,*

$$\|K^d - \Pi\|_{L_s \to L_s} \leq C_s 2^{-c_s d} \qquad (d \geq 0).$$

Here Π is conditional expectation over the variables being averaged. The constants are independent of the field size, retention side lengths, bit lengths, indicated identifiers, and frozen bits.

Proof. The sampling matrix action is transitive. Reduction of $\mathrm{SL}_H(\mathbb{Z})$ contains the elementary generators of $\mathrm{SL}_H(\mathbb{F}_P)$. Elementary row reduction generates $\mathrm{SL}_H(\mathbb{F}_P)$: row additions are elementary matrices, while the needed determinant-one rescalings are generated on two coordinates by

$$W(a) = \begin{pmatrix} 1 & a \\ 0 & 1 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ -a^{-1} & 1 \end{pmatrix}\begin{pmatrix} 1 & a \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & a \\ -a^{-1} & 0 \end{pmatrix}, \qquad W(a)W(-1) = \mathrm{diag}(a, a^{-1}).$$

The matrix $W(1)$ also supplies a row interchange with a sign. Every elementary matrix over $\mathbb{F}_P$ lifts to an integer elementary matrix. The finite group acts transitively on ordered tuples of independent

columns of length $r < H$: extend either column tuple to a basis and adjust a complementary column to make the change-of-basis determinant one. After choosing an integer lift M that moves the column tuple, choose c with $c \equiv w' - Mw \pmod{L}$. Translations leave the column tuple unchanged, so this reaches the desired pair (F', w') even when P and L differ. The action law holds because composition in both coordinates is the reduction of $(M, c)(M', c') = (MM', Mc' + c)$; no relation between the moduli is needed. The affine action on the free-bit torus is transitive by translations. There are a bounded number of components. Apply the lazy walk from Lemma 7.1 to a uniformly selected component. Let K_i denote the single-component operator and let J_{comp} be the number of components. The product operator is $J_{\mathrm{comp}}^{-1} \sum_i K_i$, with K_i acting as the identity on other coordinates. A tensor product of eigenvectors has eigenvalue $J_{\mathrm{comp}}^{-1} \sum_i \lambda_i$. All λ_i lie in $[0, 1]$; unless every factor is constant, at least one is at most $1 - \gamma_H$. Thus the gap is at least $\gamma_H / J_{\mathrm{comp}}$. A component consisting of one point contributes only its constant eigenvector. The number of components is bounded independently of the field and bit lengths, and every K_i preserves the product measure.

Writing its gap as $\gamma > 0$, we have $\|K^d - \Pi\|_{2\to 2} \le (1 - \gamma)^d$ and $\|K^d - \Pi\|_{\infty\to\infty} \le 2$, both with respect to the probability law of the slice. Interpolating between these two bounds with weight $2/s$ on the L_2 bound, the convexity theorem of Riesz [30, Theorems II and II′, p. 472] (the diagonal case of the Riesz–Thorin theorem, which applies to this real operator) gives

$$\|K^d - \Pi\|_{s\to s} \le 2^{1-2/s}(1 - \gamma)^{2d/s},$$

which has the required form. For Thorin's complex extension, see [34, Introduction, §0.1; Russian translation, pp. 43–45]. □

7.1 Generating paths without retaining their endpoints

The sampling matrix coordinates for Lemma 7.2 use the indicated identifier. The next lemma is the implementation form of the same averaging, and also supplies changes of variables for later error estimates.

Lemma 7.3 (Validated walk variants)**.** *A length-d word of the walk K of Lemma 7.2, that is, a sequence of d choices of a component and of a lazy step or a generator in $\mathcal{S}$, can be expanded into a bounded-alphabet family of variants described by $O(d)$ bits such that:*

1. *each fixed variant acts bijectively and measure-preservingly on the full environment before any endpoint tests;*
2. *its transformation is independent of the indicated identifiers;*
3. *for each walk word and each environment in the required conditional slice, exactly one variant passes the slice and wrap tests at the indicated identifiers, and this variant gives the conditional walk transformation;*
4. *the sum of valid variants, each with the probability of its original walk word, is exactly the conditional average.*

The tests can be performed when their indicated vertices are visited. In the admissible implementation range, field elements and identifiers have $O(B)$ bits, the total auxiliary bit string has $O(B)$ bits, and $d = O(B)$. In that range the transformations and tests use $O(B)$ shared scratch space, and the retained description of a variant uses $O(d)$ bits.

Proof. Consider a fixed affine generator (M, c) on a sampling matrix with retention side L. Enumerate the transformations

$$A' = MA + (c + Lz)e_1^{\mathsf{T}} \pmod{P}, \tag{7.2}$$

where e_1 selects the constant column and z ranges over a fixed finite box of integer vectors. That box contains all possible wraps: for $w \in \{0, \dots, L-1\}^H$, every coordinate of $Mw + c$ is bounded in absolute value by a fixed multiple of L plus a fixed constant. Since $L \geq 1$ and the generators are fixed, one bound for z works for every side length.

Each transformation in Equation (7.2) is invertible. Left multiplication preserves column independence, and the added term changes only the constant column. It therefore preserves uniform measure on the full sampling matrix. It does not refer to the indicated identifier.

At that identifier, let $w = Av(y)$ in its integer representatives. Check first that w is in the retention box, and then that $Mw + c + Lz$ lies in the same box coordinatewise, using integer arithmetic before reduction modulo P. The unique valid wrap has coordinates $z_i = -\lfloor (Mw+c)_i / L \rfloor$. For this vector the evaluation of A' is the representative of the intended affine action modulo L. For a word with steps (M_i, c_i, z_i), put

$$U_0 = I, \quad v_0 = 0, \qquad U_i = M_i U_{i-1}, \quad v_i = M_i v_{i-1} + c_i + L z_i \pmod{P}.$$

The full sampling matrix at step i is $A_i = U_i A + v_i e_1^{\mathsf{T}}$, so at every identifier $A_i v(y) = U_i A v(y) + v_i$. The integer wrap test uses the representative of $A_{i-1} v(y)$ and the current $c_i + L z_i$ before reduction. Inductively it selects exactly the conditional action modulo L at each step. This proves uniqueness for the whole word and also shows how to regenerate any intermediate test from the word without saving its conditioning identifier.

The construction applies separately to the bounded number of sampling matrices. On a sampling matrix of side P the ordinary field reduction suffices; alternatively, the same wrap validation may be used at its public point. The free-bit torus has ordinary invertible affine operations and requires no unknown endpoint. Frozen bits remain fixed. Thus a word has only constantly many generator and wrap choices per step, giving the $O(d)$-bit description.

For each word and initial environment in the slice, exactly one variant survives. Its weight must be the weight of that word. In particular, the sum is not divided by the number of guessed wraps. This proves the asserted exact identity. All matrices have fixed dimensions and $O(B)$-bit entries; evaluating a bounded-degree polynomial or a word's individual steps needs $O(B)$ arithmetic scratch. A test depending on a vertex can be delayed until that vertex is current, with the required matrix state regenerated from the recorded transformations. Theorem 12.1 accounts for reversible restoration and the simultaneous space used when these operations are nested. □

Remark 7.4 (Effective environment coordinates). Conditional averaging is expressed in the sampling matrix and bit coordinates of the environment supplied to that invocation. A fixed variant from Lemma 7.3 is a measure-preserving change of variables on the full environment; its retention and wrap tests remain explicit masks. Thus free and frozen bits always refer to the supplied coordinates, which may differ from those before the change of variables.

Remark 7.5. Two distinct uses of the variants should be kept separate. For centered-noise estimates, one first recognizes the sum of valid variants as the conditional walk and applies Lemma 7.2. For crude bounds and changes of variables, each fixed variant is a measure-preserving bijection, and one may then pay for their $2^{O(d)}$ number. Counting variants before a spectral estimate would needlessly weaken the useful contraction.

8 Additive estimation and conditional prefix bounds

The bounded random tables of Section 6 represent the exact hierarchy through their conditional means. In this section, earlier estimates are input tables and vectors with those interfaces. We define

their sampled raw prefixes and derive bounds under the corresponding child-error hypotheses. Two errors arise: the conditional mean can differ from the deterministic target, and a finite averaging walk leaves noise around that mean. We bound them separately.

These prefixes are intermediate arrays. Final row outputs result from applying the table functions to completed, compressed rows; Section 10 completes the recursive definition, including its endpoint-grouping rule.

The scale and source-domain comparisons first determine the accuracy needed from each child. We then average differences between consecutive child budgets. Their conditional means telescope, while their small absolute norms allow finite conditional walks to suppress the noise. The resulting prefix estimates are the input to the compression analysis in Section 9. That section proves an error reserve under exact endpoint grouping; Section 10 constructs the comparison calculation to which the reserve will apply.

8.1 Tasks and dependency order

Fix the last stage L and a top statistical budget $M_{\mathrm{top}} = O(B)$; Section 13 chooses both, with a fixed linear coefficient for M_{top}. The row-task types are the bins of C_l, the routed bins of D_l, the detector scores, the pilot and actual corrections, and an unnormalized preparation for each normalized type, at the stages and iterations where those objects are defined. The vector-task types are the detectors U_i and rewards W_l. An *allowed task* is one of these types with the following common parameters and domains. All rates belong to the grid $b_k = 2^{-Hk}$; at stage l write $f = b_l$.

type	start rate	end scale	usage rate
bin, including a routed bin	$u \le f$	$a \le K_f f$	$t \le a$
correction	$u \le f$	$a = f$	$t \le f$
detector score	$u = f$	$a = f$	f
preparation	same rates as its normalized task		
detector vector	$u = f$	$a = f$	none
reward vector	$u \le f$	$a = f$	none

For a vector, the start-rate column denotes its vertex rate, and $a = f$ is used only in the precision assignment below. A preparation is evaluated on the broad Y_a domain; its usage rate specifies the narrower slice for its conditional mean. The resulting second raw signal and normalized output carry the Y_t mask, as in Section 6.

The stage, input, iteration, type, rates, assignment of distinct sampling-matrix roles, and fixed suffix on each role sampling matrix are common to the whole table. Vertices are its arguments, never extra task parameters. Each suffix has length at most $L - l$. Each task has an integer budget $0 \le M \le M_{\mathrm{top}}$; m denotes a child budget. For a sufficiently large fixed C_{rate}, require every rate r in its tuple to satisfy

$$\log_2(1/r) \le HL + C_{\mathrm{rate}}(M_{\mathrm{top}} - M). \tag{8.1}$$

Choose the field after this finite range, so that it contains all padded identifiers and $P2^{-k} \ge 1$ for every rate b_k in the range. Section 13.1 gives this choice with $\log P = O(B)$. After Lemma 8.6 we check that every recursive call stays in this family. Final scores, witnesses, cutoffs and access classes are outputs assembled from a task's completed arrays; they do not introduce additional estimator types.

The final query is the reward W_L at the designated start copy with every appended digit zero, with $u = a = b_L$, empty suffixes and budget M_{top}. These parameters satisfy the displayed conditions. Theorem 10.2 is a statement about these tasks. Statistical budgets are distinct from the numerical

digit places of Section 11. The ordinary and root discrepancy norms are those of Equations (6.19) and (6.20); we use $s = 8$.

The source weight $X_u(x)/u$ has bounded expectation, and the factor $1/N$ normalizes the sum over source vertices. Endpoint sums remain explicit: bounded row supports or incoming degrees, as appropriate, will control them. A bound at one specified start consequently costs a factor N in moment power; Lemma 13.1 makes and pays this conversion at the end of the proof.

Here is a dependency order, with only a fixed number of task types at each stage. First come the bin tasks for C_l. At each pilot step come its input-score task $\mathcal{S}_i$, the detector vector U_i, and then the pilot correction task. Next come the actual correction tasks in iteration order, the routed bin tasks for D_l, and the reward update. Place the unnormalized preparation immediately before each normalized task of the same input. The first raw arrays of both these tasks refer only to earlier inputs in this order; the normalized task has in addition its preparation as the source of its second array.

For a correction task or its preparation, first complete its raw arrays and pre-incoming scores, then form its incoming cutoff from those scores. This assembly uses the current task's completed arrays; every recursive input to their raw calculation is an earlier task in the order just specified.

8.2 Raw sample terms

For a fixed row and endpoint let

$$\mathcal{C}_{xy}^{u,a} = \{X_u(x) = 1,\ Y_a(y) = 1\}.$$

We use conditional means and conditional walks on this slice. Write $\mathcal{B}_b$ for a normalized bin table and $\mathcal{B}_b^{(j)}$ for a bin with destination routing fraction already included; $\mathcal{F}$ denotes a normalized correction table. Their values are in units of the sampling scale and are zero off their specified membership domains. In a bin sum use only scales $b \le K_f f_{\mathrm{in}}$, where f_{in} is the stage rate of that bin's input. Coarser bins vanish on the true input by Section 6 and require no task. Finer scales omitted by the budget cutoff are handled by Lemma 8.5. In all formulas below put $\tau = \min(a, b)$.

The direct bin and correction–bin product terms, before division by the current first-array scale a, are respectively

$$T_b(x, y) = b\frac{\pi_a}{\pi_\tau}\mathcal{B}_b(x, y), \tag{8.2}$$

$$T_b^{\mathrm{prod}}(x, y) = b\frac{\pi_a}{\pi_\tau}\frac{f}{\pi_f}\sum_{z:Z_f(z)=1}\mathcal{F}(x, z)\mathcal{B}_b(z, y). \tag{8.3}$$

In the product, the left task has roles X, Z and rates $(u; f, f)$, listed as start, end, and usage. The right task has roles Z, Y and rates $(f; b, \tau)$. In the direct term the start rate stays u. The term $-E$ in D is the negative of Equation (8.2) with $b = f$ and $\mathcal{F}$ in place of $\mathcal{B}_b$. Summing the direct and product terms gives the raw inputs G_i, G_i'; adding the negative term gives D.

For a copy projection x_i, y_j from stage $l + 1$ to stage l, the raw term is

$$r_i(x)b\frac{\pi_a}{\pi_\tau}\mathcal{B}_b^{(j)}(x, y). \tag{8.4}$$

Partition by the fixed symbols i, j. With inherited suffixes ξ_X, ξ_Y, evaluate the parent bin using suffixes $i\xi_X, j\xi_Y$. By Equation (4.1), its source and endpoint evaluations are exactly those at x_i, y_j in the copy task, in the same environment. In particular, the outer slice and the narrower Y_τ usage test are unchanged pointwise. The parent-stage source multiplier is obtained from its detector

vectors on the broad parent-stage vertex domain. No child tuple contains the old source or a particular right endpoint as an extra parameter.

For the second raw array, use ζ times the preparatory table, take its conditional average on $\mathcal{C}_{xy}^{u,t}$, and extend the result by zero off $Y_t(y)$. For a detector vector use the trial

$$\text{largest}_J\{\mathcal{S}_i(x,y) : X_f(x) = 1\} \quad \text{on the slice } Y_f(y) = 1. \tag{8.5}$$

Order statistics include arbitrary zero padding. For rewards, writing $f = b_l$, the two terms for $W_{l+1}(x_i)$ on its inherited source slice are

$$r_i(x)W_l(x), \qquad r_i(x)\frac{f}{\pi_f}\sum_{z:Z_f(z)=1}\mathcal{F}(x,z)W_l(z). \tag{8.6}$$

The first W_l call keeps the inherited source domain; the second restarts on the midpoint domain. Vector answers are finally clipped into $[0,1]$.

Lemma 8.1 (True sample means)**.** *On the specified conditioning slices, the conditional means of the true sample terms in Equations* (8.2)–(8.6)*, summed over their bin scales, equal the prescribed matrix inputs and vector values. After division by a, the first row array has target $\mathsf{d}_y = G(x,y)/a$. The second has target $\zeta q_y \mathbf{1}_{\{Y_t(y)=1\}}$, with the prescribed zero extension outside its narrower usage domain.*

Proof. If $b \geq a$, the usage hit is already the conditioning hit. The conditional normalization of a bin therefore turns $b\mathcal{B}_b$ into its deterministic bin contribution. If $b < a$, nested origin boxes give

$$\mathbb{E}\left[\frac{\pi_a}{\pi_b}\mathbf{1}_{Y_b(y)}V \,\middle|\, \mathcal{C}_{xy}^{u,a}\right] = \mathbb{E}[V \mid \mathcal{C}_{xy}^{u,b}]$$

for every integrable V on the narrower slice. Thus the importance factor performs exactly the required change of conditioning.

For a fixed midpoint in Equation (8.3), first make this terminal change when necessary and then condition on $Z_f(z) = 1$. Apart from membership indicators, the true right bin cutoff and denominator use only sampling matrix Y. The true left correction uses only sampling matrices X, Z. These sampling matrices are independent, and the remaining true multipliers are deterministic. The two normalizations can therefore be applied separately. In particular, after the midpoint hit is fixed, the shared role Z does not occur in the right bin's remaining random value. This is an identity for the true random tables; the estimates will instead be compared by their joint error norms. The factor f/π_f cancels midpoint sampling and converts its correction table to the correction entry. Summing midpoints proves the product formula. For a projection, Equation (4.1) identifies the padded parent-table tests with the copy-task tests in the conditioning slice. Lemma 4.1(4) separately gives the usual joint sampling-matrix law for these fixed suffixes. The bin table identity therefore applies on that very slice, with deterministic $r_i(x)$ outside the expectation and the routing fraction included in the bin. This proves the projected identity on each fixed-symbol part. The all-scale sum is the exact partition in Equation (6.7); the paragraph following Lemma 6.1 supplies its true-table interpretation also at scales finer than the admissible sampling range.

The second array is its defining preparatory conditional mean, on its own narrower slice; it has no importance factor. The detector mean is the definition of U_i. True reward factors are deterministic, so the same midpoint argument proves Equation (8.6). □

To form a budget-m sample term, replace all earlier inputs in the corresponding formula by budget-m estimates, with their domains and common task parameters unchanged. This includes

the detector inputs to r_i; the factor for copy label zero $r_0 = 1$ remains constant. Products, source functions, and order statistics are evaluated before averaging. We first compare this sample's conditional mean with the same formula using true random tables. The scale and source-domain costs of that comparison will determine how much accuracy a budget must provide.

8.3 Mean comparisons and the cost of releasing a source

Lemma 8.1 identifies the exact conditional means. For estimated factors, the comparison has two distinct costs. The factor b/a converts a bin contribution to the current entry units. A right factor at a midpoint also replaces the old source domain of rate u by the stage domain of rate f. The following estimates account for both changes.

To compare a conditional mean of estimated terms with the true mean, we use the following homogeneous loss. Lemma 8.2 shows that it is comparable to the sth power of the root discrepancy, so it measures errors in the metric of Section 6, and is subadditive under sums and mixtures, so it passes through conditional expectations and midpoint sums term by term. The lemma's last bound also converts an unsigned contribution's error into the clipped relative error of the raw input, which may contain signed contributions. Lemma 8.3 applies it in this way. For nonnegative H, H' define

$$\mathcal{L}(H, H') = \begin{cases} |H - H'|^s/(H + H')^{s-1}, & H + H' > 0, \\ 0, & H = H' = 0. \end{cases} \tag{8.7}$$

Lemma 8.2 (Homogeneous loss)**.** *The loss in Equation* (8.7) *is comparable, by constants depending only on s, to $|H^{1/s} - (H')^{1/s}|^s$. For finite nonnegative families it satisfies*

$$\mathcal{L}\left(\sum_i H_i, \sum_i H_i'\right) \le \sum_i \mathcal{L}(H_i, H_i'),$$

and the same inequality holds with the sums replaced by integrals against a common nonnegative measure. If $H \le C_0 G$ for a nonnegative G, then for every $a > 0$ and $H' \ge 0$,

$$a \min\left\{1, \frac{|H - H'|}{a + G}\right\}^s \le C\mathcal{L}(H, H'), \tag{8.8}$$

where C depends only on s, C_0.

Proof. Put $v = \max(H^{1/s}, (H')^{1/s})$ and $w = \min(H^{1/s}, (H')^{1/s})$. Factoring $v^s - w^s$ bounds it between $v^{s-1}(v - w)$ and $sv^{s-1}(v - w)$; the denominator is between $v^{s(s-1)}$ and $2^{s-1}v^{s(s-1)}$. This proves comparability. For a mixture, apply Hölder to $|H - H'| = \mathcal{L}(H, H')^{1/s}(H + H')^{(s-1)/s}$ and then integrate; the triangle inequality first bounds the absolute difference of the integrals. This proves contraction, including finite sums.

If $H' \le 2H + a$, then $H + H' \le 3C_0 G + a \le C'(a + G)$, which proves Equation (8.8) directly. If $H' > 2H + a$, then $|H - H'| > H'/2$, $H + H' < 3H'/2$, and $H' > a$. Thus $\mathcal{L}(H, H') \ge cH' \ge ca$, while the left side is at most a. □

Every unsigned true term of the signed formula for D is bounded by a constant times D, by Lemma 5.4; the corresponding assertion for the other raw inputs follows from their nonnegative sums. Consequently the preceding lemma can be applied separately to all true contributions. Triangle inequalities combine their clipped relative errors even when a contribution has a negative sign.

Lemma 8.3 (Scale and domain comparison)**.** *Compare a budget-m sample term with the same term using true random inputs, with the interfaces of Section 6 and the common tuples and support bounds specified above. Index its earlier table and vector inputs by i, counting the detector vectors used to form a source multiplier as inputs, rather than treating that multiplier as an independently estimated vector. For a table input with rates (u_i, a_i) put $\mathfrak{e}_i = \mathfrak{d}_{u_i,a_i}(H_i, H_i')$; for a vector input on its prescribed rate-u_i domain put $\mathfrak{e}_i = \|V_i - V_i'\|_{u_i;s}$. Primes denote the budget-m versions and all norms use the indicated role sampling matrices.*

For a first-array unsigned term at bin scale b, let H, H' be its true and estimated conditional means on $\mathcal{C}_{xy}^{u,a}$, including its scale and importance factors but before division by a. Let G be the task's nonnegative true raw input matrix, so $\mathsf{d}_y = G(x,y)/a$. Then, with a constant C depending only on the fixed construction constants,

$$\left\| \min\left\{1, \frac{|H - H'|}{a + G}\right\} \right\|_{u,a;s}^{s} \le C\kappa_b \sum_i \frac{u_i}{u}\, \mathfrak{e}_i^s, \qquad \kappa_b = \begin{cases} b/a, & b \ge a, \\ (b/a)^{s-1}, & b < a. \end{cases} \tag{8.9}$$

The sign of a contribution does not affect this bound. A table retaining the start rate has weight one. A right factor restarted on the stage-rate f domain has weight f/u. A detector-vector input to a source multiplier on a broad vertex domain of rate f_{src} has weight f_{src}/u; in a copy projection this rate is f_{par}. Their ordinary errors control the multiplier's root discrepancy by its cutoff's Lipschitz sth root. Second-array preparations and detector trials have weight one.

For the second array let H_2, H_2' be ζ times the preparatory conditional means on $\mathcal{C}_{xy}^{u,t}$, extended by zero off Y_t. For a detector or reward vector term let h, h' be its conditional means on the prescribed vertex slice of rate u. The corresponding bounds are

$$\|H_2 - H_2'\|_{u,a;s}^s \le C \sum_i \frac{u_i}{u} \mathfrak{e}_i^s, \qquad \|h - h'\|_{u;s}^s \le C \sum_i \frac{u_i}{u} \mathfrak{e}_i^s,$$

with the inputs of the respective term in each sum; a detector vector has $u = f$ and its prescribed role sampling matrix. The second-array mask remains Y_t, so no π_a/π_t factor occurs.

Proof. First suppose $b \ge a$. Write the nonnegative true and estimated mean contributions as $H = b\mathbb{E}[V \mid \mathcal{C}_{xy}^{u,a}]$ and $H' = b\mathbb{E}[V' \mid \mathcal{C}_{xy}^{u,a}]$; in a product V includes its midpoint sum and the bounded factor f/π_f. Unsigned-term domination gives $H \le C_0 G(x,y)$. Since $1 + \mathsf{d}_y = (a + G(x,y))/a$, Lemma 8.2 gives

$$\min\left\{1, \frac{|H - H'|}{a + G(x,y)}\right\}^s \le \frac{C}{a} \mathcal{L}(H, H') \le C \frac{b}{a} \mathbb{E}[\mathcal{L}(V, V') \mid \mathcal{C}_{xy}^{u,a}].$$

The second inequality is mixture contraction and homogeneity. This is the source of the factor b/a in norm power. To separate a product, for nonnegative bounded A, B, A', B' use

$$|(AB)^{1/s} - (A'B')^{1/s}| \le B^{1/s}|A^{1/s} - (A')^{1/s}| + (A')^{1/s}|B^{1/s} - (B')^{1/s}|.$$

Each coefficient vanishes outside the support of its own factor. Mixture contraction first separates the midpoint sum, and the bounded factors then give the support-weighted sums below.

To determine the source-domain factor, write $\Delta_s\mathcal{F}$ and $\Delta_s\mathcal{B}_b$ for the root discrepancies. After integration on the source/end slice, the norm power is at most the prefactor $O(b/a)/N$ times sums of the forms

$$\mathbb{E} \sum_{x,z,y} \frac{X_u(x)}{u} Z_f(z) Y_a(y) |\Delta_s \mathcal{F}(x,z)|^s \mathbf{1}_{\mathcal{B}_b(z,y) \ne 0},$$

$$\mathbb{E} \sum_{x,z,y} \frac{X_u(x)}{u} Z_f(z) Y_a(y) \mathbf{1}_{\mathcal{F}(x,z) \ne 0} |\Delta_s \mathcal{B}_b(z,y)|^s.$$

Either support indicator can come from either compared version. In the first sum the right row cap eliminates y at constant cost. In the second sum the left incoming cap eliminates x at cost $O(1/u)$. The right child's row metric has weight $1/f$, leaving exactly $O(f/u)$. Its tuple is the same for every old source, so this elimination compares the same right error throughout. More explicitly, for either left support relation $\mathcal{P}$ with incoming degree at most K_c and a common right discrepancy $e(z,y)$, the relevant inequality is

$$\begin{aligned}
&\frac{1}{N}\mathbb{E}\sum_x \frac{X_u(x)}{u} \sum_{z:(x,z)\in\mathcal{P}} Z_f(z) \sum_{y:Y_b(y)=1} |e(z,y)|^s \\
&\quad\le \frac{K_c}{Nu}\mathbb{E}\sum_z Z_f(z) \sum_{y:Y_b(y)=1} |e(z,y)|^s \\
&\quad= K_c\frac{f}{u}\,\|e\|^s_{f,b;s},
\end{aligned}$$

where the source sampling matrix in the last norm is Z. The incoming cap applies pointwise, so the discrepancy can depend on the same environment as $\mathcal{P}$. Using $Y_a \le Y_b$ enlarges the remaining endpoint domain to the child's own domain. A direct term simply retains its source and endpoint sums. In a projected term, the source-multiplier error eliminates y by a row cap and changes the vertex weight from $1/u$ to $1/f_{\mathrm{par}}$; this costs $O(f_{\mathrm{par}}/u)$. Partitioning the fixed copy symbols keeps every tuple common during these sums.

For $b < a$, change the terminal conditioning to Y_b *before* taking a power. If $V - V'$ is the difference with the importance factor removed, the normalized mean difference is $(b/a)\mathbb{E}[V - V' \mid \mathcal{C}^{u,b}_{xy}]$. Jensen therefore bounds its sth power by $(b/a)^s\mathbb{E}[|V - V'|^s \mid \mathcal{C}^{u,b}_{xy}]$. Multiplication by the outer slice probability changes this to

$$(b/a)^s \frac{\Pr(\mathcal{C}^{u,a}_{xy})}{\Pr(\mathcal{C}^{u,b}_{xy})}\mathbb{E}[\mathbf{1}_{\mathcal{C}^{u,b}_{xy}}|V - V'|^s].$$

The two role sampling matrices are independent and their boxes nested, so this ratio is exactly π_a/π_b. There are only constantly many nonzero left paths in the union of the two rows. Ordinary bounded-factor difference inequalities and the finite-sum power inequality therefore apply with constant cost. The integrated change from Y_a to Y_b leaves π_a/π_b outside the power. Its total factor is

$$(b/a)^s\frac{\pi_a}{\pi_b} = O((b/a)^{s-1}).$$

The same row and incoming sums as above then apply, now with Y_b. Ordinary differences of bounded nonnegative values are controlled by their root differences. Absolute mean error bounds clipped relative mean error, so this proves the fine-bin assertion.

For the second raw array, Jensen is applied directly on $\mathcal{C}^{u,t}_{xy}$. Its integrated endpoint mask is Y_t, already contained in the preparatory norm's Y_a domain. There is no importance factor and hence no π_a/π_t penalty. The fixed scaling ζ does not increase the cost.

An order-statistic discrepancy is at most the supremum of its input discrepancies. For the detector trial, summing its power over Y_f/f is bounded by the score error sum over X_f/f and Y_f; the rates match exactly. For rewards, the finite left-path count gives an ordinary product comparison, and the same incoming cap eliminates old starts for a right-vector error. Source functions have bounded Lipschitz roots in their clipped detector inputs, so their costs are the broad vertex costs already computed. Every estimate here is pointwise or follows from Jensen and summation; none uses independence of its errors. □

For products of a correction with a deterministic bin contribution, we will use the following bounds.

Lemma 8.4 (Bounds for a fine product contribution)**.** *Let C be the transition matrix at the current stage, and let F be any finite pilot or actual correction there. If a matrix $A_b \geq 0$ on the same vertex set satisfies $A_b\mathbf{1} \leq \mathbf{1}$ and $A_b(x,y) \leq c_b b$ for some $b > 0$, then*

$$(FA_b)\mathbf{1} \leq k_*\mathbf{1}, \qquad (FA_b)(x,y) \leq k_* c_b b.$$

If also $A_b \leq C$, then $FA_b \leq FC$. These estimates have constants independent of the bin scale b.

Proof. Use $F\mathbf{1} \leq k_*\mathbf{1}$ from Lemma 5.2: $(FA_b)\mathbf{1} \leq F\mathbf{1}$, and $(FA_b)(x,y) \leq c_b b \sum_z F(x,z)$. The last assertion follows by multiplying $A_b \leq C$ on the left by the nonnegative matrix F. □

The preceding comparison measures errors in a retained sample term. Omitting a fine bin instead discards its whole true contribution; the next bound measures that loss.

Lemma 8.5 (Fine omission)**.** *For an omitted fine bin of scale $b < a$, the $\|\cdot\|_{u,a;s}$ norm of its true contribution divided by a is at most $C(b/a)^{(s-1)/s}$. This applies to direct terms, routed terms, and correction–bin products.*

Proof. The deterministic contribution H has entries at most Cb and bounded row mass. For a product these bounds follow from Lemma 8.4. Therefore

$$\sum_y \pi_a |H(x,y)/a|^s \leq \frac{\pi_a}{a^s}(Cb)^{s-1} \sum_y H(x,y) \leq C'(b/a)^{s-1}.$$

Integrating source hits gives $\pi_u/u = O(1)$, and summing at most N rows cancels the normalization $1/N$. □

8.4 Choosing precision to pay for scale and domain changes

The preceding bounds explain the rate weights in the accuracy parameter. Write $h_b = \log_2(b/a)$ for the offset of a bin scale. In norm, the coarse-bin cost is $(b/a)^{1/s}$ and the fine-bin cost is $(b/a)^{(s-1)/s}$. Since $s = 8$, a weight $1/2$ between these two exponents leaves a geometric margin in both directions. The same weight pays for the source-release cost $(f/u)^{1/s}$. We now make this accuracy convention precise; the additive schedule will then trade lower child budgets for longer averaging walks.

Assign positive integer ranks i_* in the task order fixed above, so every dependency decreases rank by at least one. The finite number of pilot steps and of preparations is an absolute constant, so $i_* = O(l+1)$. A task at integer budget $M \geq 0$ has implicit precision

$$p = \frac{M}{K_{\mathrm{prec}}} - \frac{1}{2}\log_2\frac{1}{a} - \frac{1}{2}\log_2\frac{1}{u} - A_* i_*. \tag{8.10}$$

For a stage-l vector use $a = f = b_l$; detector vectors also have $u = f$, whereas rewards retain their inherited start rate. The constants K_{prec} and A_* will be chosen below. Decreasing task rank by one increases precision by A_* when budget and rates are fixed. If $p \leq 0$, return the zero table or vector and empty access classes. Otherwise the task is called active. The intended induction bound is $C_b 2^{-p}$ for every required ordinary vector norm and every required root discrepancy norm, including the pre-incoming scores and witnesses. A fixed C_b makes this assertion trivial for inactive tasks: values and row supports are uniformly bounded, and $\mathbb{E}X_u/u = \pi_u/u = O(1)$.

At a normalized correction task, the column multiplier is assembled from earlier detector-vector estimates at budget $M - c_{\mathrm{col}}$, for a fixed positive integer c_{col}. This is a separate final assembly call. In choosing A_*, require also $K_{\mathrm{prec}} A_* \geq c_{\mathrm{col}}$. Since $i_* \geq 1$ and the rate costs are nonnegative,

activity then implies $M > c_{\mathrm{col}}$, so this child budget is nonnegative and smaller than M. All other calls used in one additive sample term have the common budget denoted by m.

Suppose a child at budget m has parameters a', u', i'_*. Subtracting Equation (8.10) for the parent gives the exact identity

$$p' - p = -\frac{M-m}{K_{\mathrm{prec}}} + \frac{1}{2}\log_2\frac{a'}{a} + \frac{1}{2}\log_2\frac{u'}{u} + A_*(i_* - i'_*). \tag{8.11}$$

In a bin comparison $a' \geq cb$ for a fixed $c > 0$: a bin child has $a' = b$, while a correction has $a' = f$ and $b \leq K_f f$. The start is either retained or reset to the indicated broad stage rate. Fixed-stage changes in vector calls only change fixed constants. Thus, after taking s-th roots of the costs in Lemma 8.3, the scale contribution times child accuracy is bounded by

$$CC_b\, 2^{-p-A_*+(M-m)/K_{\mathrm{prec}}} 2^{-3|h_b|/8}. \tag{8.12}$$

Indeed for $h_b \geq 0$ the exponent is $h_b/s - h_b/2 = -3h_b/8$; for $h_b < 0$ it is $(s-1)h_b/s - h_b/2 = 3h_b/8$. For a child that restarts at rate f, the source-domain comparison costs $(f/u)^{1/s}$ in norm, while its precision gains $\frac{1}{2}\log_2(f/u)$. Multiplying the cost by the resulting accuracy factor gives $(f/u)^{1/s-1/2} = (f/u)^{-3/8} \leq C$, using the allowed start-rate condition $u \leq f$.

For the separate final column-vector call, $a = f$, its child uses $a' = u' = f$, and $M - m = c_{\mathrm{col}}$. Thus its norm cost times child accuracy is at most

$$CC_b 2^{-p}\, 2^{c_{\mathrm{col}}/K_{\mathrm{prec}} - A_*} (f/u)^{-3/8}.$$

The term $A_*(i_* - i'_*) \geq A_*$ in Equation (8.11) pays this fixed budget loss. This calculation also explains why the source-domain ratio is charged in the error analysis rather than inserted as a numerical coefficient in a sample.

8.5 The additive schedule

Every budget-zero task is inactive under the preceding precision rule. Hence a budget-zero sample term is zero: it contains at least one estimated table or vector factor. Exact base terms $S(x,y)/a$ and $W_0(x)$ are inserted separately.

For every first-array term labelled by scale b, set $c_b = \lceil c_0(1 + |h_b|)\rceil$, where c_0 is a fixed positive constant. This rule includes the sparse direct term $-E$ with $b = f$, as well as direct bins, products, and projection terms. Every child factor in a term, including the detector-vector inputs to a projected source multiplier, uses that term's common budget m and cutoff c_b. Use zero scale offset only for second-array preparatory means, detector-vector trials, and reward terms with fixed stage-rate changes. The separate final column assembly retains its fixed drop c_{col}. Denote the corresponding normalized sample term at budget m by $T_{b,m}$, including division by a for the first row array. For $1 \leq m \leq M - c_b$ form

$$T_{b,m} - T_{b,m-1}. \tag{8.13}$$

Apply to this difference the conditional walk of length $\ell(M-m)$, where ℓ is a sufficiently large fixed integer. Use the same generator word and the same guessed wraps for its two parts.

Group increments by dyadic slack $d \leq M - m < 2d$. Add groups in decreasing powers of two $d \leq M$ down to 1, including empty groups. Put exact base contributions in the first group. Let S_j be the uncompressed prefix through the group with slack d_j. For a row task, superscripts $1, 2$ denote its first and optional second raw arrays. Write S_j^{mean} for the prefix obtained by replacing

each walk by uniform averaging on its slice, and put $S_j^{\mathrm{noise}} = S_j - S_j^{\mathrm{mean}}$. For an available bin the last budget reached by this mean prefix is

$$m_j(b) = \max\{0, M - \max(d_j, c_b)\}. \tag{8.14}$$

This exact cutoff will matter in the error bound. Bins with $c_b \geq M$ have no layers and are omitted.

The walk is applied to an array indexed by actual endpoints. Thus, for each term and pair, its input is the single function

$$F_{xy}(\sigma) = T_{b,m}(x, y; \sigma) - T_{b,m-1}(x, y; \sigma),$$

with all contributions at that endpoint already summed. A path description is only a way to enumerate those contributions. In particular, the two budgets must be grouped at the same actual endpoint before their difference is measured. The following lemma establishes this identity for the path enumeration and also records the description length needed later by the controller.

Lemma 8.6 (Immediate paths)**.** *At a row group of slack d, immediate candidates have descriptions of $O(d)$ bits. With exact endpoint grouping their sums equal the specified conditional-walk increments. A product visits its midpoint by a correction rotation-map evaluation before using any factor at that midpoint. Its right-hand task tuple does not depend on the discarded old source. Every child estimator in such a candidate has budget at most $M - d$.*

Proof. A descriptor records the finitely many formula and copy-part choices, its scale and budget offsets, its sign in Equation (8.13), the generator word and guessed wraps, and a bounded number of constituent ports. An included scale obeys $|h_b| = O(d)$, and the walk has length $O(d)$. Lemma 7.3 therefore bounds all these records by $O(d)$ bits. Port alphabets are fixed by Lemma 6.5. There are only $O((1+d)^2)$ choices of budgets and scales before the exponential word/variant choices.

Check each constituent port, the outer source and endpoint memberships, and the guessed wraps at the vertices where their coordinates can be evaluated. For a fixed endpoint and original generator word, unique wrap validation leaves exactly its genuine conditional transformation. Give the surviving term its original walk-word probability, multiplied by the formula coefficients and sign. Guessed wraps do not change that probability. Constituent ports enumerate entries once. Hence summing over true equal endpoints is precisely the walk of the difference, even though its endpoint was not known when the variant was specified.

The first factor of every product is a correction, whose retained ports have bounded degree in both directions. Its rotation-map evaluation supplies the midpoint and a bounded return token. The right factor can then be queried using the common Z, Y tuple prescribed above; that tuple contains no old source. A direct terminal motion without a bounded inverse port has no further factor query at its endpoint. These are combinatorial access requirements; the controller section implements the pairing and terminal operations. Finally, the slack definition gives $m \leq M - d$, also for the $m - 1$ term. □

Closure of the allowed tasks. We check the family of Section 8.1 against all the calls just specified. At a fixed stage, a direct bin keeps u and has end scale $b \leq K_f f$ and usage $\min(a, b)$. A product's correction has rates $(u; f, f)$, while its right bin has $(f; b, \min(a, b))$. The sparse direct term uses a correction with $(u; f, \min(a, f))$. These satisfy the row conditions in every case. A preparation keeps the normalized task's rates and roles; a detector trial calls its score table with $(f; f, f)$. Detector-vector calls for column multipliers use their broad rate f.

A projection from stage $l+1$ to l increases the stage rate from b_{l+1} to b_l. Its inherited start rate still obeys $u \leq b_l$; the routed child has a permitted bin scale and usage $\min(a, b)$. Its source

multiplier uses detector vectors on the broad parent-stage domain. Prepending the fixed copy symbol to each inherited suffix increases its length by one, within the new bound $L-l$. Reward calls likewise keep the inherited source rate at the parent vertex or restart at the midpoint rate b_l, as in Equation (8.6). Sampling-matrix renaming preserves distinct roles, and none of these operations adds an old source to a child tuple.

All rate operations copy a rate, reset it to a stage rate, choose a bin scale, or take a minimum, so they stay on the grid. For a bin child at budget m, availability gives $c_b \le M - m$ and hence $|\log_2(b/a)| \le (M-m)/c_0$. Thus its reciprocal-rate logarithm increases by at most $(M-m)/c_0$; the same bound holds for the minimum usage rate. Taking $C_{\mathrm{rate}} \ge 1/c_0$ makes Equation (8.1) hold for that child. Stage resets are bounded by HL; unchanged rates satisfy the larger range at a smaller budget. The budget-$m-1$ part has a still larger range. The separate column call has the positive drop checked above. All other sample calls have $0 \le m-1 < m < M$, and inactive tasks make no calls. The dependency order decreases at every estimator call. Final local assembly and prefix operations keep their task tuple, so they require no further rate or suffix case.

Consequently the top reward query and all its descendants are allowed, and every allowed task calls only allowed tasks. The bound (8.1) is $O(B)$ when $L, M_{\mathrm{top}} = O(B)$, so one field of size $2^{O(B)}$ admits the entire family, including the budget-zero endpoints of available differences. Omitted finer deterministic contributions need no hash tables.

We can now estimate each mean prefix by telescoping the conditional means to budget $m_j(b)$. Its deterministic omission error and its child-estimation error are treated separately. The remaining noise will be bounded by applying the conditional spectral estimate to the endpoint functions F_{xy} above.

Lemma 8.7 (Mean prefixes)**.** *Assume each earlier table has root discrepancy at most $C_b 2^{-p'}$ and each earlier vector has ordinary error at most $C_b 2^{-p'}$, with p' given by that child's parameters in Equation* (8.10)*. For an active task there is a fixed B_1, independent of the later choice of A_*, such that every mean prefix has norm at most $c_* 2^{-p+B_1 d_j}$ against its target. For the first array the norm uses clipped relative discrepancy; for the second and for vectors it uses ordinary absolute discrepancy. The coefficient c_* can be made arbitrarily small by increasing A_*.*

Proof. Available scales. For an available scale, conditional expectations of Equation (8.13) telescope to its term at $m = m_j(b)$ in Equation (8.14). If this budget is zero, the term is zero and the inactive-child bound still applies. More generally, the child bound remains valid whenever its precision is nonpositive, by the bounded-output estimate for inactive tasks. In every case

$$M - m \le d_j + c_0(1 + |h_b|) + 1.$$

Choose K_{prec} so that $c_0/K_{\mathrm{prec}} < 1/8$. Equation (8.12) is then bounded by

$$C' C_b \, 2^{-p-A_*+d_j/K_{\mathrm{prec}}} 2^{-|h_b|/4}.$$

The rate grid is geometric, so summing this bound over available scales costs a fixed constant. There are only a fixed number of formula and factor types. Unbinned calls use the same calculation without the scale sum. Minkowski and subadditivity of clipped relative discrepancies combine all these terms. Exact base contributions are already present in every prefix and have no error.

Wholly omitted scales. A bin with $c_b \ge M$ contributes no layer. We bound its deterministic contribution directly by Lemma 8.5. First we show that every such bin is fine. Put $r = \log_2(1/a) \ge 0$. A coarse bin has $0 \le h_b \le r$ because all grid rates are at most one. Activity gives $M > K_{\mathrm{prec}}(r/2 + A_* i_*)$, even after discarding the nonnegative start-rate cost. With $K_{\mathrm{prec}}/2 > c_0$ and A_* sufficiently

large, this implies $M > \lceil c_0(1+r) \rceil$, so a wholly omitted bin is fine. For the sparse direct term $b = f$, the same argument gives availability when $f \geq a$. When $f < a$, the admissibility condition $a \leq K_f f$ gives $|h_f| \leq \log_2 K_f$, and increasing A_* ensures $M > c_f$. Thus this term is never wholly omitted in an active task. The inequality $c_b \geq M$ implies

$$|h_b| \geq (M-1)/c_0 - 1.$$

Lemma 8.5 and a geometric sum consequently bound all such omissions by $C2^{-(s-1)M/(sc_0)}$. Since $p \leq M/K_{\mathrm{prec}}$ and $K_{\mathrm{prec}} > sc_0/(s-1)$, this is at most $C2^{-p-\eta M}$ for some fixed $\eta > 0$, after absorbing the fixed rounding offset. Finally $M > K_{\mathrm{prec}} A_* i_* \geq K_{\mathrm{prec}} A_*$ makes its coefficient arbitrarily small as A_* increases. Taking, for example, any fixed $B_1 > 1/K_{\mathrm{prec}}$ proves the claim, including every last-budget and no-budget cutoff loss. □

8.6 Absolute difference bounds and averaging noise

The conditional-mean errors are now bounded. To control the noise left by finite averaging, we need an absolute norm bound on each paired difference before the walk is applied. This coarser bound will also be used for the fingerprint comparison.

Lemma 8.8 (Absolute differences)**.** *Under the earlier-task error hypotheses of Lemma 8.7, the ordinary raw norm of an unaveraged paired difference in a slack-d group is at most*

$$CC_b\, 2^{-p-A_*+C_0 d}, \tag{8.15}$$

where C_0 is fixed independently of the walk length coefficient. For two coupled versions of a budget-r sample term, if the corresponding child discrepancies are bounded by $2^{-p'_r}\lambda_r$, their normalized raw difference is bounded by $2^{-p+C'_0 d}\lambda_r$, summed over the finitely many child types, for a fixed C'_0 that is also independent of the walk length coefficient. For a paired layer at budgets m and $m-1$, apply this estimate to both parts. Its coupled discrepancy is consequently bounded by $2^{-p+C'_0 d}(\lambda_m + \lambda_{m-1})$, after enlarging the fixed constant, or by the corresponding maximum over all children in the group. Every child in either part has budget at most $M-d$. For a fixed guessed measure-preserving variant and its valid endpoint restriction the same bounds hold. Counting all variants separately may increase C'_0 by a constant depending on the chosen walk length, but not on B or on the block constants D_1, D_2, ϵ of Section 10.

Proof. Subtract the *same true random-table term* from the budget-m and budget-$(m-1)$ terms, and use the triangle inequality. A product has only constantly many left paths in the union of the compared rows. Ordinary bounded-factor differences and the row and incoming caps therefore give the sums used in Lemma 8.3, without any averaging. For a coarse bin the absolute coefficient costs b/a in the norm rather than its s-th root. For a fine bin the importance factor can stay inside the power. Both are bounded by $2^{O(|h_b|)}$. Equation (8.11) pays the only separate possibly large domain ratio, f/u, as before. All other scale and accuracy changes are $2^{O(d)}$ because $|h_b| = O(d)$ and $M - m = O(d)$ in this group. Order statistics and bounded source functions obey the same pointwise comparisons. This proves Equation (8.15). Replacing comparison to truth by a comparison of the two coupled child versions gives the second assertion by the identical inequalities.

For a fixed guessed variant, valid paths require the transformed memberships. Change variables by its full-environment measure-preserving bijection and then discard surplus validity guards. This leaves exactly the child-domain sums just bounded. When combining the paired terms, combine their contributions at the same actual endpoint before measuring the difference. The number of

distinct variant descriptors is $2^{O(d)}$, so treating them separately merely enlarges the crude exponent. The estimate before such separate counting is a statement about the single-seed sample function and has no dependence on the eventual walk length coefficient. □

Lemma 8.9 (Conditional prefix bounds)**.** *Assume exact local grouping, the earlier task error bounds, and that the sample functions depend only on variables varied by the conditional walk. Their uniform slice means are then deterministic at each indicated pair or vertex. Write S_j^{mean} for a mean prefix and $S_j^{\mathrm{noise}} = S_j - S_j^{\mathrm{mean}}$ for its noise. In Equation* (8.16)*, $\mathsf{d} = (\mathsf{d}_y)$ is the first-array target $\mathsf{d}_y = G(x,y)/a$ of Lemma 8.1, while d_j is the slack of group j. There are fixed B_1 and arbitrarily large prescribed K_n such that*

$$\left\| \min\left(1, \frac{|S_j^{1,\mathrm{mean}} - \mathsf{d}|}{1+\mathsf{d}}\right) \right\|_{u,a;s} \le c_* 2^{-p+B_1 d_j}, \tag{8.16}$$

$$\|S_j^{\mathrm{noise}}\|_{u,a;s} \le c_* 2^{-p-K_n d_j}. \tag{8.17}$$

The second-array mean uses its ordinary absolute norm, extended by zero off its usage domain, and satisfies the first bound. Vector means and noises satisfy the corresponding vertex bounds. The coefficient c_* can be chosen as small as needed by increasing the rank coefficient A_* of Equation (8.10) *and the walk-length coefficient ℓ.*

Proof. The mean assertion is Lemma 8.7. For a difference layer, write F_{xy} for its sample function on $\mathcal{C}_{xy}^{u,a}$, $\mathcal{A}_{xy}$ for the conditional walk operator of length $\ell(M-m)$, and Π_{xy} for uniform averaging on that slice. Put $w_{xy} = \Pr(\mathcal{C}_{xy}^{u,a})/(Nu)$ and $r_{M-m} = C2^{-c\ell(M-m)}$. Lemma 7.2 gives a uniform operator bound, so

$$\sum_{x,y} w_{xy} \|(\mathcal{A}_{xy} - \Pi_{xy})F_{xy}\|_{L_s}^s \le r_{M-m}^s \sum_{x,y} w_{xy} \|F_{xy}\|_{L_s}^s.$$

The slice weights make the sum on the right precisely the integrated raw norm to the sth power, bounded by Lemma 8.8. Thus the uniform operator estimate applies directly to the integrated input bound, even when different slices have different input norms. For the second array replace Y_a by Y_t in the slice and its weight; the mask stays Y_t throughout. Vectors use the same calculation on their one-vertex slices. No extra conditioning factor appears.

There are $O((1+d)^2)$ budget/scale terms in a group. Choose ℓ so that the contraction dominates $2^{C_0 d}$ and this polynomial, leaving at least $2^{-(K_n+1)d}$. Summing groups with slack at least d_j gives Equation (8.17). Exact base terms have no noise. This contraction applies to the paired difference as one function of the same seed. The sum of valid variants is the conditional walk, so the gap applies to that sum before any separate variant counting. Increasing A_* supplies an arbitrarily small coefficient in both this estimate and the mean bound. □

The determinism hypothesis in Lemma 8.9 applies at each fixed pair or vertex, in the task's effective environment as in Remark 7.4. Uniform averaging must integrate every coordinate on which its sample function depends. In Section 10, the comparison calculation removes dependence on the current block's seed, while the parent walk varies all earlier seeds used by lower-block calculations; Lemma 10.4 verifies this hypothesis. The resulting first-array mean is a deterministic number, and the second-array mean has only its prescribed Y_t membership mask. These are the means used in the conditional heavy-count argument of Lemma 9.4.

9 Compression of additive row estimates

The additive schedule produces a raw prefix on all retained endpoints after each slack group. We replace each array by a finite list of exceptions to a common default. Our objective is to control the bounded row functions, including their scores and support witnesses; a large raw entry need not have a small absolute error.

The update changes only small entries until its first overflow, when list capacity forces its threshold above the prescribed floor. Before then, the total earlier change is much smaller than the next floor. At the first overflow, either many retained endpoints are truly heavy or the prefix errors account for the excess. The heavy-count event is rare even after conditioning on one specified endpoint. We use this rarity for the deterministic conditional-mean errors, and use the absolute norm bound for the remaining noise. A comparison with a reference receiving no further increments controls the subsequent updates.

We first fix the increments supplied by exact endpoint grouping and prove the comparison without range clamps. We then show that clamping preserves the relevant row functions and assemble the exact-grouping error reserve. Section 10 compares the actual and comparison calculations with those clamps in place.

9.1 The row interface

Fix a row task, its start rate u, and its first-array endpoint rate a. Write $\mathsf{d}_y \geq 0$ for its true first raw input, in units of a; d without a subscript denotes the whole array (d_y). The group slacks $d_1, \ldots, d_{m_*}$ introduced below, written d_i, d_j, carry group indices. The task's second raw input, when present, is $v_y = \zeta \mathbf{1}_{\{Y_t(y)=1\}} q_y$. Here q_y is the deterministic conditional mean, $0 \leq q_y \leq 1$, and $q_y = 0$ whenever d_y is below the positive-start threshold of the preparatory function h. Thus d_y and q_y are deterministic at each specified endpoint; v_y carries the prescribed narrower membership mask. There is a fixed constant C_{mass} such that

$$a \sum_y \mathsf{d}_y \leq C_{\mathrm{mass}}. \tag{9.1}$$

The endpoint sets in this section are the retained sets, with the stipulated common padding on their hash arguments.

We apply a fixed finite family of pre-incoming row functions to a raw array. Denote this family by $\mathcal{H}$. It includes the main pre-functions, auxiliary scores, and witnesses required by the preceding construction. Each function uses the first entry, possibly the second entry, and the Kth largest first entry. For each member $H \in \mathcal{H}$ there are fixed positive thresholds g_H, e_H, with $3g_H < e_H$, such that its row cutoff is zero when that order statistic is at least g_H, and its entry factor is zero when the first entry is at most e_H. The cutoff is constant and full in a neighborhood of zero. The sth roots of the output functions are bounded and Lipschitz in their relevant entry and order statistic arguments, where $s = 8$. All these arguments can be clipped to fixed sensitivity ranges without changing the functions. These are exactly the threshold buffers and saturations imposed when constructing the random tables; the thresholds for the required functions are listed in Table 1. In particular, for every real raw array, including arrays with negative entries,

$$\sum_{H \in \mathcal{H}} \#\{y : H(y) \neq 0\} \leq C_{\mathrm{supp}}, \tag{9.2}$$

where C_{supp} is fixed. Indeed a nonzero cutoff implies that fewer than K entries exceed its close threshold, and every positive output entry exceeds the larger entry threshold. This observation applies also to the broader witnesses.

Choose $\eta > 0$ below all positive entry thresholds, all positive-start thresholds for order statistic sensitivity, and the threshold for h, with enough fixed separation that perturbations of size several times η remain inside these flat regions. Choose $\zeta \le \eta/100$. These constants are fixed throughout the proof.

Let $d_1 > d_2 > \cdots > d_{m_*} = 1$ be the decreasing powers of two indexing slack groups. Empty groups are included. Let $S_i = (S_i^1, S_i^2)$ be the uncompressed prefix through group i, and let $\Delta_i = S_i - S_{i-1}$, with $S_0 = 0$. Define the clipped endpoint errors and their row supremum by

$$\begin{aligned} e_i(y) &= \max\left\{\min\left(1, \frac{|S_i^1(y) - \mathsf{d}_y|}{1 + \mathsf{d}_y}\right), \min(1, |S_i^2(y) - v_y|)\right\}, \\ e_i &= \sup_y e_i(y). \end{aligned} \tag{9.3}$$

omitting array two when absent. Write $\beta_i(y)$ for the maximum of the mean discrepancies, with the same clipping and first-array normalization as in Equation (9.3), and $\gamma_i(y)$ for the maximum absolute prefix noise. Thus $e_i(y) \le \beta_i(y) + \gamma_i(y)$. At a specified row and endpoint, the first-array mean discrepancy is deterministic. The second-array mean discrepancy is a deterministic magnitude times its narrower membership mask. Prefix noise may depend on the shared environment. The deterministic assertion is a hypothesis of the compression argument: the slice average defining a mean must vary every environment coordinate on which its sample function depends. Lemma 8.9 states this requirement, and Lemma 10.4 verifies it for the comparison calculations to which we will apply the result. We will use conditional rarity to control these fixed mean-error magnitudes; the noise will be controlled by its unconditional norm bound.

For row integration use

$$\int F \, d\mu = \frac{1}{N} \mathbb{E} \sum_x \frac{X_u(x)}{u} F(x).$$

Endpoint integration means inserting the sum over retained endpoints in this expression. The prefix estimates used as hypotheses below are

$$\left(\int \sum_y \beta_i(y)^s \, d\mu\right)^{1/s} \le c_* 2^{-p + B_1 d_i}, \qquad \left(\int \sum_y \gamma_i(y)^s \, d\mu\right)^{1/s} \le c_* 2^{-p - K_n d_i}. \tag{9.4}$$

Lemma 8.9 supplies these estimates for the additive schedule. Here and below endpoint sums use the relevant masks. In particular these bounds control the $L_s(\mu)$ norms of the corresponding row suprema, which we denote by β_i and γ_i.

These hypotheses will give the active-task estimate in Proposition 9.7. More precisely, Proposition 9.5 bounds the compressed pre-incoming root discrepancies by $C_{\mathrm{buf}} c_* 2^{-p}$, and Lemma 9.6 preserves those row-function values. Lemma 6.6 then bounds the incoming-cutoff error by the same pre-incoming discrepancies. Its column-vector term has factor f/u in moment power, which the precision gain in Equation (8.11) pays. Choosing c_* small and the rank coefficient A_* of Equation (8.10) large leaves the bound $(C_b/2)2^{-p}$. The rest of this section proves the compression and clamping inputs to that deduction.

9.2 The update map

Adjoin arbitrarily many dummy positions, all with target and increments zero. A compressed array has default $(z, 0)$, shared also by the dummies, and a finite list of exceptions. Initially $z = 0$ and the list is empty. Order statistics of an array with this default mean the order statistics of its finite exceptional values with as many default values as needed. For an order statistic of order r, the finite list together with at least r default copies gives exactly the prescribed order statistic. Additional real list positions whose values equal the default do not change this order statistic. Dummy positions serve only this order statistic computation; endpoint sums in the error bounds are over real retained IDs. The margin statement of Lemma 6.4 ensures that a default position cannot expose a row port. At group j add Δ_j by actual endpoint equality, obtaining w_y, and set

$$\begin{aligned} \rho_j &= 2^{-c_1 d_j}, \qquad n_j = 2^{c_2 d_j}, \qquad m_y = \max_{\nu} |w_y^{\nu}|, \\ u_0 &= \max\{z, \rho_j, \mathrm{largest}_{n_j}(m)\}, \\ q_0 &= \max\{0, \mathrm{largest}_K(w^1)\}, \\ z' &= \max\{u_0, \min(q_0, 3u_0)\}. \end{aligned} \tag{9.5}$$

Here c_1, c_2 are fixed sufficiently large positive integers, and c_2 is chosen also so that $n_j \geq K$ in every group. With $b = (z', 0)$, the updated array is

$$T_j^{\nu}(y) = b^{\nu} + \mathrm{clip}(w_y^{\nu} - b^{\nu}, \pm 7(m_y - 2u_0)_+). \tag{9.6}$$

The notation $\mathrm{clip}(r, \pm A) = \max(-A, \min(r, A))$ is used.

The floor ρ_j fixes the scale of changes before the first overflow. Since $u_0 \geq \rho_j$, every position of size at most $2\rho_j$ is merged into the default. Including z in u_0 prevents the default from decreasing, while the n_jth largest size leaves fewer than n_j positions above u_0 and therefore controls the number of exceptions. The threshold u_0 exceeds the floor only if the old default already does or at least n_j positions have size above ρ_j, the overflow case treated in Lemma 9.3 and Proposition 9.5. The new default z' lies in $[u_0, 3u_0]$, and it equals $\min(q_0, 3u_0) \leq q_0$ whenever $q_0 \geq u_0$. In that case positions with first entry at least q_0 keep first entry at least q_0, and the others stay at most q_0, so the Kth largest first-coordinate value, and with it every row cutoff, is unchanged. Lemma 9.2 uses this choice to preserve the relevant row outputs in its reference calculation with no further increments. Positions of size at most $2u_0$ are replaced by the new default, positions of size at least $3u_0$ are unchanged, and the clipping radius $7(m_y - 2u_0)_+$ interpolates between these two regions. Above size $3u_0$, its factor 7 allows the entire displacement from the new default, whose first coordinate is at most $3u_0$. This makes the update local and uniformly Lipschitz (Lemma 9.1).

Lemma 9.1 (Local compression properties)**.** *The update has the following properties. It replaces every position of size at most $2u_0$ by the new default, is identity at every position of size at least $3u_0$, and lies coordinatewise between the input and the new default. The default is nondecreasing. The map from the input array and old default to the updated array and new default is uniformly Lipschitz in the sup norm. A list containing all exceptions can be chosen with at most n_j positions, using only the previous list and the new candidates.*

Proof. The zero-radius assertion is immediate. Since $z' \leq 3u_0$, if $m_y \geq 3u_0$ then

$$|w_y^1 - z'| \leq m_y + 3u_0 \leq 7(m_y - 2u_0), \qquad |w_y^2| \leq 7(m_y - 2u_0),$$

which proves identity there. Clipping a displacement toward zero proves the coordinatewise assertion. Also $z' \geq u_0 \geq z$.

If two input arrays and their defaults differ by at most δ, their size arrays, size order statistics, u_0's, and q_0's differ by at most δ. Their new defaults differ by at most 3δ, and their clipping radii by at most 21δ. The elementary bound

$$|\operatorname{clip}(r, \pm A) - \operatorname{clip}(r', \pm A')| \le |r - r'| + |A - A'|$$

therefore gives, for example, a Lipschitz constant 28 for the full update. This constant is independent of the number and magnitudes of entries.

Fewer than n_j positions have size strictly larger than u_0. A coarse test supported above u_0 and full above $2u_0$ retains all possible exceptions within the stated cap. A position outside the old list and new candidate set has input $(z, 0)$, has size at most u_0, and hence becomes the new default. No other position need be inspected. The selected list may retain some real positions that become exactly default after compression. Such positions still represent the same full array. No padding dummy is selected, since its input size is at most u_0. □

In particular an absolute perturbation introduced at group i propagates through the remaining updates and the final root functions with factor at most

$$CC_1^{m_*-i+1} \le C(d_i + 1)^{\log_2 C_1}, \tag{9.7}$$

for a fixed $C_1 > 1$, enlarged below as needed. This statement concerns full arrays and defaults; it does not require their exceptional lists to agree.

9.3 Deterministic comparison arguments

Call group j an overflow if its u_0 exceeds ρ_j. Before the first overflow every compression change has magnitude at most $6\rho_i$ and occurs only at input size below $3\rho_i$. Because successive slacks halve, choosing c_1 large gives, uniformly in j,

$$6\sum_{i<j} \rho_i \le \rho_j/100, \qquad 6\sum_i \rho_i \le \eta/100. \tag{9.8}$$

The same estimates cover dummy defaults. For example, before the first overflow the preceding default is at most $3\rho_{j-1} \ll \rho_j$.

Lemma 9.2 (Preceding compression error and reference with no further increments)**.** *Let $L_{j-1} = T_{j-1} - S_{j-1}$ before the first overflow at j. Then $\|L_{j-1}\|_\infty \le 6\sum_{i<j} \rho_i$, and at every endpoint with $\mathsf{d}_y \ge \eta$,*

$$\max_\nu |L_{j-1}^\nu(y)| \le C\sum_{i<j} \rho_i e_i(y). \tag{9.9}$$

Start a reference calculation at the input of group j with the true array $(\mathsf{d}, v) + L_{j-1}$, the same preceding default, and no further increments. If q is the Kth largest first-coordinate value of the initial reference array, then at every group $i \ge j$ the reference Kth largest first-coordinate value, n_ith largest size, threshold u_0 and new default are at most $\max\{q, \eta\}$. Every subsequent compression preserves its relevant row outputs. Their root discrepancy from the true outputs is bounded by $C\sum_{i<j} \rho_i e_i$.

Proof. The uniform preceding compression error bound follows by adding the changes described above. If a change occurs at group i at a position with $\mathsf{d}_y \ge \eta$, then $|w_y^1| \le 3\rho_i$ and the preceding preceding compression error is much smaller than ρ_i. Thus

$$\frac{|S_i^1(y) - \mathsf{d}_y|}{1 + \mathsf{d}_y} \ge \frac{\mathsf{d}_y - 3\rho_i - 6\sum_{k<i} \rho_k}{1 + \mathsf{d}_y} \ge c_\eta > 0.$$

The change of size at most $6\rho_i$ is consequently bounded by $C\rho_i e_i(y)$. Adding such changes proves Equation (9.9).

In the reference array negative first magnitudes and all second magnitudes are at most η, by the choices of ζ and the floors. Let q be its initial Kth largest first-coordinate value. The default is nonnegative, so $q \geq 0$. If $q > \eta$, then $n_i \geq K$ implies that the n_ith largest size is at most q. The floor and preceding default are also at most q, so $u_0 \leq q$ and $z' \leq q$. Entries at most q stay at most q. Entries at least q are unchanged if $q \geq 3u_0$; otherwise $z' = q$, and they stay at least q. The Kth largest value is therefore unchanged. Negative first magnitudes and second magnitudes do not grow, so this argument repeats at every subsequent group. If instead $q \leq \eta$, the size order statistic, u_0, and new default are at most η, and the next Kth largest value is again at most η. By induction over the groups, in both cases every reference Kth largest first-coordinate value, n_ith largest size, threshold u_0 and default is at most $\max\{q, \eta\}$, which proves the bound on order statistics and thresholds asserted in the lemma.

Consider an individual row function. In the first case its order statistic stays fixed. If its cutoff is not already zero, $q < g_H$, so every entry in its support has first coordinate greater than $e_H > 3q \geq 3u_0$ and both arrays there are untouched. A position below entry support cannot cross into support because the default is at most $q < e_H$. In the second case the order statistic stays in the common full-cutoff region; the same identity argument applies to every position in entry support. Thus each reference output is preserved.

Finally, positions with $\mathsf{d}_y < \eta$ stay below every entry threshold and cannot change an order statistic within its sensitivity region. Perturbations at all other positions satisfy Equation (9.9). Clip first entries and order statistics to the fixed sensitivity ranges and use Lipschitz continuity of the row functions' sth roots. This proves the last assertion. □

If no overflow occurs, then

$$\sup_{H,y} |H(T_{m_*}; y)^{1/s} - H(\mathsf{d}, v; y)^{1/s}| \leq C \left(e_{m_*} + \sum_i \rho_i e_i \right). \tag{9.10}$$

Proof. Put $L_i = T_i - S_i$, with $L_0 = 0$. The input at group i is $w = S_i + L_{i-1}$, so $L_i - L_{i-1} = T_i - w$. Since $u_0 = \rho_i$, Lemma 9.1 bounds this change by $6\rho_i$ and makes it zero at every position of size at least $3\rho_i$. Consequently $\|L_i\|_\infty \leq 6 \sum_{k \leq i} \rho_k \leq \eta/100$ by Equation (9.8), including at dummy positions. If a position with $\mathsf{d}_y \geq \eta$ changes at group i, then $|S_i^1(y)| \leq 3\rho_i + 6 \sum_{k<i} \rho_k$, and Equation (9.3) gives

$$e_i(y) \geq \frac{\mathsf{d}_y - 3\rho_i - 6 \sum_{k<i} \rho_k}{1 + \mathsf{d}_y} \geq \frac{\eta}{2(1+\eta)}.$$

Summing the changes therefore gives, for $\mathsf{d}_y \geq \eta$,

$$\max_\nu |L_{m_*}^\nu(y)| \leq C \sum_i \rho_i e_i(y).$$

Write $E = e_{m_*} + \sum_i \rho_i e_i$. Bounded output ranges prove the claim when E exceeds a sufficiently small fixed constant. Otherwise $e_{m_*} < 1$, so Equation (9.3) and the preceding bound give

$$|T_{m_*}^1(y) - \mathsf{d}_y| \leq (1 + \mathsf{d}_y) e_{m_*} + C \sum_i \rho_i e_i, \qquad |T_{m_*}^2(y) - v_y| \leq CE \quad (\mathsf{d}_y \geq \eta).$$

For $\mathsf{d}_y < \eta$, the uniform bound on L_{m_*} instead gives $T_{m_*}^1(y) \leq \eta + (1+\eta) e_{m_*} + \eta/100 < 2\eta$. Both outputs at such a position vanish by the choice of η.

Fix $B \geq 1$ above all first-entry and order statistic sensitivity ranges, and let $\pi = \mathrm{clip}_{[2\eta,B]}$. The choice of η and the fixed saturations in Section 9.1 ensure that applying π to either argument leaves every row function unchanged. At positions with $\eta \leq \mathsf{d}_y \leq 2B+1$, the preceding first-entry bound and the 1-Lipschitz property of clipping give $|\pi(T^1_{m_*}(y)) - \pi(\mathsf{d}_y)| \leq CE$. If $\mathsf{d}_y > 2B+1$, that bound gives $T^1_{m_*}(y) > (\mathsf{d}_y - 1)/2 > B$ for sufficiently small E, so both clipped values equal B. At low-target positions both clipped values equal 2η; this also holds at dummies, whose true value is zero and whose final default is at most $\eta/100$.

Let $q = \mathrm{largest}_K(\mathsf{d})$ and $\widetilde{q} = \mathrm{largest}_K(T^1_{m_*})$ on the common padded index set. Monotone clipping commutes with taking an order statistic, so Equation (6.4) gives

$$|\pi(\widetilde{q}) - \pi(q)| = |\,\mathrm{largest}_K(\pi(T^1_{m_*})) - \mathrm{largest}_K(\pi(\mathsf{d}))| \leq \sup_y |\pi(T^1_{m_*}(y)) - \pi(\mathsf{d}_y)| \leq CE.$$

Thus the order statistic arguments of all row cutoffs differ by at most CE after clipping to their sensitivity ranges. Combining this bound with the entrywise bounds and the Lipschitz sth roots from Section 9.1 proves Equation (9.10). □

Lemma 9.3 (Stability after the first overflow)**.** *At a first overflow in group j, the discrepancy of final row-root outputs from the reference with no further increments of Lemma 9.2 is at most*

$$C \sum_{i \geq j} C_1^{m_* - i + 1} e_i. \tag{9.11}$$

The constant does not depend on the largest true first entry.

Proof. Write E for the sum in Equation (9.11) without the initial constant. If E exceeds a sufficiently small fixed constant, bounded output ranges suffice. Otherwise all clipped errors involved are strictly below 1. Hence

$$|S^1_i(y) - \mathsf{d}_y| \leq (1 + \mathsf{d}_y) e_i, \qquad |S^2_i(y) - v_y| \leq e_i,$$

and later increments have first magnitude at most $(1 + \mathsf{d}_y)(e_i + e_{i-1})$ and second magnitude at most $e_i + e_{i-1}$. The input difference at group j is precisely $S_j - (\mathsf{d}, v)$; both calculations include the same previous preceding compression error. The factors $1 + \mathsf{d}_y$ prevent a direct absolute-error bound with a uniform constant when a true entry is large. The following two cases use the row cutoff and saturation to remove that dependence.

Choose G larger than every cutoff close threshold. A default at least G closes all cutoffs, since the dummy entries force every first-coordinate order statistic to be at least the default. Closure is permanent under the unclamped update. Choose L_0 much larger than G, all sensitivity thresholds, and 1. If at least K true first entries exceed L_0, the reference cutoffs are closed. In the perturbed calculation either some $u_0 \geq G$, which forces permanent closure, or every $u_0 < G$. In the latter case those K coordinates stay larger than half their true value: their initial error and the total later additive variation are at most $C(1 + \mathsf{d}_y) \sum_{i \geq j} e_i$, and their first value is then larger than $3G$. Inductively each such position lies in the compression identity region. These K positive entries close all cutoffs as well.

It remains to consider the case of fewer than K true entries above L_0. The reference calculation starts from $(\mathsf{d}, v) + L_{j-1}$, where $\|L_{j-1}\|_\infty \leq \eta/100$ by Lemma 9.2 and Equation (9.8); its dummy coordinates carry the preceding default, which is below η because no earlier group overflows. Since fewer than K true first entries exceed L_0, its initial Kth largest first-coordinate value is at most $L_0 + \eta/100$. Lemma 9.2 therefore bounds every reference Kth largest first-coordinate value, n_ith largest size, threshold u_0 and default by $\max\{L_0 + \eta/100, \eta\} \leq 2L_0$. Fix L_1 much larger than L_0

and all upper saturation thresholds. At the at most $K-1$ positions where $\mathsf{d}_y > L_1$, replace the first input at group j, in both calculations, by one common positive sentinel larger than L_1, and give that coordinate zero later first-array increments. Do not modify second arrays. The reference order statistics, thresholds, and outputs are unchanged: these coordinates stay above the relevant order statistics and in all upper saturation regions. The sentinels are comparison devices in this proof. Fewer than K such coordinates occur, so replacing them by common large values removes their possibly large absolute differences while preserving every order statistic that lies in a sensitivity range.

For a high coordinate $\mathsf{d}_y > L_1 \geq 1$, the preceding variation bound is at most $2C\mathsf{d}_y \sum_{i \geq j} e_i$. Choosing the small-error case so that this sum is below $1/(4C)$ keeps that coordinate above $\mathsf{d}_y/2$. This choice is uniform in d_y; bounded outputs already cover the complementary case. All remaining first-coordinate input errors and increments now have absolute bounds by $C(1+L_1)e_i$ and $C(1+L_1)(e_i+e_{i-1})$, respectively. Lemma 9.1, iterated from j to the last group, therefore bounds every array and threshold discrepancy by CE, on taking C_1 larger than the fixed update constant. If E is sufficiently small, all these thresholds remain below $2L_0+1$. The final root-output discrepancy is at most CE.

To remove the modification in the perturbed calculation, argue inductively. Every original coordinate with $\mathsf{d}_y > L_1$ remains above $\mathsf{d}_y/2$, because its total raw additive variation is at most $C(1+\mathsf{d}_y)\sum e_i$. It is thus above the bounded thresholds and in the identity and saturation regions. Replacing fewer than K such positive coordinates by sentinels cannot change either a Kth largest first-coordinate value or an n_ith largest size whose sentinel-computed value is at most $2L_0+1$: both the original and modified values are above that bound, and $n_i \geq K$. The defaults and thresholds therefore agree, the compression is identity in both arrays at these positions, and all other coordinates agree. The relevant output behavior is the same. This completes the induction and the proof. □

9.4 Charging overflow events

The deterministic comparisons leave two cases to integrate: no overflow, and the first overflow. To handle the latter, we separate a large retained set of true heavy endpoints from overflow caused by errors at light endpoints. For this subsection let $\mathcal{O}_j$ denote the event that group j is the first overflow. These events are pairwise disjoint. The heavy-count event defined next depends only on the true row and its retained endpoints; it does not itself assert that any overflow occurs.

At first overflow in group j, the previous default is smaller than ρ_j. Thus at least n_j retained real positions have input size strictly greater than ρ_j. Define the true heavy-count event

$$\Theta_j = \{\#\{y \text{ retained} : \mathsf{d}_y > \rho_j/4\} \geq n_j/2\}\,.$$

The dummy positions do not contribute to this count, since their input is the preceding default. On $\mathcal{O}_j \cap \Theta_j^c$, at least $n_j/2$ of the overflowing positions have $\mathsf{d}_y \leq \rho_j/4$. Their second targets vanish, because the floors lie below the h-threshold. Equation (9.8) then implies $e_j(y) \geq c\rho_j$ at all these positions. Consequently the indicator of this false-type first overflow is bounded by

$$\frac{C}{n_j \rho_j^s} \sum_y e_j(y)^s. \tag{9.12}$$

A bounded row-output error may be charged to this indicator. On $\mathcal{O}_j \cap \Theta_j$, we instead use the stability estimate and the conditional rarity of Θ_j at each endpoint.

Lemma 9.4 (Conditional rarity of a true heavy count)**.** *At every specified endpoint,*

$$\mathbb{P}(\Theta_j \mid X_u(x) = Y_a(y) = 1) \le \frac{C}{n_j \rho_j}. \tag{9.13}$$

The same bound holds with the narrower endpoint condition $Y_t(y) = 1$. Moreover, for every prefix mean row supremum,

$$\|\mathbf{1}_{\Theta_j} \beta_i\|_{L_s(\mu)} \le C c_* 2^{-p+B_1 d_i} (n_j \rho_j)^{-1/s}. \tag{9.14}$$

Proof. By Equation (9.1), the deterministic heavy set

$$A_j(x) = \{y : \mathsf{d}_y > \rho_j/4\} \quad \text{satisfies} \quad |A_j(x)| \le \frac{4C_{\text{mass}}}{a\rho_j}.$$

Conditional on one specified endpoint hit, each other endpoint in this set is retained with probability at most Ca, by Lemma 4.1. This remains true if the specified hit uses the narrower rate. The forced endpoint contributes at most 1, so the expected retained heavy count is at most $1+Ca|A_j(x)| \le C'/\rho_j$. The independent start condition has no effect. Markov's inequality at the threshold $n_j/2$ proves Equation (9.13). This calculation uses the conditional pair bound and linearity of expectation; it permits dependence among the heavy-endpoint hits.

For the first-array mean error at a fixed row and endpoint, write its deterministic magnitude as $b_i(y)$. Then

$$\mathbf{1}_{\Theta_j} \sup_{y : Y_a(y)=1} b_i(y)^s \le \sum_y \mathbf{1}_{\Theta_j} Y_a(y) b_i(y)^s.$$

Condition on the start and that endpoint before taking expectations. Equation (9.13) bounds the resulting expectation by $C/(n_j\rho_j)$ times the endpoint-integrated mean-error power. Indeed the factor $b_i(y)^s$ is fixed under this conditioning, so

$$\mathbb{E}\big[\mathbf{1}_{\Theta_j} X_u(x) Y_a(y) b_i(y)^s\big] \le \frac{C}{n_j \rho_j} \mathbb{E}\big[X_u(x) Y_a(y) b_i(y)^s\big].$$

The second-array mean error has a deterministic magnitude multiplied by $Y_t(y)$; repeat the same argument using the narrower conditional bound. Taking roots and using Equation (9.4) proves Equation (9.14). The conditioning step uses the deterministic mean-error magnitude at the specified endpoint. Prefix noise can depend on the heavy-count event, so no analogous conditional estimate is asserted for it. □

Proposition 9.5 (Integrated compression bound)**.** *Under the row hypotheses of Section 9.1, including the deterministic conditional means and the prefix bounds* (9.4)*, choose fixed sufficiently large c_1, then c_2, and sufficiently rapid prefix-noise decay K_n. The compressed array satisfies*

$$\left(\int \sum_{H \in \mathcal{H}} \sum_y |H(T_{m_*}; y)^{1/s} - H(\mathsf{d}, v; y)^{1/s}|^s \, d\mu \right)^{1/s} \le C_{\text{buf}} c_* 2^{-p}, \tag{9.15}$$

where C_{buf} is independent of the number of groups, row width, and requested precision. In particular, its coefficient can be made arbitrarily small by choosing c_* small.

Proof. Equation (9.2) converts every row sup bound above into a bound for its summed output power with a fixed factor. In fact the support of a discrepancy lies in the union of the true and computed supports, whose total size over $\mathcal{H}$ is at most $2C_{\text{supp}}$. Thus this step introduces no factor

from the ambient number of endpoints. The preceding compression error contributions have norm at most

$$Cc_*2^{-p}\sum_i \rho_i\big(2^{B_1 d_i}+2^{-K_n d_i}\big).$$

On the event of no overflow, Equation (9.10) adds the final-prefix cost, at most $Cc_*2^{-p}(2^{B_1}+2^{-K_n})$. By Equation (9.12) and the disjointness of the possible first-overflow events, the total false-type error power is at most

$$Cc_*^s 2^{-sp}\sum_j (n_j\rho_j^s)^{-1}\big(2^{sB_1 d_j}+2^{-sK_n d_j}\big). \tag{9.16}$$

For a true-type first overflow at j, use Lemma 9.3 and split $e_i \le \beta_i+\gamma_i$. For the mean part, enlarge the actual first-overflow event to Θ_j, apply Equation (9.14), and use Minkowski's inequality. Its total norm is bounded by

$$Cc_*2^{-p}\sum_j (n_j\rho_j)^{-1/s}\sum_{i\ge j} C_1^{m_*-i+1}2^{B_1 d_i}. \tag{9.17}$$

For the noise part, use the uniqueness of the first-overflow group on each row. Its tail noise sum is bounded pointwise by the single full sum $\sum_i C_1^{m_*-i+1}\gamma_i$, regardless of dependence between noise and overflow. More explicitly,

$$\sum_j \mathbf{1}_{\mathcal{O}_j}\sum_{i\ge j} C_1^{m_*-i+1}\gamma_i \le \sum_i C_1^{m_*-i+1}\gamma_i,$$

because at most one indicator on the left is nonzero. The absolute prefix-noise bound therefore gives norm at most

$$Cc_*2^{-p}\sum_i C_1^{m_*-i+1}2^{-K_n d_i}, \tag{9.18}$$

without an extra sum over j.

Choose $c_1 > B_1$ large enough for the floor conditions. Choose $c_2 > sc_1+sB_1$ with a fixed positive margin and also large enough that $n_j \ge K$. Equation (9.16) then converges uniformly. For its mean term the exponential factor is exactly $2^{-(c_2-sc_1-sB_1)d_j}$. For Equation (9.17), its inner sum is at most $C(d_j+1)^A 2^{B_1 d_j}$ for a fixed A, by Equation (9.7) and dyadic grouping; the stronger choice of c_2 just made ensures convergence of the outer sum, whose exponential factor is $2^{-((c_2-c_1)/s-B_1)d_j}$. Taking K_n sufficiently large handles Equation (9.18). The preceding compression error sum converges by $c_1 > B_1$. All constants are fixed before choosing the arbitrarily small prefix coefficient c_*, giving Equation (9.15) and the asserted spare coefficient. □

Proposition 9.5 bounds the compressed pre-incoming functions, including scores and witnesses. Before assembling the final table estimates, we impose the numerical range bounds needed to evaluate these arrays.

9.5 Bounding numerical ranges

In exact endpoint grouping, the additive increments are independent of which exceptions are stored. The path description and bounded child ranges give a fixed D for which $\|\Delta_i\|_\infty \le 2^{Dd_i}$, after increasing D to include immediate term counts. This also bounds a group when all its same-fingerprint increments are summed into one coordinate, with a possibly larger fixed D. Fix a common cutoff closure level $G \ge \max_H g_H$, and then choose $R_{\mathrm{sens}} > 10G$ above every sensitivity threshold. Let $R_i = 2^{B_2 d_i}$ and choose B_2 sufficiently large that

$$R_i \ge 10R_{\mathrm{sens}} + 10\sum_{k>i}\|\Delta_k\|_\infty \quad \text{for every group } i. \tag{9.19}$$

Dyadic grouping permits this uniform choice because $d_{i+1} = d_i/2$. After each compression, clip every coordinate and the default componentwise to $[-R_i, R_i]$. An off-list coordinate is clipped exactly as the default is, so it remains a default coordinate. These clamps therefore preserve the finite-list representation.

Lemma 9.6 (Range clamping)**.** *For fixed common increments satisfying Equation* (9.19)*, these clamps do not change the final relevant row functions. Each clamp is sup-norm nonexpansive, so in comparisons of two calculations it preserves the uniform Lipschitz propagation bound. The analogous assertion holds for vector sums clipped beyond their final sensitivity range and all remaining additive variation.*

Proof. Compare the clamped and unclamped calculations with the same increments. For the fixed level $G < R_{\text{sens}}/10$, once both defaults are at least G, all cutoffs are zero because of the dummies, and remain so: compression cannot lower the default and every clamp bound exceeds G. It remains to compare the calculations before this happens.

Maintain the following invariant. Their defaults agree while below G. Their coordinates either agree, or have matching signs and magnitudes well beyond R_{sens}; any such differing component has enough remaining margin that the total future increments cannot bring it into $[-R_{\text{sens}}, R_{\text{sens}}]$. At the first clamp of a component this follows from Equation (9.19). A subsequent clamp restores the same margin using the current R_i. Between clamps, common additions consume at most the available tail margin.

After an addition, entries whose components differ therefore have size larger than R_{sens} in both calculations. Their size order statistics either are equal below G or are both at least G: differing values lie above this comparison range and all other size values agree. In the latter case both new defaults close the cutoffs. In the former case the two values of u_0 agree and are below G. Only the first-coordinate order statistic clipped at $3u_0$ is needed to compute z'. Differing positive first components are above this range; differing negative first components are below zero and do not affect its nonnegative clipped value. Consequently the two new defaults agree as well. If this default is at least G, both calculations close. Otherwise every differing position lies in the compression identity region because its size exceeds $R_{\text{sens}} > 3G$; equal coordinates at other positions undergo the same update. Thus the invariant continues.

Before closure, all order statistic arguments and all entry arguments within their sensitivity ranges agree. Differing large components have matching signs and identical saturated behavior. The final row functions therefore agree. A scalar clipping map is 1-Lipschitz, which gives the comparison assertion. For a vector sum, the same tail-margin argument ensures that a clipped value stays on the same saturated side until the final bounded clipping; no order statistic argument is needed. □

The equality assertion of Lemma 9.6 is used for the comparison error bound: exact endpoint grouping supplies the same increment arrays regardless of the chosen list presentation. It preserves the values of every relevant $H \in \mathcal{H}$, including scores and smooth witnesses. In the adaptive comparison of Section 10, both the actual and comparison calculations are clamped from their definition. Their possibly different bucket increments are compared directly, and the nonexpansiveness part of Lemma 9.6 preserves the local perturbation bound. This use allows different real representatives, including retained positions whose current value equals the default.

9.6 The error reserve with exact endpoint grouping

The prefix estimates and compression bound now give the current task's error estimate under exact endpoint grouping. The hypotheses still require each conditional average to vary every random coordinate used by its sample function. We combine the compressed pre-incoming bounds with the table assembly comparison; the next section will verify this averaging hypothesis for the block comparison calculation.

Proposition 9.7 (Error reserve with exact endpoint grouping)**.** *Suppose earlier row tasks have root discrepancy and earlier vector tasks have ordinary error at most $C_b 2^{-p'}$, with each child's own precision, and satisfy the support and common-parameter properties of Section 6. Suppose endpoint grouping is exact and each conditional walk varies every coordinate on which its sample terms depend. After compression and the range clamps above, all active row outputs, pre-incoming scores, and witnesses have error at most $(C_b/2)2^{-p}$. The same holds for the final clipped vectors and the assembled correction outputs, after choosing the fixed constants in the order specified below.*

Proof. To apply Proposition 9.5, use the deterministic-mean and absolute-noise bounds of Lemma 8.9. The first target is nonnegative with bounded deterministic row mass before division by a, and its second target is bounded by a fixed multiple of ζ and vanishes below the preparatory support threshold. The row functions with separated thresholds and their witnesses have fixed Lipschitz constants for their sth roots and fixed support bounds. These verify its remaining hypotheses. The compression proposition shows that all their errors are a fixed multiple of $c_* 2^{-p}$ when its convergent series of floors, capacities, and noise weights are chosen. This includes the pre-incoming quantities required by the incoming comparison. Exact endpoint grouping gives common increment arrays for the clamped and unclamped calculations. Lemma 9.6 therefore preserves these row-function values, including scores and witnesses, so the same estimates hold after clamping.

Lemma 6.6 then charges the incoming cutoffs by those same pre-output norms. Its separate column-vector calls have a fixed positive budget slack and decrease task rank. Equation (8.11) pays their broad-domain factor, and increasing A_* makes their remaining coefficient as small as required. A final vector needs no compression; its clipping is nonexpansive and the bounds at $d_j = 1$ apply directly.

Choose the fixed table range/support bound C_b first. Choose c_0 and K_{prec} to retain the margin in Equation (8.12) and fix B_1. The compression proof then chooses its floor exponent c_1, capacity exponent c_2, and required noise decay K_n. Choose the rank coefficient A_* and the walk-length coefficient ℓ sufficiently large to supply the needed small c_*. The total coefficient can thus be made at most $C_b/2$. Inactive tasks retain their separate trivial C_b bound. The remaining half of the active-task error bound is reserved for the coupled fingerprint error. □

The reserve applies to a local exact-grouping calculation with the stated averaging property. The actual algorithm will also use short endpoint fingerprints. In Section 10 we define an exact-grouping comparison within each budget block, verify that it has this averaging property, and bound its difference from the algorithm.

10 Adaptive fingerprints within budget blocks

We now replace endpoint comparisons at small budget slacks by short fingerprints. The endpoint universe has size $2^{O(B)}$, whereas the fingerprint range has size B. These comparisons change the computed arrays. To control the change, we compare the algorithm with a calculation that uses exact equality within one budget block and the actual algorithms below that block.

This comparison has two roles. Its conditional averages give the deterministic means needed by the compression reserve, and its candidate endpoints form a set independent of the current block's fingerprint. We charge actual paths leaving that set to errors in lower calls, then bound collisions inside it. The resulting coupled induction proves the error bound for the actual estimator.

We use the root error metric $\mathfrak{d}_{u,a}$ for bounded nonnegative row outputs, its ordinary version $\|\cdot\|_{u,a;s}$ for raw arrays, and the ordinary vertex metric for vectors. For a row quantity $Z(x,\sigma)$ use

$$\|Z\|_{u;s}^{s} = \frac{1}{N}\mathbb{E}_{\sigma}\sum_{x}\frac{X_u(x)}{u}|Z(x,\sigma)|^{s}. \tag{10.1}$$

The value of s remains 8. Supremums over endpoints below are over actual ID positions, with the appropriate retained-domain masks; virtual default positions are never counted with the multiplicity of the ambient universe.

10.1 Uniform seeds and their access rule

The fixed-pair estimate below combines polynomial evaluation, as in classical algebraic fingerprinting [32], with affine universal hashing [11]. Its proof is included to specify the fair-bit distribution and space bound. The adaptive use of these fingerprints requires the separate block comparison that follows.

Lemma 10.1 (Short fingerprints)**.** *Suppose every endpoint has an injective, fixed-length binary encoding of length $L_{\mathrm{ID}} \le c_{\mathrm{ID}}B$. There is a uniform construction from $O(\log B)$ independent fair bits of a map $k : \{\text{endpoint IDs}\} \longrightarrow \{1,\dots,B\}$ such that, for distinct fixed IDs v, w,*

$$\Pr[k(v) = k(w)] \le C/B. \tag{10.2}$$

Evaluating the map at a presented endpoint needs $O(\log B)$ arithmetic scratch in addition to access to its encoding.

Proof. Pad all encodings to the same length and put $P_v(T) = \sum_{i=0}^{L_{\mathrm{ID}}-1} v_i T^i$. Choose a prime $B^3 \le q \le 2B^3$, increasing the fixed lower bound on B if necessary. Thus $L_{\mathrm{ID}} < q$. Let $\ell = \lceil \log_2 q \rceil$. Split a string of 3ℓ fair bits into independent integers R_1, R_2, R_3 in $\{0,\dots,2^{\ell}-1\}$, and reduce them modulo q to obtain r, α, β. Each residue has probability at most $2/q$. Let $\tau : \mathbb{F}_q \longrightarrow \{1,\dots,B\}$ partition the standard residues into consecutive sets of sizes differing by at most one, and set

$$k(v) = \tau\big(\alpha P_v(r) + \beta\big).$$

All computations are in $\mathbb{F}_q$ before applying τ.

The nonzero polynomial $P_v - P_w$ has degree at most $L_{\mathrm{ID}} - 1$, so it has at most $L_{\mathrm{ID}} - 1$ roots. This root bound follows by repeatedly factoring $T - r_0$ at a root; it applies here because the coefficient difference 1 or -1 is nonzero in $\mathbb{F}_q$. Consequently

$$\Pr[P_v(r) = P_w(r)] \le 2(L_{\mathrm{ID}} - 1)/q.$$

For distinct evaluations z, z', the map $(\alpha,\beta) \mapsto (\alpha z + \beta, \alpha z' + \beta)$ is a bijection of $\mathbb{F}_q^2$. Under uniform parameters its two outputs are uniform and independent. The probability that their τ-values agree is

$$q^{-2}\sum_{j=1}^{B}|\tau^{-1}(j)|^2 \le \frac{\lceil q/B \rceil}{q} \le \frac{1}{B} + \frac{1}{q}.$$

The actual joint law of (α, β) is pointwise at most four times the uniform law and is independent of r. Adding the two cases gives $2(L_{\mathrm{ID}} - 1)/q + 4/B + 4/q = O(1/B)$. A polynomial evaluation by successively multiplying by the argument and adding the next coefficient and the residue arithmetic use $O(\log q) = O(\log B)$ bits. The prime can be chosen uniformly by searching and trial division in that space; one exists in the indicated interval by Bertrand's postulate, which Section 13.1 also uses to choose the environment field. □

The block comparison below uses only the weaker consequence $CB^{-1/2}$ of Equation (10.2), valid since $B \geq 1$. Any fixed negative power of B suffices here: its gain absorbs the exponential growth with block width when that width is a sufficiently small multiple of $\log_2 B$. The exponent enters only through the direct collision charge in Equation (10.10) and the choice of ϵ in Equation (10.14).

Fix a small positive rational ϵ, to be chosen last. This coefficient controls the fingerprint block width. Write

$$h_* = \lfloor \epsilon \log_2 B \rfloor, \qquad M_b = bh_*, \qquad \mathcal{I}_b = \{M_b, \dots, M_b + h_* - 1\}. \tag{10.3}$$

Enlarge the fixed minimum B so $h_* \geq 1$ and $h_* \geq (\epsilon/2) \log_2 B$. Give each block meeting $\{0, \dots, M_{\mathrm{top}}\}$ an independent seed from Lemma 10.1. Pad each seed to a multiple of the fixed integer H by unused independent bits. Since $M_{\mathrm{top}} = O(B)$, the total number of bits is

$$O\big((1 + B/h_*)(\log B + H)\big) = O(B). \tag{10.4}$$

Here and throughout, constants may depend on the fixed ϵ and H. We enumerate the original fair-bit strings when enumerating environments; reducing chunks modulo q does not change that sampling convention.

The following access rule is part of the algorithm. A task whose budget lies in $\mathcal{I}_b$ may inspect seeds with indices at most b. Its conditional walks vary all earlier seeds as free bits and freeze the seed of block b and every later seed. The restriction applies to structural choices, port orders, validity tests, and predecessor tests as well as to numerical values. The controller construction below implements this rule for every access operation, including inverse queries. It is not enough to impose it only on the final output values.

This rule is stated in the coordinates of the task's *effective environment*. Write σ for that environment when analyzing a task. A fixed ancestor variant may mix its free seed coordinates before supplying σ to a descendant; the descendant's table depends on the resulting environment, not on the word that produced it. Thus the rule does not forbid reading a later-indexed bit of the original master environment while replaying that word. It requires the descendant's values, port orders, and access operations to be functions only of its permitted effective coordinates. Lemma 12.12 implements this distinction. Each fixed variant is measure preserving by Lemma 7.3, so the integrated estimates may change variables to its effective environment. At a block-b task, its own variants act only on effective seeds earlier than b and freeze its effective block-b seed and all later ones.

10.2 The update rule and the comparison calculation

We now complete the recursive definition of the actual task; inactive tasks still return zero outputs and empty access classes. For an active row task, the additive schedule of Section 8 supplies candidate increments from smaller-budget outputs. Process the groups in decreasing slack. In each group, distribute increments by the rule below, then use the compression, range clamps, and stored list rule of Section 9. After the last group, form the pre-incoming row functions and uncapped row

access classes from the completed arrays, then assemble any incoming cutoffs and column multipliers as in Section 6. The separate column-vector calls have smaller budget. Vector tasks use the same schedule to sum their scalar terms, with their prescribed range clamps and final clipping; they require no endpoint grouping or row-list compression.

The row-group distribution rule is as follows. For a dyadic group with slack $d > 2h_*$, compare endpoint IDs exactly. For a group with $d \le 2h_*$, use the fingerprint from the task's block as follows. First discard invalid candidates, including invalid guessed wraps, individually. Sum all new candidate increments having the same fingerprint. Add a bucket's sum to every old representative with that fingerprint. If the bucket has no old representative, introduce exactly its first valid new candidate, with its previous value equal to the old default, and add the bucket sum there. Do not combine or discard old representatives in this fingerprint step. Apply the same full-array compression, clamping, and stored list rule as in the preceding section.

Distinct actual representatives remain distinct. Indeed two introduced representatives have different fingerprints, and an introduced representative's fingerprint differs from every old fingerprint. Increments at one and the same actual endpoint are always combined, even when other endpoints collide with it. Stored values are never summed by fingerprint. The two arrays use the same fingerprints and the same representative selection for the entire group.

For an analytical comparison at a budget $M \in \mathcal{I}_b$, define the *comparison calculation for block b* by replacing fingerprint comparisons by exact ID comparisons throughout that block, recursively in every child whose budget still belongs to $\mathcal{I}_b$. Children below M_b are the actual algorithms, with exactly the same environments in the two calculations. This replacement applies to the entire numerical and structural calculation: candidate validity, representative selection, compression, exposed classes, and their port orders. Descriptor enumeration uses fixed index orders. Each finite-accuracy test for a stored list or exposed class retains its prescribed accuracy level and tie convention, independently of any requested output accuracy. All these operations use the corresponding comparison operands. Thus the comparison calculation specifies its access choices as well as its represented arrays.

Denote the actual and comparison versions by superscripts ac and cmp. Both still perform compression and clamping. The comparison calculation is a finite analytical recursion.

Clamping is part of both definitions. The comparison error reserve uses Lemma 9.6 for exact-ID increments, which are independent of the stored exception list. That lemma preserves the relevant row functions and witnesses. The coupling below compares the two clamped calculations directly. Its propagation bound uses the Lipschitz compression map and the nonexpansiveness of each common clamp.

10.3 The estimator bound and its deduction

The following theorem is the statistical input to the median construction in Section 13.

Theorem 10.2 (Error bounds for the actual estimators)**.** *For every allowed row task of Section 8.1, let H be one of its true row outputs and $\bar{H}$ its actual estimate, with the same stage, rates, sampling-matrix roles, and padding suffixes. This includes the pre-incoming scores and support witnesses. With precision p defined in Equation* (8.10)*, the bound is*

$$\mathfrak{d}_{u,a}(\bar{H}, H) \le C_b 2^{-p}.$$

Every allowed clipped vector has ordinary error at most $C_b 2^{-p}$ in its prescribed vertex norm. Write $\bar{W}_l$ for the actual reward estimate at the fixed task's budget, source rate u, roles, and suffixes. Then

every allowed reward query satisfies

$$\frac{1}{N}\mathbb{E}_\sigma \sum_{x\in V_l} \frac{X_u(x)}{u}|\bar{W}_l(x) - W_l(x)|^s \le C_b^s 2^{-sp}. \tag{10.5}$$

These estimates are uniform over the allowed task parameters and stage, and the constants are independent of the input and of B.

Proposition 10.3 (Uniform block coupling)**.** *The fixed constants can be chosen with $D_1 > D_2 + 1$ and $2^{-(D_1-D_2)h_*} \le C_b/2$ so that, for every task of budget $M \in \mathcal{I}_b$ and precision p, each prescribed actual–comparison output discrepancy, including every pre-incoming score and witness, is at most*

$$2^{-p-D_1h_*+D_2(M-M_b)}.$$

The same assertion holds for the ordinary vector metrics.

Proof of Theorem 10.2. Proposition 9.7 gives the active comparison calculation error at most $(C_b/2)2^{-p}$ once its averaging hypothesis holds. Lemma 10.4 verifies that hypothesis. Proposition 10.3 supplies the actual–comparison discrepancy. Since $0 \le M - M_b < h_*$, its coefficient satisfies

$$2^{-D_1h_*+D_2(M-M_b)} \le 2^{-(D_1-D_2)h_*} \le C_b/2.$$

The triangle inequality gives the asserted active-task bounds. Inactive tasks satisfy the same bounds by their fixed ranges and row supports. The simultaneous induction proving Proposition 10.3 is given in Section 10.7; it includes the earlier-task hypotheses of the comparison estimate and both kinds of output. Applying its ordinary vector bound to W_l and taking the sth power gives Equation (10.5). □

10.4 Independence of the comparison calculation

Lemma 10.4 (Independence of the comparison block)**.** *Every comparison value and structural choice for block b is independent of that block's effective seed and of later effective seeds. In a task of budget $M \in \mathcal{I}_b$, the actual and comparison row states immediately before the first group with $d \le 2h_*$ are identical and independent of that seed. Their stored lists at this boundary have size $2^{O(h_*)}$. The deterministic slice-mean assertion required by Lemma 8.9 holds for the comparison calculation.*

Proof. Induct on the budget and, within a task, on its prescribed sequence of operations. Lower-block algorithms ignore the seed of block b and later seeds by the access rule; same-block comparison children ignore them by induction. Candidate tests use these children, exact endpoint grouping precedes compression, and the compressed row determines its exposed classes and pre-incoming scores. Incoming order statistics use those completed scores. Separate column-vector queries likewise use a smaller-budget comparison child in this block or an actual child below it. All statements here concern the current effective environment: each same-block transformation preserves its current and later seeds, and each lower-block transformation acts on a subset of its earlier seeds. Each numerical recipe uses the same fixed operand order, and each structural test uses its fixed accuracy level and tie rule. Its result therefore depends only on earlier comparison operands, including their finite numerical approximations. The fixed index orders then give seed-independent representatives and ports.

In particular, exact full-array addition, compression, and the prescribed row functions are seed independent. Reordering paths or choosing a different representative of the same endpoint would not

change an exact sum; retaining an extra default-valued position would not change the represented array. The fixed conventions above also remove these presentation choices from the structural calculation.

For a group with $d > 2h_*$, every child budget is at most $M - d < M_b$. Both runs use the same lower-block children and exact comparisons there. All their list operations, including their deterministic tie conventions, ignore the present seed. Thus their boundary states agree. The last larger dyadic slack, if there is one, is at most $4h_*$, so its stored list has at most $2^{O(h_*)}$ positions. If there is no larger group, the boundary list is empty.

Finally, the comparison estimand depends only on sampling matrices and earlier seeds. These are precisely the variables averaged by its conditional walk, subject to its stated vertex conditions. Same-block seeds are irrelevant, and lower-block seeds are free variables of this walk. Projection onto the uniform measure of the slice therefore gives a deterministic quantity at its indicated pair or vertex, with the prescribed retained-domain masks. In particular, no random dependence on a frozen seed remains in this mean. For the second array, its deterministic mean magnitude is extended by its prescribed narrower endpoint mask, as required in Section 9. The seed rule is required of all access operations, including inverse queries and port orders; Lemma 12.12 supplies its implementation. Lemmas 7.2 and 7.3 therefore apply to this function. This verifies the deterministic-mean hypothesis of Proposition 9.7. □

10.5 Candidate endpoint sets from witness supports and escaped paths

To control adaptive candidates, we first bound a set of endpoints determined by the comparison calculation. An actual path leaving this set will be charged to a child witness discrepancy.

We recall the relevant interfaces from Lemma 6.5. Every exposed actual path port has a broader smooth witness equal to one at that port. Each witness has bounded row support. The actual correction ports, including exposed ports of weight zero, have incoming multiplicity at most K_c. Tables contain one entry per pair of actual IDs. After fixing a copy part, the child tuple is common over the whole table; in particular, a right child reached from a midpoint does not depend on the discarded old start. All these statements hold for arbitrary computed raw arrays, not just for accurate ones.

The incoming cap applies to the entire exposed correction relation, including its zero-weight entries. The broader auxiliary score class needs only a row bound: it is queried for scores and witnesses, whereas the left factor of a two-factor path is a capped correction table. This distinction permits the source sum below to be removed even when an offending candidate has zero numerical weight.

Let $\mathcal{G}_b$ be the sigma-field generated by all coordinates of the task's effective environment except the seed of block b. Fix a row x and condition on $\mathcal{G}_b$ with $X_u(x) = 1$. Take its boundary list from Lemma 10.4, including every stored exact-default slot. For every small group and every immediate descriptor variant, evaluate the comparison factors and their witnesses in the environment obtained by that fixed variant. This transformation leaves the present and later seeds frozen. Include the endpoints in each direct witness support and, for a two-factor path, the endpoints in the right witness supports at all midpoints in the left witness support. Include local base endpoints when applicable. Let $\mathcal{E}_x$ be the union of these endpoints and the boundary list. Set $\mathcal{E}_x = \varnothing$ when $X_u(x) = 0$. Unused or invalid comparison descriptors may enlarge this set. Actual candidates must still satisfy their wrap tests and all transformed source, midpoint, and terminal memberships; every error estimate below retains the memberships needed for its child metrics.

Lemma 10.5 (Size and independence of the candidate endpoint set)**.** *The set $\mathcal{E}_x$ is independent of the present seed and has size at most $2^{C_E h_*}$ for a fixed C_E. All comparison nondefault representatives in the small groups belong to this set. Consequently, conditional on the other data,*

$$\Pr[\textit{two distinct IDs in } \mathcal{E}_x \textit{ have the same fingerprint}] \le C\, 2^{2C_E h_*} B^{-1/2}. \tag{10.6}$$

Proof. Each fixed variant is independent of the present seed and leaves it unchanged. Lemma 10.4 therefore makes its comparison witness supports $\mathcal{G}_b$-measurable. We use these supports as sets of actual IDs, not the order of a port enumeration. At a group of slack d there are $2^{O(d)}$ immediate descriptors, including variants, difference parts, and copy choices. The number of endpoints contributed by a direct witness is bounded. A two-factor path contributes a bounded number of left witness midpoints and, for each, a bounded number of right witness endpoints. Summing over the dyadic groups with $d \le 2h_*$ and adding the boundary list proves the size bound.

Comparison exposed ports lie in their comparison witness supports. Apart from the boundary representatives, exact updates can therefore create nondefault values only at the included endpoints. Compression maps all untouched default positions to the new default. Any extra exact-default slots in a stored list may be omitted when describing its full array. This proves the representative claim. Equation (10.6) is now the pair union bound from Lemma 10.1, conditional on the fixed candidate endpoint set. □

Let $\mathcal{F}_x$ be the event that some valid actual candidate in the small groups has endpoint outside $\mathcal{E}_x$, whether or not its increment is zero. As representatives are only carried forward or chosen among new candidates, all actual representatives stay in the candidate endpoint set outside this event. This includes default-valued representatives: they start in the full boundary list or inherit a candidate endpoint. Dummy copies used to compute order statistics are not new real representatives. Thus omitting an exact-default comparison slot when describing its full array does not omit an actual address from the collision analysis.

Lemma 10.6 (Charging an escaped candidate)**.** *Suppose actual–comparison child discrepancies at budget m have their prescribed metric bounds $2^{-p_m}\lambda_m$. The contribution of $\mathcal{F}_x$ to any bounded final pre-incoming row output has norm at most*

$$2^{-p} \sum_{\substack{d \le 2h_* \\ m \in \mathcal{I}_b,\ m \le M-d}} 2^{Cd} \lambda_m, \tag{10.7}$$

where the sum includes the finitely many dependency types and their descriptor multiplicities in its constant C. Equivalently, one may use the maximum of λ_m over the children of each slack group and increase C.

Proof. An actual endpoint outside the candidate endpoint set has an offending path factor. For a direct path its actual port is outside the comparison witness support. For a two-factor path, either its actual left port is outside its comparison witness support, or its midpoint lies in that support and its actual right port is outside the corresponding comparison right witness support. At the offending port the actual witness is one and the comparison witness is zero. Thus the indicator of this event is at most the s-th power of their root discrepancy there. Union bounding over candidate paths gives a sum of such nonnegative powers. The unit-witness property holds also for exposed zero-weight ports; no lower bound on the candidate's increment is being assumed.

Each output under consideration is bounded and has bounded row support. The union of the two output supports is still bounded, by Equation (9.2). Hence its entire row error power on $\mathcal{F}_x$ is

at most a fixed multiple of $\mathbf{1}_{\mathcal{F}_x}$. We charge this row-failure indicator, rather than the size of the offending increment. There is no endpoint sum left on the output side to pay for.

The right-factor charge is the case where the old start must be summed out. Fix a guessed variant and a two-factor path $x \to z \to y$. Change variables by its measure-preserving transformation, as in Lemma 7.3; in the following display all memberships, ports, and witness values are evaluated in that transformed environment. Write $W_R^{\mathrm{ac}}, W_R^{\mathrm{cmp}}$ for the actual and comparison right witnesses, $e_R(z,y) = W_R^{\mathrm{ac}}(z,y)^{1/s} - W_R^{\mathrm{cmp}}(z,y)^{1/s}$ for their root discrepancy, and $\mathcal{P}_L^{\mathrm{ac}}$ for the actual left exposed-port relation. The candidate's validity retains $X_u(x)$, $Z_f(z)$, and the right child's endpoint membership $Y_{a'}(y)$. Dropping only surplus guards, the right-port contribution is at most a constant times

$$\begin{aligned}
&\frac{1}{N}\mathbb{E}\sum_x \frac{X_u(x)}{u} \sum_{z:(x,z)\in\mathcal{P}_L^{\mathrm{ac}}} Z_f(z) \sum_{y:Y_{a'}(y)=1} |e_R(z,y)|^s \\
&\qquad \le \frac{K_c}{Nu}\mathbb{E}\sum_z Z_f(z) \sum_{y:Y_{a'}(y)=1} |e_R(z,y)|^s = K_c \frac{f}{u}\,\mathfrak{d}_{f,a'}(W_R^{\mathrm{ac}}, W_R^{\mathrm{cmp}})^s. \qquad (10.8)
\end{aligned}$$

The middle inequality uses the incoming cap of the *actual* left port relation. The right task's tuple is independent of x, so the same discrepancy is being summed when the old start is removed. This condition is necessary for the displayed equality with the right metric. For the fixed variant and copy part, the right tuple has only the prescribed task parameters and transformed environment. The midpoint z is its vertex argument, and the old source x is not a parameter. Its final cutoff and column queries use that same effective tuple, rather than the ancestral descriptor that happened to discover the entry.

For a direct or left offending port, sum the child's witness discrepancy over its own row and endpoint domain; the bounded number of possible right ports incurs only a fixed factor. Projected paths are partitioned by the fixed copy symbols, with constant multiplicity. Normalization and preparation dependencies make the same direct witness comparison. If a candidate arose from a narrower terminal slice, its validity mask may be dropped in this row-event estimate. Thus no factor π_a/π_t is incurred. In contrast, the transformed source hit cannot be omitted before the change of variables and the incoming-cap sum; it has been retained in Equation (10.8).

At slack d, scale ratios, variants, and immediate path counts cost at most $2^{O(d)}$, as in Lemma 8.8. The possible reset factor in Equation (10.8) is absorbed by the precision gain. Admissible row tasks have $u \le f$, as specified in Section 6. Equation (8.11) gives, after the scale-dependent $O(d)$ terms,

$$\begin{aligned}
p_m &\ge p - O(d) + \tfrac{1}{2}\log_2(f/u), \\
(f/u)^{1/s} 2^{-p_m} &\le 2^{-p+O(d)} (f/u)^{1/8-1/2} \le 2^{-p+O(d)}.
\end{aligned}$$

Take s-th roots and use Minkowski to sum the charges. A child below M_b is identical in the two versions and contributes zero. All remaining children satisfy $m \le M - d$ and belong to $\mathcal{I}_b$, which proves Equation (10.7). □

10.6 Bucket algebra and the collision charge

The escape estimate leaves rows whose candidates stay in $\mathcal{E}_x$. On these rows we separate child discrepancies from collisions among the comparison endpoints. We compare full arrays with their defaults, so different stored list presentations require no extra error term. The following observation explains the fingerprint update rule.

Lemma 10.7 (Only increments are aliased)**.** *Consider one small group on $\mathcal{F}_x^c$. Let $\Delta_{\mathrm{ac}}(y), \Delta_{\mathrm{cmp}}(y)$ be its unaliased increments, namely its true-ID sums using actual and comparison children, respectively. Let $k_d \le 2^{Cd}$ bound its number of immediate descriptors, and let $\mathcal{C}_x$ denote a collision on $\mathcal{E}_x$. The discrepancy between the actual bucket-distributed increment and the comparison true-ID increment is at most*

$$(k_d + 1)\left(\sup_y |\Delta_{\mathrm{ac}}(y) - \Delta_{\mathrm{cmp}}(y)| + \mathbf{1}_{\mathcal{C}_x} \sup_y |\Delta_{\mathrm{cmp}}(y)|\right) \tag{10.9}$$

in sup norm, simultaneously for both arrays. The old full-array discrepancy is added to this bound with coefficient one.

Proof. When all candidate endpoint set fingerprints are distinct, the rule gives exactly the actual true-ID increment: each old occurrence is updated at its own ID, and each genuinely new ID has its own bucket. Thus the discrepancy is simply $\Delta_{\mathrm{ac}} - \Delta_{\mathrm{cmp}}$.

When there is a collision, each updated representative receives a sum of increments at at most k_d distinct actual addresses. Its magnitude is at most $k_d \sup_y |\Delta_{\mathrm{ac}}(y)|$. At positions not updated it is zero. At either type of position subtracting the comparison increment costs at most a further $\sup_y |\Delta_{\mathrm{cmp}}(y)|$. Apply

$$\sup_y |\Delta_{\mathrm{ac}}(y)| \le \sup_y |\Delta_{\mathrm{ac}}(y) - \Delta_{\mathrm{cmp}}(y)| + \sup_y |\Delta_{\mathrm{cmp}}(y)|$$

to obtain Equation (10.9). This also covers an increment duplicated at several old representatives and a missing increment at an unchosen new representative.

At no point are old values combined or moved to a different ID. An unlisted old position has exactly the old default, and a newly selected representative starts with that value. Therefore the full-array comparison before compression is the old discrepancy plus the increment discrepancy just bounded. There is no term involving the height of an old value multiplied by a collision indicator. □

In applying this lemma to a group, combine contributions at equal actual endpoints *within each paired difference* before measuring its comparison height. At a fixed endpoint and original generator word, the outer slice and wrap tests are the same for budgets m and $m-1$. Their constituent port sets may differ, but each set contains all nonzero entries and enumerates each actual ID pair once; an absent entry contributes zero. Their true-ID sums are therefore exactly the two terms of the paired difference. Subtracting the same true random-table term from both sides, as in Lemma 8.8, preserves its cancellation. One can then sum the resulting sup bounds over difference layers. This remains legitimate if a first-bucket representative is chosen across several layers: the proof of Lemma 10.7 applies to their total increment, and the triangle inequality bounds the total by the sum of the paired bounds. A descriptor with zero increment contributes only to k_d in this bound. It may affect representative selection, which is already covered by Lemma 10.7 and the escape event.

Lemma 10.8 (Direct cost of collisions)**.** *The direct collision contribution to any final pre-incoming output has norm at most*

$$2^{-p}\,2^{Ch_*}B^{-1/(2s)}, \tag{10.10}$$

for a fixed C independent of the later choices of D_1, D_2, ϵ. All remaining contributions on $\mathcal{F}_x^c$ are bounded by actual–comparison child discrepancy norms with an amplification at most 2^{Cd} at slack d.

Proof. For each small dyadic group, let $\Delta_d^{\mathrm{cmp},\nu}(x,y)$ be its complete comparison true-ID increment in array $\nu \in \{1,2\}$, including any exact base term assigned to that group. Take an absent second array to be zero, and choose a deterministic descriptor bound $k_d \le 2^{Cd}$. By Lemma 8.8, the comparison paired increments at a group of slack d obey their crude absolute estimates in the retained endpoint metric. These estimates hold for each fixed valid variant after its measure-preserving change of variables. Counting variants and summing difference layers by the triangle inequality costs $2^{O(d)}$. Within each paired layer the true-ID cancellation is performed before this inequality is applied. If a local unpaired base term occurs in a small group, its first-group slack satisfies $M = O(d)$, so the same crude bound holds in its normalized raw units. Since $\sup_y |Z_y|^s \le \sum_y |Z_y|^s$ over actual ID positions, the row norm of $\max_\nu \sup_y |\Delta_d^{\mathrm{cmp},\nu}(x,y)|$ is at most $2^{-p+O(d)}$.

Lemmas 9.1 and 9.6, together with Equation (9.7), propagate an input discrepancy at slack d to the final array with a factor polynomial in $d+1$. At each step the discrepancy is measured between the clamped full arrays, including their defaults. The next bucket increment is compared by Lemma 10.7; compression then applies its uniform Lipschitz bound and the common clamp cannot enlarge the discrepancy. This recurrence does not require their later bucket distributions or stored lists to agree. Row functions with Lipschitz sth roots and bounded final support then give the same type of bound for the output metric. Choose fixed C_P, A_P so that $C_P(d+1)^{A_P}$ bounds this propagation. For a retained row define

$$H_x = C_P \sum_{\substack{d \le 2h_* \\ d \text{ a group slack}}} (d+1)^{A_P}(k_d+1) \max_\nu \sup_y |\Delta_d^{\mathrm{cmp},\nu}(x,y)|.$$

Set $H_x = 0$ when $X_u(x) = 0$. This quantity bounds the propagated comparison-height terms in Equation (10.9). The preceding estimates give

$$\|H\|_{u;s} \le 2^{-p+C_0h_*}.$$

All increments defining H_x are comparison, and its other factors are deterministic. Thus H_x and $\mathcal{E}_x$ are $\mathcal{G}_b$-measurable. Conditioning on this sigma-field gives

$$\mathbb{E}[\mathbf{1}_{\mathcal{C}_x} H_x^s \mid \mathcal{G}_b] = H_x^s \Pr(\mathcal{C}_x \mid \mathcal{G}_b) \le C\,2^{2C_E h_*} B^{-1/2} H_x^s.$$

The row membership $X_u(x)$ is also $\mathcal{G}_b$-measurable. Integrating the last inequality in the row measure therefore yields

$$\|\mathbf{1}_{\mathcal{C}} H\|_{u;s}^s \le C\,2^{2C_E h_*} B^{-1/2} \|H\|_{u;s}^s.$$

Taking roots proves Equation (10.10) after enlarging C. We have not conditioned on $\mathcal{F}_x^c$ in this independence calculation: the actual error there is bounded by $\mathbf{1}_{\mathcal{C}_x} H_x$, which is then integrated on the whole space. Conditioning on the seed-dependent stay-inside event is unnecessary and could destroy the stated independence.

For the first term of Equation (10.9), split products using bounded-factor differences, actual or comparison row caps, and the actual or comparison incoming cap of the represented two-way factor,

exactly as in Lemma 8.8. These are comparisons of unaliased sums using the respective children. Fixed valid variants, scale costs, and counts are all at most $2^{O(d)}$, with the possible source-release factor absorbed by the same precision gain as in Equation (10.8). Compression propagation is again absorbed in 2^{Cd}. This proves the remaining assertion. □

10.7 Closing the coupled induction

Proof of Proposition 10.3. Write the claimed discrepancy bound as

$$2^{-p}\lambda_M, \qquad \lambda_M = 2^{-D_1 h_* + D_2(M-M_b)}. \tag{10.11}$$

Perform strong induction on the statistical budget, simultaneously for the actual target estimates, the block comparison target estimates needed at that budget, and Equation (10.11). An inactive task returns the same zero outputs and empty paths in both versions, so its coupling error is zero. Lower-block children are identical in a block coupling and already have their actual target bounds. Same-block comparison children have the comparison target bounds at smaller budgets. Therefore Lemma 10.4, the prefix estimate, and the compression analysis give the comparison active target bounds with the spare constants in Proposition 9.7.

For the coupling, Lemmas 10.6–10.8 bound the pre-incoming quantities. Incoming cutoffs are then compared by their order-statistic bounds and the Lipschitz bounds on their sth roots. In their multiplier effect, restrict to the union of the final supports of the two tables; these have bounded incoming multiplicity. Thus Lemma 6.6 costs only a fixed factor on pre-incoming errors. Its separately queried column vectors have a positive constant budget drop and their f/u metric factor is covered by their precision gain. Vector raw sums have no endpoint grouping, but their same-block children may differ. The same crude individual-variant comparisons and nonexpansive clamps bound those differences. Detector order-statistic comparisons are already part of the assembly estimate.

Collect the finite number of output types and absorb their fixed assembly factors into one fixed exponent C. For a child budget $m \in \mathcal{I}_b$ used at group slack d, we have

$$\lambda_m = \lambda_M 2^{-D_2(M-m)} \le \lambda_M 2^{-D_2 d}. \tag{10.12}$$

For a separate post-assembly query with constant positive budget drop, use $d = 1$ in this inequality. Replacing the dyadic group sum by the larger sum over positive integers, all coupled child charges are at most

$$2^{-p}\lambda_M \sum_{d=1}^{2h_*} 2^{-(D_2-C)d}. \tag{10.13}$$

Counts of budget layers and dependency types are included in 2^{Cd}. Groups with larger slack have identical children and contribute zero. Together with Equation (10.10), this is the complete coupling recurrence.

Choose $D_2 > C$ so large that the infinite geometric sum in Equation (10.13) is at most $1/4$. Then choose $D_1 > D_2 + 1$. Finally choose rational $\epsilon > 0$ so small that

$$\frac{1}{2s\epsilon} > C + D_1 + 2. \tag{10.14}$$

Since $h_* \le \epsilon \log_2 B$, the direct relative charge satisfies

$$2^{Ch_*} B^{-1/(2s)} \le 2^{-(1/(2s\epsilon)-C)h_*} \le \tfrac{1}{4}\, 2^{-D_1 h_*} \le \tfrac{1}{4}\lambda_M,$$

after increasing the fixed minimum B to absorb any fixed coefficient. The child and direct charges fit within Equation (10.11), even with room for fixed norm conventions already included in C.

Moreover $0 \le M - M_b < h_*$ implies

$$\lambda_M \le 2^{-(D_1-D_2)h_*}.$$

This is uniformly as small as required after the same minimum-size padding. The triangle inequality therefore adds the coupling bound to the spare comparison active bound, for example $C_b 2^{-p}/2$, without exceeding $C_b 2^{-p}$. Inactive target bounds are the previously fixed bounded-range bounds. This completes all three inductions.

The order of choices is consequential. Fix the analytical hierarchy, table thresholds, and trivial-case bound first. Choose the precision and cutoff constants for the mean comparisons; then the compression floors, capacities, and walk lengths so the comparison sums converge with their spare coefficients. Fix clamping ranges and the crude exponents in the fixed-variant comparisons next. These choices determine C. Only after them choose D_2, D_1, ϵ as above. Thus reducing ϵ to obtain the direct collision gain does not alter an earlier comparison-coefficient requirement.

Later choices of the common stage-size bound, top-budget coefficient, and environment-field exponent do not change this crude exponent C. Immediate descriptor counts depend on relative slack, fixed support bounds and fixed walk length. The averaging walk gap is uniform in both field size and the length of its single free-bit component. Likewise, an enlargement of the fixed identifier-length coefficient only changes the polynomial-evaluation collision term in Lemma 10.1 to at most $2c_{\mathrm{ID}}/B^2$; increasing the fixed minimum $B \ge c_{\mathrm{ID}}$ bounds it by $2/B$. These enlargements can therefore be absorbed after D_2, D_1, ϵ have been fixed. □

11 Finite numerical queries with bounded suspended storage

The preceding sections define finite estimator tables and bound their statistical errors. We now compute digits of those estimator values. For a fixed table tuple, including its statistical budget, the structural conventions are fixed and each entry has one exact mathematical value. Increasing the digit position requests a later digit of that same value. Each request is a finite deterministic computation.

A query returns one bounded digit. During its evaluation, the space needed by a suspended caller must be bounded by the size of its structural descriptors and a constant times the decrease from its output digit position to the requested operand digit position. An operand digit position may exceed the output digit position by an amount proportional to descriptor size; the structural term also pays for this overshoot. These differences will telescope when arithmetic queries are nested inside graph procedures. Thus the relevant question is which data remain live at each operand request.

11.1 Streams, records, and uniformity

We use integer digit positions $\nu, j \ge 0$. Known binary scales are separated from numerical values, so a normalized value belongs to $[-1, 1]$. The fixed digit alphabet is

$$\mathcal{A}_{\mathrm{dig}} = \{-6, -5, \dots, 5, 6\}.$$

Every digit is encoded by a fixed number of bits. An operand stream $(x_j)_{j\ge 0}$ denotes the exact real number

$$x = \sum_{j\ge 0} x_j 2^{-j}, \qquad x_j \in \mathcal{A}_{\mathrm{dig}}. \tag{11.1}$$

In particular, for its truncation $x^{[K]} = \sum_{j=0}^{K} x_j 2^{-j}$,

$$|x - x^{[K]}| \le 6\, 2^{-K}. \tag{11.2}$$

Redundant signed-digit representations have a long history in arithmetic [5]. The redundancy lets consecutive finite approximations define an exact stream even when they use different truncations.

An arithmetic root has public parameters, its digit position, and a description of its operand interfaces. We require these parameters to be regenerable from the outer controller records without operand or graph queries. They are fixed inputs to each numerical computation; their full binary encodings are not copied at every gate. An operand label specifies its interface and descriptor; it never specifies an unknown vertex address from a previous cursor position.

The following lemma reduces each digit query to a fixed number of residue bits.

Lemma 11.1 (Exact streams from approximation residues)**.** *Suppose $|z| \le 1$ and, for each $\nu \ge 0$, a deterministic integer $N_\nu(z)$ satisfies*

$$|N_\nu(z) - 2^\nu z| < 2. \tag{11.3}$$

Set

$$z_0 = N_0(z), \qquad z_\nu = N_\nu(z) - 2N_{\nu-1}(z) \quad (\nu \ge 1). \tag{11.4}$$

Then $z_\nu \in \mathcal{A}_{\mathrm{dig}}$ and these digits represent z exactly. For any fixed integer $q \ge 4$, each output digit is determined by the required N_ν residues modulo 2^q.

Proof. Writing $e_\nu = N_\nu - 2^\nu z$ gives $z_\nu = e_\nu - 2e_{\nu-1}$ for $\nu > 0$, hence $|z_\nu| < 6$; the weaker alphabet bound $|z_\nu| \le 6$ is convenient throughout. At place zero, $|N_0| < 3$, so $|N_0| \le 2$. The partial sum through place T is $2^{-T}N_T$, which tends to z by Equation (11.3). Finally, reduction modulo 2^q is injective on the possible signed digits because $2^q > 12$. Thus the signed difference can be recovered from the two residues without forming the full integers N_ν. Concretely, for $\nu > 0$ compute $(N_\nu \bmod 2^q) - 2(N_{\nu-1} \bmod 2^q)$ modulo 2^q, then use the fixed finite table which sends each occurring residue to its unique representative in $\mathcal{A}_{\mathrm{dig}}$. At $\nu = 0$, decode the residue of N_0 in the same way. This final decoding has constant storage and introduces no comparison of unknown exact real numbers. □

The integer N_ν is always defined before taking its residue. Different places may use different truncations or finite circuits. Only determinism and Equation (11.3) are required; consistency between the finite approximations is supplied by Equation (11.4). In particular, when a digit at ν uses $N_{\nu-1}$, the latter is evaluated by its own place-$(\nu-1)$ definition.

We compute the required residues by finite Boolean circuits. To bound what survives an operand query, we evaluate those circuits from paths of child choices and regenerate their absolute addresses as needed. Here $d \ge 1$ measures the size of the immediate structural descriptors; in the application it is the current group's slack.

The local record of a suspended numerical query contains every local bit that must be retained across an operand query: its binary gate path, return modes, pending bounded results, stored descriptor choices, and any explicitly specified retained word. The input and master choice, the common vertex cursor, and one reusable scratch area are shared facilities. Temporary gate addresses, digit places and scan counters may occupy scratch while they are being computed. Any such data needed after a recursive graph query must either be in the charged local record or be reconstructible from the root parameters and that record. An operand-dependent word retained across the query is charged even if it was originally formed in scratch.

11.2 The numerical interface

We state the composition result with its syntactic hypotheses. They concern finite arithmetic recipes.

Application to graph procedures requires a separate certificate. The semantic descriptor domains and structural validity conventions are fixed independently of the requested digit position; selected operands already have defined digit interfaces; reciprocal arguments lie in the stipulated positive intervals; and root parameters remain recoverable after lower calls. Digit-position-dependent truncations and comparison circuits may change N_ν, but not its target recipe value. Lemmas 12.11 and 12.12 verify these conditions for the estimator procedures. An incoming order statistic over the whole vertex universe has no prescribed $2^{O(d)}$ descriptor bound and uses the separate construction of Lemma 12.7.

The theorem uses the following terms. The *recipe dependency depth* of a recipe is the largest number of operations on a dependency path through it, ending at an already supplied operand interface and counting each indexed sum, average or order statistic as a single operation whatever the depth of its internal tree. The internal depth of an indexed sum, average or order statistic is accounted for separately by the order-statistic and sum lemmas below.

A *discrete operand* is a supplied operand of a recipe that returns a discrete domain, validity or branch decision, a bit or a bounded symbol. Its definition does not depend on the requested numerical digit position, and it is requested at digit position zero. The theorem bounds the records retained above this request; the implementation of the operand is a separate obligation. A *comparison with a fixed margin* is a discrete decision computed inside a recipe by a finite-margin comparison with known operand magnitude bounds $2^{\kappa d}$ and prescribed buffer widths at least $2^{-\kappa d}$, for a fixed κ. It uses $O(d)$ normalized digit places, with approximation and tie conventions fixed from the structural parameters, independently of the requested digit position; its structural starting digit position is zero. The test of Lemma 11.9, with a threshold $u \geq \rho \geq 2^{-\kappa d}$, accepts every value greater than $2u$ and accepts only values greater than u.

A *public coefficient leaf* is a leaf whose normalized numerical value is determined by the public root parameters alone, without any operand value; its digit procedure must satisfy hypothesis 8 below. Known rate and field coefficients and walk probabilities are examples (Remark 11.10), whereas the coefficients formed in Lemma 11.6 from an operand prefix are not.

The *suspended local record* at an operand request is the local record retained across that request, as described above. The *intrinsic local record length* is the total length, at any instant, of records used to evaluate the recipe itself, including its internal arithmetic calls and retained words. Its maximum therefore counts both active and suspended records within the recipe. It excludes the shared facilities described above and records inside external operand, structural or graph procedures, whose storage requires the separate graph certificate.

Theorem 11.2 (Uniform numerical interface)**.** *Fix the digit alphabet, positive reciprocal intervals, and constants bounding the following recipe class. A recipe at slack $d \geq 1$ satisfies the following conditions.*

1. *It has a fixed number of stages built from bounded arithmetic operations, fixed piecewise formulas using maxima, minima and clipping, reciprocals on the fixed positive compact intervals, indexed finite order statistics, and sums or averages followed by known rescalings and bounded clipping.*
2. *Each order statistic has a public semantic list length and an $O(d)$-bit rank index; absent descriptors have prescribed constant dummy values, and out-of-range requests return a prescribed normalized constant.*
3. *An empty sum is zero; an average has a public nonempty domain.*

4. *Binary scale exponents have magnitude $O(d)$, and terms and intermediate outputs have known range bounds $2^{O(d)}$.*
5. *The stage and descriptor descriptions are generated from the root parameters by index computations using $O(B + d + \nu)$ scratch and no operand queries, and the total recipe dependency depth is bounded independently of d.*
6. *Each indexed domain has size $2^{O(d)}$ and descriptors of length $O(d)$.*
7. *Discrete domain and validity decisions are discrete operands or comparisons with fixed margins with their own fixed precision conventions.*
8. *Public coefficient leaves have exact normalized digit procedures using $O(B + d + \nu)$ shared scratch and making no operand or recursive graph queries.*

Then each normalized recipe value has a deterministic exact $\mathcal{A}_{\mathrm{dig}}$-stream. For its digit at digit position ν there are uniform constants a, b, c such that every requested operand digit j satisfies

$$j \le \nu + cd, \qquad \text{total suspended local binary record length} \le ad + b(\nu - j). \tag{11.5}$$

An output code bit satisfies the same bounds. A discrete operand is requested at digit position zero and has suspended local length at most $ad + b\nu$. The maximum intrinsic local record length is $O(d + \nu)$.

All gate, digit and descriptor addresses can be regenerated from root parameters and the saved local records using $O(B + d + \nu)$ shared scratch and no recursive graph calls. When $d, \nu = O(B)$ this is $O(B)$ scratch. If a public rational coefficient has $O(B)$-bit numerator and denominator that can be generated in $O(B)$ scratch, its queried digits in this digit position range also require only that shared $O(B)$ scratch.

The proof in Section 11.8 combines the primitive record bounds below. Their common coefficient on the digit position decrement makes those decrements telescope; the multiplication and reciprocal constructions establish this bound for their long intermediate calculations.

11.3 Evaluation from regenerated addresses

In particular, reconstructing an absolute digit place may use a binary counter in scratch, while the suspended record stores only the choices needed to recover that place. The next lemma makes this distinction precise for the finite circuits used below.

Lemma 11.3 (Evaluation from regenerated gate addresses)**.** *Consider a finite Boolean circuit of bounded fan-in, with a specified output gate. Conditional gates may evaluate their selector first and then only their selected branch. Suppose the type and immediate predecessors of a gate, or its operand label and digit place if it is an input gate, can be computed from the root parameters and a path of child choices from the output gate. Suppose these computations use $O(B + \nu + d)$ scratch bits and make no operand or graph queries. Then depth-first evaluation retains $O(1)$ bits per gate on its current path, and requires no saved absolute gate address per gate. Its suspended record above an operand input has length at most a constant times the length of that path, plus any explicitly retained root-local record.*

Proof. At a gate, save its next-child or return mode and the already computed Boolean input, if any. At a conditional gate also save the selected branch bit. The bounded fan-in makes this a constant-size record. To locate the next child, scan the saved path from its root and regenerate successive gate descriptions in scratch. The final description determines the next child choice. On return, its Boolean result is combined with the saved bit and the local record is popped. All

temporary addresses and scan counters are discarded before an operand query; a later scan recreates them from the same root and saved choices. Circuit acyclicity gives termination. This is an evaluation of the circuit unfolded into a formula; repeated gates are recomputed, so no array of previously computed values is stored. □

11.4 Truncations, comparisons, and binary arithmetic

Lemma 11.4 (Finite comparisons and bounded elementary operations)**.** *On normalized input streams, a pair average, maximum, minimum, or fixed binary shift with normalized output admits deterministic integers satisfying Equation* (11.3)*. Their residue procedures, for any fixed q, have the following properties. There are constants C_0, C_1 such that an operand place j obeys $j \le \nu + C_0$, its suspended local record has length at most $C_0 + C_1(\nu - j)$, and the intrinsic local space is $O(\nu+1)$. Fixed finite compositions of these normalized operations have the same bounds, with enlarged constants. Fixed Boolean operations on returned digit codes, including sign decoding and conditional selection of code bits, add only constant local overhead.*

Proof. First consider comparing two truncations through $K = \nu + h$, where h is a sufficiently large fixed integer. Their difference digits have magnitude at most 12. Scanning from place zero to K, update a signed prefix by $p \leftarrow 2p + \delta$. While $|p| \le 12$, retain p exactly. Once $p > 12$ or $p < -12$, retain only the corresponding absorbing sign state. This sign is correct for every finite continuation: the remaining suffix, in the current scale, has magnitude strictly less than 12. The state space is finite, so each transition has a fixed bounded-fan-in Boolean circuit. The sign at the end determines the exact order of the two finite truncations. The path from that sign bit to an input at place j has length $O(K - j + 1) = O(\nu - j + 1)$.

Let v_ν be the average, maximum, or minimum of the two truncations, and put $N_\nu = \lfloor 2^\nu v_\nu \rfloor$. Each operation is 1-Lipschitz for the sup norm on its operands. Consequently $|v_\nu - v| \le 6\,2^{-K}$, where v is the corresponding exact output. Choosing h so that $6\,2^{-h} < 1/2$ gives $|N_\nu - 2^\nu v| < 3/2$.

To obtain $N_\nu \bmod 2^q$, only a fixed number of low places of the selected truncation are needed. More explicitly, at scale 2^K, terms of input place $j \le \nu - q$ are multiples of 2^{q+h}. Modulo this modulus the remaining $q + h$ places suffice. For an average with integer input prefixes A_K, B_K at scale 2^K, use $N_\nu = \lfloor (A_K + B_K)/2^{h+1} \rfloor$. Its residue needs modulus 2^{q+h+1} and one additional retained place: only places $j \le \nu - q - 1$ can be omitted. The finite number of remaining digits can be added and shifted by a constant-size Boolean circuit. A maximum or minimum first uses the comparison bit to select these residue bits. Its long comparison branch has the path bound proved above, and its residue branch reaches only places within a fixed distance of ν. A fixed shift changes these places by a fixed amount. Fixed normalization factors are handled in this way whenever an intermediate range exceeds $[-1, 1]$.

The topology of the comparison chain and the low-residue circuits depends only on the root digit position, the fixed margins, and the gate path. Regenerating a place by replaying that path needs no saved binary place counter at each transition. Lemma 11.3 therefore gives the suspended-space bounds. Their longest paths have length $O(\nu + 1)$.

For composition, use one common coefficient of $(\nu - j)$ for the finite primitive list. Increasing such a coefficient by Δ requires increasing its constant term by at most ΔC_0, because the operand overshoot is at most C_0. With these common coefficients, the digit position differences telescope along the composed path. The finite number of constant terms remains a constant. □

We will also use the following explicit Boolean arithmetic facts. A carry bit in binary addition is described by a generate/propagate pair (g, p). Composition of adjacent intervals is the associative

operation

$$(g_2, p_2) \circ (g_1, p_1) = (g_2 \vee (p_2 \wedge g_1),\ p_2 \wedge p_1).$$

Balanced trees of these pairs give every carry in $O(\log(W+1))$ depth for W-bit words. A balanced tree adding R such words modulo 2^W has depth

$$O\big(\log(R+1)\log(W+1)\big). \tag{11.6}$$

The trees can be padded by zero words to powers of two, so their gate addresses are computable by ordinary index arithmetic. For counts of R Boolean bits one can do better. In one column, three bits satisfy

$$a + b + c = (a \oplus b \oplus c) + 2\big((a \wedge b) \vee (a \wedge c) \vee (b \wedge c)\big).$$

Apply this constant-depth circuit independently in every column, placing the second output in the next column. Thus three equal-width rows become two rows of the same total integer value; a carry is not propagated through the columns in this round. In $O(\log(R+1))$ rounds only two rows remain. The word width is $O(\log(R+1))$; a final carry computation gives total depth $O(\log(R+1))$. These constructions use bounded fan-in throughout.

11.5 Multiplication by minimum-index bands

Grouping products by their smaller input index leaves a gap between the output digit position and the requested input digit positions; that gap pays for the retained circuit path within each band.

Lemma 11.5 (Multiplication)**.** *For normalized exact input streams x and y, multiplication has all the conclusions of Lemma 11.4. In particular, its suspended storage at an operand place j is $O(1) + c(\nu - j)$, despite the potentially long product expansion.*

Proof. Fix q and write $A = 6$. Partition ordered pairs (i, j) of nonnegative places into a zero band, where $\min(i, j) = 0$, and positive bands

$$\mathcal{P}_D = \{(i, j) : D \le \min(i, j) < 2D\}, \qquad D = 1, 2, 4, \ldots.$$

Set $h_0 = h_\circ$ for the zero band and $h_D = h_\circ + \lfloor D/4 \rfloor$ for positive bands. The fixed integer $h_\circ$ will be chosen below. Retain in band D only pairs with $i + j \le \nu + h_D$, and let

$$S_D = \sum_{\substack{(i,j)\in\mathcal{P}_D \\ i+j\le\nu+h_D}} x_i y_j 2^{\nu+h_D-i-j}. \tag{11.7}$$

For the zero band the same formula uses its defining pair set. These are mathematical integers.

At any fixed diagonal $i + j = s$, a positive band contains at most $2D$ ordered pairs. Indeed the smaller place has D possibilities, and an off-diagonal pair has two orientations. The zero band has at most two pairs. Thus, in units of $2^{-\nu}$, the total omitted product terms have absolute sum at most

$$E_{\text{omit}} \le 2A^2 \sum_{D\in\{0,1,2,4,\ldots\}} (D+1)2^{-h_D}. \tag{11.8}$$

The geometric summation of the diagonal weights gives this bound also when the truncation threshold lies below the first possible diagonal. Both the series in Equation (11.8) and $\sum_D 2^{-h_D}$ converge. Choose $h_\circ$ sufficiently large that their displayed combined error bound, including coefficient $2A^2$, is less than $1/2$.

There are only finitely many nonempty retained bands. For a positive one, $2D \le \nu + h_D$, whence

$$D \le \frac{4}{7}(\nu + h_\circ). \tag{11.9}$$

The left side of the nonemptiness condition grows monotonically faster than its right side along the dyadic bands. We may therefore stop at the last band satisfying this inequality, retaining empty bands within the finite prefix if necessary.

Starting with the most distant retained band, define integers recursively by

$$F_D = S_D + \left\lfloor 2^{h_D - h_{D^+}} F_{D^+} \right\rfloor, \qquad N_\nu = \left\lfloor 2^{-h_0} F_0 \right\rfloor, \tag{11.10}$$

where D^+ is the next band, and the last missing tail is zero. One internal floor, divided back to scale 2^ν, changes the sum by less than 2^{-h_D}. Induction down the bands consequently shows that the difference between $2^{-h_0}F_0$ and the sum of all retained terms in scale-ν units is at most $\sum_D 2^{-h_D}$. Equation (11.8) and the choice of $h_\circ$ make the error before the final floor less than $1/2$. The final floor costs less than one more unit. Therefore Equation (11.3) holds for $z = xy$.

We next implement Equation (11.10) using residues. At band D use modulus 2^{q+h_D}. An integer term in Equation (11.7) with $i + j \le \nu - q$ is a multiple of this modulus and can be omitted from the circuit. It remains present in the mathematical integer S_D. Only the diagonals

$$\nu - q < i + j \le \nu + h_D \tag{11.11}$$

need to be processed, giving $O((D+1)(h_D + q + 1)) = O((D+1)^2)$ candidate pairs.

Signed floor shifts are well defined on the required residues. For $h' \ge h$ and $a - b = k2^{q+h'}$,

$$\left\lfloor \frac{a}{2^{h'-h}} \right\rfloor - \left\lfloor \frac{b}{2^{h'-h}} \right\rfloor = k2^{q+h}. \tag{11.12}$$

This identity holds for all signed integers a, b. Thus a next-band residue modulo $2^{q+h'}$ supplies the correctly floor-shifted tail modulo 2^{q+h}. Apply the same observation to the final shift by h_0 to obtain $N_\nu \bmod 2^q$. In the finite binary circuit, this shift just discards the lowest $h' - h$ bits of the canonical $(q + h')$-bit residue. The displayed identity proves that this is valid also when the underlying integer is negative.

For completeness, the finite gate structure can be indexed without storing an absolute operand place at each gate. For a positive band enumerate triples (r, s, o), where $D \le r < 2D$, $\nu - q < s \le \nu + h_D$, and $o \in \{0, 1\}$. The triple supplies $(r, s - r)$ if $s - r \ge r$ and $o = 0$; it supplies $(s - r, r)$ if $s - r > r$ and $o = 1$; otherwise it supplies a zero term. This enumerates the relevant pairs exactly once. For the zero band use $r = 0$ with the same orientation rule. Negative candidate places are invalid and supply zero. The offsets $r - D$ and $s - (\nu - q + 1)$ have $O(\log(D + 2))$ bits. They and o are obtained from paths in padded binary selection trees. The full places are then computed from the root digit position in scratch.

Each nonzero term is a constant-bit signed product $x_i y_j$, shifted into a word of width $W_D = q + h_D = O(D + 1)$ and interpreted modulo 2^{W_D}. A bit of this word is a fixed Boolean function of the two bounded digit codes, the bit index and the known shift; negative products use their ordinary residue, equivalently sign extension before reduction. Add these words and the shifted tail using Equation (11.6). A band contributes depth $O(\log^2(D + 2))$. Bit selection for the shifted tail is wiring and does not require evaluating a full tail word.

The sum of these depths through the bands up to D is $O(\log^3(D + 2))$, hence at most $C(D + 1)$ for a fixed C. For an original input digit in a positive band, its partner place is at least D. Equation (11.11) gives

$$j \le \nu + h_D - D \le \nu + h_\circ - \frac{3}{4}D. \tag{11.13}$$

Equivalently, $\nu - j \geq \frac{3}{4}D - h_\circ$, so the full path cost through these bands satisfies

$$C(D+1) \leq C\left(1 + \frac{4}{3}h_\circ\right) + \frac{4C}{3}(\nu - j).$$

This gives $C_0 + C_1(\nu - j)$ with fixed constants. The zero band has constant depth and only constant overshoot. All its gate descriptions, term coordinates, bit positions and tail shifts are computable by replaying paths and doing integer index arithmetic from ν and q. Lemma 11.3 converts these depth bounds to suspended storage. Finally, Equation (11.9) bounds every intrinsic path and local word width by $O(\nu + 1)$. □

11.6 Reciprocals and the early-word convention

A prefix through roughly half the output digit position makes the remainder in a first-order reciprocal expansion quadratically small. Releasing that prefix before later input queries keeps its long word from surviving a call with little remaining digit position gap.

Lemma 11.6 (Reciprocal on a positive compact interval)**.** *Fix constants $0 < a \leq b$. On inputs $x \in [a, b]$, a reciprocal, after a fixed known normalization, has the same operand-position, suspended-record, and intrinsic-space bounds as in Lemma 11.4. Its local record may contain an $O(\nu + 1)$-bit word only while querying input places $j \leq \lfloor \nu/2 \rfloor + O(1)$.*

Proof. The input can first be normalized by a fixed binary scale. It is enough to prove the result with a fixed digit alphabet and fixed positive interval; all constants below may depend on a, b. Choose a fixed power of two C large enough that the desired output $z = 1/(Cx)$ lies in $[0, 1]$ and that the coefficients below have absolute value at most one. Put

$$k = \lfloor \nu/2 \rfloor + c, \qquad x^0 = x^{[k]}, \qquad e = x - x^0,$$

where c is a sufficiently large fixed integer. By Equation (11.2), $|e| \leq 6\,2^{-k}$. Choosing c first ensures $x^0 \in [a/2, 2b]$ and $|e| \leq 1$ at every ν. Increasing C if necessary gives bounded coefficients

$$\alpha = \frac{1}{Cx^0}, \qquad \beta = -\frac{1}{C(x^0)^2}.$$

The exact algebraic identity

$$\frac{1}{Cx} = \alpha + \beta e + \frac{e^2}{C(x^0)^2 x} \tag{11.14}$$

shows that the linear expression $T_\nu = \alpha + \beta e$ differs from z by at most a fixed constant times 2^{-2k}. Choose c large enough that

$$2^\nu |T_\nu - z| \leq \frac{1}{4} \quad \text{for every } \nu \geq 0. \tag{11.15}$$

Clipping T_ν to $[-1, 1]$ cannot increase this error; denote the clipped expression by $\widehat{T}_\nu$.

For this fixed original ν, the coefficients α, β are exact rational numbers. Their digits are produced by integer arithmetic, as described below. The suffix e has the exact stream with digit zero through k and digit x_j thereafter. Thus Lemmas 11.4 and 11.5 provide exact streams for $\widehat{T}_\nu$, using only a fixed number of normalized operations. Explicitly, form the product βe, then the pair average $u = (\alpha + \beta e)/2$, clip u to $[-1/2, 1/2]$ by a maximum and a minimum, and finally apply the fixed shift by 2. Every exported intermediate value is normalized, and the final value is $\widehat{T}_\nu$. Forming this expression introduces no additional reciprocal operation.

Let $P_s(\widehat{T}_\nu)$ be the integer approximation furnished by those operations at place $s = \nu + 3$, and define

$$N_\nu(z) = \left\lfloor P_{\nu+3}(\widehat{T}_\nu)/8 \right\rfloor . \tag{11.16}$$

The cutoff k and the coefficients remain those of the original ν, not those of s. Equations (11.3) and (11.15) give

$$|N_\nu(z) - 2^\nu z| < 1 + \frac{2}{8} + \frac{1}{4} = \frac{3}{2} < 2.$$

To obtain this integer modulo 2^q, ask for the approximation $P_{\nu+3}$ modulo 2^{q+3} and use Equation (11.12).

It remains to justify storage in the coefficient computations. The signed numerator of x^0 in units 2^{-k} has $O(k+1)$ bits. Form it by querying places $0, \dots, k$ and updating the integer prefix. The numerator, a place counter, and all data that must survive the next query are part of the local record. To return a coefficient digit of place r, numerator squaring and integer division require $O(k+r+1)$ bits. Here $r \le \nu + O(1)$, because the enclosing expression has fixed depth and constant overshoot. After the input prefix is obtained, these integer computations make no further operand queries. Standard binary long multiplication and division, performed by bounded loops with their partial products or remainder, use only this linear number of bits. For $r > 0$, computing floors of the coefficient scaled at r and $r-1$ gives its exact difference digit; at $r = 0$ use the place-zero floor alone, as in Lemma 11.1. Clipping is unnecessary because the coefficients were normalized in advance.

Every original-input query during this formation has $j \le k \le \nu/2 + O(1)$. Hence

$$\nu + 1 \le C_2 + C_3(\nu - j) \tag{11.17}$$

for fixed constants, at all such queries. The enclosing gate path also has length $O(\nu+1)$, so the complete suspended local record satisfies the required bound there. Equivalently, the coefficient branch is charged against the original reciprocal digit position ν, not against its possibly small coefficient digit place r.

On a suffix branch with $j > k$, the coefficient word is absent. The Boolean evaluation asks for a coefficient digit at a separate leaf, returns its bounded code bits, and discards the coefficient word. If the other child of a gate subsequently asks for a late suffix digit, only the bounded pending bits and gate choices are retained. The path bound from the preceding arithmetic lemmas therefore gives $O(1) + c(\nu - j)$ also on this branch. Repeated coefficient requests rebuild the word. The parameter k and the choice of coefficient are regenerated from the original ν and the saved gate path; no earlier cursor address is required. All intrinsic records have $O(\nu+1)$ bits. □

The coefficients in this proof are operand-dependent, even though they are rational once x^0 is known. They therefore do not qualify as public coefficient leaves. A coefficient request first forms its charged early prefix, then uses scratch for integer arithmetic, returns only the requested bounded digit, and releases that prefix before a suffix branch is evaluated. The original reciprocal digit position ν remains regenerable throughout. This order of operations is the lifetime rule used in the suspended-record bound. In particular, $j \le \lfloor \nu/2 \rfloor + c$ gives the explicit payment $\nu + 1 \le 2(\nu - j) + 2c + 1$ for the early word.

11.7 Indexed order statistics, sums, scales, and discrete operands

Lemma 11.7 (Indexed finite order statistics)**.** *Fix $\kappa > 0$ and $d \geq 1$. Consider at most $2^{\kappa d}$ normalized values, indexed by descriptors of length at most κd, and an order statistic specified by an $O(d)$-bit index. The semantic list length n is public; the requested index is in $\{1, \ldots, n\}$, or an out-of-range request returns a prescribed normalized constant. Absent descriptors may be assigned prescribed constant dummy values by structural validity predicates. This order statistic has deterministic integer approximations N_ν satisfying Equation* (11.3)*. Their residue procedures have constants a, b, c, depending only on the descriptor bounds, such that an operand digit query at j obeys*

$$j \leq \nu + cd, \qquad L \leq ad + b(\nu - j), \tag{11.18}$$

where L is the total suspended local record length. A structural validity query is assigned digit position zero and has suspended length $O(d + \nu)$. The intrinsic local length is $O(d + \nu)$.

Proof. For $1 \leq k \leq n$, the kth largest of the semantic n-entry list is its $(n - k + 1)$st ascending order statistic, before any extra padding. Pad to a fixed descriptor domain whose cardinality is a power of two, adding entries of value $+1$. These entries do not change any of the first n ascending order statistics. An empty list or an out-of-range request returns its specified default immediately. Replace invalid original descriptors by the prescribed dummies. An operand access evaluates validity first and enters the actual value branch only when valid; an invalid access returns the dummy's digit. Thus a dummy does not require a query to an undefined value. For each valid numerical value use its truncation through $K = \nu + h$ for fixed large h. Order these finite rational values, breaking ties by descriptor index. Compare every pair using the signed comparison of Lemma 11.4. For each descriptor count how many descriptors precede it, then compare that count to the requested rank minus one. Exactly one descriptor is selected. Select its low-residue bits using Boolean AND and OR trees and finally floor to output scale 2^ν. The selected descriptor is internal to this residue computation. It may change with ν; the exact order statistic being approximated is fixed by the semantic list and the requested index.

If all coordinates of two finite lists differ by at most η, their corresponding order statistics differ by at most η. Indeed, at least the specified number of the perturbed entries are no larger than the original order statistic plus η; interchanging the lists gives the reverse inequality. Here $\eta = 6\,2^{-K}$. Thus the selected truncation, followed by the final floor, satisfies Equation (11.3) for sufficiently large fixed h. Descriptor ties affect neither this estimate nor the uniqueness of the finite selection. This argument perturbs the entire padded exact list. Some truncated original entries may exceed $+1$ slightly, but their error, and hence the error of the selected padded order statistic, is still at most η.

Each count has $O(d)$ bits. Carry-save counting as described above, the comparison to the requested rank, and selection among $2^{O(d)}$ descriptors add $O(d)$ Boolean depth above the pairwise comparison bits and selected residue bits. A place-j input to a pairwise comparison consequently has path length $O(d) + O(\nu - j)$; a residue input is within a fixed distance of ν and has path length $O(d)$. The paths to validity predicates have length at most the whole circuit depth $O(d + \nu)$, and the predicates themselves are separate discrete operands of digit position zero.

Each gate is specified by its pair of descriptor indices, its counting-tree or selection-tree position, and a comparator or bit position. The paths determine these indices in $O(d + \nu)$ scratch, using only the fixed domain and root parameters. No numerical comparison is needed to determine the circuit topology. Apply Lemma 11.3. The constants in Equation (11.18) can absorb the fixed digit overshoot because $d \geq 1$. □

For the compression order statistics, copies of a previously computed default are ordinary indexed operands: their digit interfaces query the default's exact stream one bounded digit at a time. A slot

that chooses between an entry and the default first evaluates its structural flag and then queries the selected, already-defined stream. Treating this as a defined operand interface leaves no absent value for the order-statistic procedure to evaluate. The required default copies belong to the semantic list; the extra power-of-two padding in the proof is then supplied by the constant $+1$. The structural choice adds only a fixed conditional branch, so the same digit-position and suspended-record bounds apply.

An order statistic over the full incoming vertex universe need not have a $2^{O(d)}$ descriptor domain. That operation uses the terminal configuration procedure in the next section.

Lemma 11.8 (Bounded sums and known scales)**.** *Suppose an immediate descriptor domain has at most $2^{\kappa d}$ terms, all required binary scale exponents have magnitude at most κd, and every intermediate output has a known range bound $2^{\kappa d}$. A sum over an empty domain is zero, and an average is taken over a public nonempty domain. Balanced sums or averages of these terms, followed by the required known rescalings and bounded clipping, satisfy Equation* (11.18) *and have intrinsic local space $O(d + \nu)$.*

Proof. Choose one common known power-of-two range bound for the immediate terms and divide each term by it, so every term lies in $[-1, 1]$. Pad their number to a power of two 2^r, $r = O(d)$, and use a complete binary tree of pair averages. Every internal value stays normalized. Along one path there are r elementary operations. Choose common coefficients in their bounds, as in the last paragraph of the proof of Lemma 11.4. The constant terms sum to $O(r)$ and the differences of successive digit digit positions telescope. Thus the local length is $O(d) + b(\nu - j)$ and the total overshoot is $O(d)$. Restore the common normalization scale only after the tree of pair averages. For a sum, combine this factor with the known padding count 2^r in one binary shift of magnitude $O(d)$. For an average over an original domain of size m, combine the scale restoration with multiplication of the padded average by the public factor $2^r/m$. With minimal padding the latter factor is in $[1, 2)$, so it requires only one normalized public multiplication and a fixed shift in addition to the common scale restoration.

More explicitly, for a shift by 2^v, truncate the operand at $K = \max(0, \nu+v+h)$. If $\nu+v+h \geq 0$, its scaled truncation error in scale-ν units is at most $6\,2^{-h}$. If the scaled output is known to be normalized, floor in the desired output scale. Otherwise, when a bounded clip is required, apply that clip to the scaled truncation before flooring. For the normalized clip to $[-1, 1]$, for example, define the approximation using $\min(1, \max(-1, 2^v x^{[K]}))$; its error cannot increase. This shift-and-clip procedure does not first export an unbounded intermediate as a normalized stream. The low residue requires the constant number of places near $\nu + v$, and a comparison needed for clipping has a constant-state chain through K. Comparisons to the scaled clipping endpoints are equivalently comparisons of $x^{[K]}$ to the known endpoints divided by 2^v; their finite constants and shifted place data are public. When $v \geq 0$ these endpoints belong to $[-1, 1]$; when $v < 0$ the normalized operand is already in the clip interval after scaling, so no clipping comparison is needed. Its path cost is $O(1) + O(\max(0, \nu + v - j))$, which is at most $O(d) + O(\nu - j)$ on an actual operand query. If $\nu + v + h < 0$, zero is already a sufficiently accurate approximation after increasing h, and no operand query is needed. A fixed number of such rescalings and clips preserves the bounds by telescoping. Known non-power-of-two coefficients can instead be handled as public normalized streams and bounded multiplication. □

Lemma 11.9 (Finite comparisons with fixed margins)**.** *Let $d \geq 1$, let s and u have known magnitude bound $2^{\kappa d}$, and suppose $u \geq \rho \geq 2^{-\kappa d}$. There is a deterministic finite test that accepts every $s > 2u$ and accepts only $s > u$. It uses $O(d)$-place digit queries after normalization, with intrinsic and suspended local lengths $O(d)$. Its result is independent of any subsequent numerical output digit position.*

Proof. Compute finite rational approximations $\widetilde{s}, \widetilde{u}$ with individual errors at most $\rho/16$. The known range and lower bound on ρ make $O(d)$ normalized digits sufficient. Accept exactly when $\widetilde{s} > \frac{3}{2}\widetilde{u}$, using a fixed convention at equality. If $s > 2u$, the left side minus the right side is greater than $u/2 - 5\rho/32 > 0$. If $s \leq u$, it is at most $-u/2 + 5\rho/32 < 0$. The finite comparison and all previous arithmetic have the bounds already proved. The test chooses its precision from d and ρ, not from an enclosing numerical digit position. With its structural starting digit position defined as zero, its operand requests have $j = O(d)$; increasing a in Equation (11.18) pays also for the term $-bj$. □

11.8 Proof of the numerical interface

Proof of Theorem 11.2. The primitive constructions define deterministic integers N_ν. Lemma 11.1 turns them into exact streams. When composing primitives, their operands are these exact streams; numerical approximation errors are not accumulated along the recipe as if they were statistical errors. Instead, each primitive's deterministic N_ν is an approximation to its own exact mathematical value. The construction uses only residues of N_ν and $N_{\nu-1}$ and a constant number of output code bits, which adds fixed overshoot and fixed local overhead.

For the record bound, choose one common positive digit position coefficient b for the finite primitive and stage types. In any bound $L \leq a_0 d + b_0(\nu - j)$ with $j - \nu \leq c_0 d$, replacing b_0 by $b \geq b_0$ is justified by replacing a_0 by $a_0 + (b - b_0)c_0$. Thus the same b can be used everywhere. There are a bounded number of stages, each with overhead $O(d)$; within a balanced sum or average the elementary overheads have total $O(d)$ by Lemma 11.8. Along any path the intermediate digit position differences telescope, giving Equation (11.5) and total overshoot $O(d)$. The same sum, terminated at an internal arithmetic operation rather than an operand call, gives the intrinsic bound explicitly as follows. If that operation has current digit position u, its own intrinsic record has length at most $C(d + u)$, while the outer saved portion has length at most $Ad + b(\nu - u)$. Choose the common b also to satisfy $b \geq C$. Their sum is then at most $(A + C)d + b\nu$, because $u \geq 0$. Increasing the stage constants as above makes this same choice of b valid in all local bounds. The early-word case is already included in Lemma 11.6: those retained bits are part of the local record and are paid for at the actual original-input digit position by Equation (11.17).

Structural validity branches are finite branches in this circuit organization, with length at most its full intrinsic bound. They ask the structural operand at digit position zero. Comparisons with fixed margins are separate recipes with structural starting digit position zero and precision fixed by Lemma 11.9; they are not altered when a surrounding value is requested more accurately. Thus no discontinuous comparison of exact reals at unbounded precision is part of the interface.

All circuit constructions were given with padded domains and index rules. A saved gate choice selects an operand, a subtree, a bit of a known-width word, or one of the finite primitive modes. Scanning these choices from the root recovers the current descriptor and digit places by integer arithmetic. The reciprocal's extra records additionally specify its early-prefix formation state. None of these computations refers to an ancestor cursor address or requires an operand value to discover the next gate. Their integers have $O(B + d + \nu)$ bits; repeated scans reuse one scratch area. Lemma 11.3 therefore applies at every Boolean portion of the recipe.

Finally, for a public coefficient P/Q, $Q > 0$, producing a place-r approximation uses integer division of $2^r P$ by Q, with signed floors when needed. The inputs and intermediate remainders have $O(B + r)$ bits. Standard finite binary division and multiplication use that much space, and extracting the low residue or difference digit requires no larger word. All operand bits in this computation are public, so it makes no recursive graph call and can use shared scratch throughout. For $r = O(B)$ its scratch bound is $O(B)$. In contrast, any word whose bits depend on recursive operands and must survive a new operand request has already been charged to the local record. □

Remark 11.10 (Use of digit positions). Theorem 11.2 is a finite-request statement for every ν. Its exact-stream conclusion follows by considering these finite requests separately, and does not require a machine to choose structural domains or port orders at infinite precision. For the actual top-level computation all reached digit positions will be $O(B)$ by the allowance calculation in the next section. Known rate and field coefficients have $O(B)$-bit descriptions, and a fixed rational generator probability raised to a walk length $O(d)$ has numerator and denominator of $O(d)$ bits; these are instances of the public-coefficient provision above.

12 Uniform controller implementation

We now implement the table and vector queries defined in the preceding sections. Theorem 11.2 supplies their numerical operations. We must arrange the recursive graph accesses so that the records retained by all callers fit in logarithmic space at the same time.

We keep one vertex identifier, called the cursor. Consider a product term $\mathcal{F}(x, z)\mathcal{B}_b(z, y)$ from Equation (8.3). For a selected valid product descriptor, a query begins with the cursor at x. The correction port moves it to z and supplies a bounded inverse port. The right factor can then be queried at z, using its own common table tuple; that tuple contains no copy of x. Returning along the inverse port restores x. Arithmetic requests the two factor digits separately, so it need not retain a complete left-factor value while computing a right-factor digit. If the operation instead asks to reach y, the right factor's terminal motion is the last recursive delegation; only local label completion follows it. These are the paths that Lemma 8.6 requires us to realize.

Incoming access reverses a different object: a finite program whose canonical initial states describe the forward row ports. We make the program deterministic with halting success states. The component of a specified success state is then a tree directed toward that state; its canonical initial states are precisely the valid row ports ending there. A depth-first traversal of that tree enumerates them and returns to the success state without saving a second copy of its vertex address. This gives both capped incoming ports and incoming score order statistics. To use the traversal we must define program states and their predecessor queries carefully: working states of a completed lower query are not themselves part of the calling program graph.

The storage bound follows the same distinction. A caller retains its own descriptor choices and numerical path while a lower procedure runs. A slack-d layer pays $O(d)$ structural bits; a query from digit place ν to place j pays for its numerical records through the bound $O(d) + c_{\mathrm{num}}(\nu - j)$ of Theorem 11.2, for a fixed coefficient c_{num}. We assign a decreasing allowance to the finite operation order within a prefix, the move to its preceding prefix of slack $2d$, and the move to a child of budget at most $M - d$. The resulting differences telescope over the physical stack.

12.1 Parameters and interfaces

Fix the input and a master environment. The latter includes the sampling matrices and all the fingerprint seeds. A *table tuple* consists of the estimator type and stage, its statistical budget, scale and usage rates, sampling-matrix roles and suffixes, and its effective environment. The tuple specifies an entire table, rather than an individual row or entry. Usage rates belong to this tuple; the separate integer digit position ν specifies a requested numerical digit. A generator and wrap word is a call-record descriptor used to derive a child's effective environment. It is not an additional semantic parameter of the child table: after this derivation, the child's values, forward port orders and guards depend on the resulting effective tuple, not on the ancestral word which produced it. Distinct walk descriptors are still separate summands with their prescribed weights, even if they induce the same environment transformation. The variants of Lemma 7.3 can be generated without their conditioning vertices. A vector tuple is defined in the same way, omitting the unneeded terminal parameters. All admissible absolute tuples have $O(B)$ bits.

The mutable cursor records a vertex and its stage interpretation. Other shared facilities are the input, the top root parameters, the master environment, a catalytic bit vector of length B, and one reusable scratch area. Its size is $O(B)$ in the actual computation, where the top digit position is $O(B)$; for a general top digit position ν it may use $O(B + M_{\mathrm{top}} + \nu + 1)$ bits. Scratch may be used to perform arbitrary local integer computations in this space. It has no content which must survive a recursive graph operation. Every such operation starts its scratch calculations canonically. Before every input access, its requested position is regenerated in shared scratch. The physical input head stays between the real endmarkers; off-data simulated positions use the base symbol routine from Lemma 2.1. These rules also apply during inaccurate calculations and guarded artificial predecessor probes. The requested coordinate and scan counters coexist only within this pure routine and are discarded before a recursive graph operation. It is harmless to copy an address temporarily inside a scratch calculation, but such a copy is discarded before a recursive graph operation begins.

The finite interfaces to be constructed are as follows. A port is an index in a specified row class; the class may have holes.

1. A *restoring test* returns one bit. A *quantity query* returns one bounded signed digit of a specified value. Both restore the cursor and the catalytic vector.
2. A *terminal motion* starts at a row and a port and finishes at that entry's endpoint. No return address is provided. An *excursion* makes the same visit, performs a prescribed local action there, and returns. The action may use scratch and the catalyst but makes no recursive graph query.
3. A *rotation-map evaluation* maps a valid row port to its endpoint and a bounded inverse port. Its inverse has the same table tuple. On invalid ports the restoring existence test returns false; an exported motion may instead return failure without moving the cursor.

At the final row interface all port alphabets are bounded by constants. At a prefix of slack d, indices and immediate descriptors have $O(d)$ bits. These two statements must be distinguished: an entire terminal path represented by one prefix port need not have a description of length $O(d)$.

A port code may be in range and still name a hole in its class. Its existence test returns false, its quantity query returns the zero digit, and its motion or excursion fails with cursor and catalyst unchanged. Order-statistic operands retain their separately prescribed default or padding values: an absent real port is not a virtual default coordinate.

Theorem 12.1 (Realization of the estimator interfaces)**.** *Fix the construction constants and an admissible top budget $M_{\mathrm{top}} = O(B)$. For every actual task tuple reachable from it, the row, vector and prefix interfaces defined in the preceding sections admit deterministic uniform implementations by restoring queries and cursor motions. The following assertions hold for every master environment, without an accuracy promise on its estimated values.*

1. *Every test or numerical digit query terminates, returns its defined value, and restores the cursor and catalyst. Its result is independent of incoming catalyst and scratch contents. Structural port and order choices are independent of a higher numerical request.*
2. *Terminal motions give the specified endpoints. Their excursions restore the cursor. Capped correction motions are partial rotation maps with bounded inverse ports and safe behavior on invalid ports. Incoming score order statistics compute the stipulated zero-padded order statistics.*
3. *For a root numerical request at digit position ν, all records retained by the call and its descendants have total length $O(M_{\mathrm{top}}+\nu+1)$, and every descendant digit position is $O(M_{\mathrm{top}}+\nu+1)$; the shared facilities use $O(B+M_{\mathrm{top}}+\nu+1)$ space. In particular an actual top request with $\nu = O(B)$ uses total deterministic $O(B)$ space.*

These implementations retain no second vertex address across recursive graph operations.

The proof also constructs every lower operation instance admitted by the finite parameter ranges and permitted parent-to-child recipes. This includes calls made while testing possible predecessor states, whether or not those states occur in a designated execution. The larger construction is needed to compute incoming accesses by inverse traversal.

12.2 Program states and record bounds

A terminal program has a fixed root tuple and root digit position. Its initial configuration is the cursor x, a distinguished initiation tag, and the original row port p in a separate recoverable field. Distinct ports give distinct initiation states. In a scoring program the success local state contains a quantized score; otherwise it has only a success tag. The starting row and port are not root parameters. The program universe thus includes all rows and all ports of this same table. Similarly, a scoring universe at a fixed precision includes all possible quantized scores.

There are two different forms of invocation inside this program. A restoring bit query, or a lower rotation-map evaluation, is an *opaque transition query*: its temporary working configurations are not vertices of the terminal program graph. They are computed by the lower procedure when the transition is probed. A terminal delegation with no bounded inverse port is instead *explicit*: it pushes the lower terminal controller into the local state. Thus the vertices of the program graph comprise the cursor and a finite chain of explicit controllers, but not the working states of opaque transition queries. This distinction will also be used when graphs are traversed inversely.

Definition 12.2 (Records determining descendant parameters)**.** A local record is a delimited binary word with a tag from a fixed finite alphabet. It may contain immediate descriptor choices, bounded port or copy symbols, offsets, Boolean evaluation paths and pending bits, and the early numerical words permitted by Theorem 11.2. Head positions in such words are marked locally. A root tuple, a root operation and its digit position, together with the active records, determine every descendant tuple and digit position by a scratch computation. This computation does not call a graph procedure and does not use a cursor that has been left behind.

The records use delimited encodings, with marked positions rather than an independent absolute stack pointer at every level. Self-delimiting encodings enlarge a nonempty word of length r by

$O(r+1)$ bits. Constant tags also identify a root boundary, the active candidate copy during a neighbor test, and the owner of a pending endpoint action. These tags permit a scratch scan to select the appropriate active view of the records. An inactive copy is not part of a descendant's program state or call recipe.

A root boundary is a constant tag, not a stored copy of an absolute table tuple. The logical root parameters of a nested query are obtained by replay up to that boundary; the temporary absolute tuple is then discarded before another graph query. Thus the phrase "fixed root tuple" specifies which mathematical program graph is being used. It does not allocate a fresh $O(B)$-bit parameter word at every nesting level. Lemma 12.12 supplies the replay procedure.

A predecessor probe may retain edited copies of its local controller state while a lower query runs. We therefore first bound the caller's own records with raw constants a_0, b_0, a', fixed by the recipes and the allowance formula (12.7), before $A_{\mathrm{space}}, D_{\mathrm{space}}$ are chosen. Lemma 12.4 then converts raw bounds into physical bounds with larger constants $A_{\mathrm{space}}, D_{\mathrm{space}}$, which pay for the fixed number of copies made by predecessor probes and are chosen from a_0, b_0, a' in Equation (12.3). A lower query's working space forms a separate physical stack segment; it is not copied into the calling program graph. Keeping the two layers separate keeps the certificate independent of the constants chosen from it at every depth, as used in the proof of Theorem 12.1.

Definition 12.3 (Raw allowance certificate)**.** A raw controller is the caller's own selection, arithmetic, scoring or terminal-path record, excluding the working states of a query it calls. In a terminal program graph an explicit chain contains only its terminal-path controllers and, in a scoring program, its score suffix or completed score word. The interiors of cap traversals, inverse score order statistics, numerical digit queries and rotation-map evaluations are opaque queries; they are never explicitly pushed working graphs in this chain.

A raw controller has an integer allowance $w \geq 1$ and a digit position $\nu \geq 0$. Fixed raw constants $a_0, b_0, a' > 0$ give the following certificate at a call to a lower query or an explicitly pushed raw terminal controller at (w', j):

$$w' < w, \qquad j - \nu \leq a'(w - w'), \qquad \ell \leq a_0(w - w') + b_0(\nu - j). \tag{12.1}$$

Here ℓ counts only the suspended caller record, including its delimiters, return modes and pending data. It does not count the callee's physical working space. Without a pending call its own raw records have length at most $a_0 w + b_0 \nu$. A structural operation has a canonical starting digit position zero; numerical queries used by its finite-margin tests are included in this certificate.

Lemma 12.11 proves this certificate for the actual recipes, before any guards restrict artificial program states. The guards in the next lemma enforce its bounds also on artificial states encountered by predecessor probes.

We use the following finite instruction types.

1. *Local edit.* Read a bounded number of marked local symbols, change a bounded number of symbols and a finite mode, and advance the marked local heads by one position. A push opens a delimiter and writes a tag in a bounded number of steps. A pop is performed only after the child word has been erased down to that tag, also by local steps.
2. *Pure or restoring branch.* Compute one bit, either in scratch without a graph call or by a lower restoring test or digit query, and change a bounded local symbol or mode. A bounded signed digit is handled as a fixed number of bits. A longer selection result is written one bit at a time, with its index stored in the local record.
3. *Rotation-map cursor step.* With its motion tuple and short token already selected, apply one lower partial rotation map, or one elementary base or label map. Its output token and finite

mode are the only simultaneous local changes. The linear-size motion tuple is regenerated from records; it is not written as an opaque payload.

4. *Explicit terminal call.* Create a lower controller with a call recipe determining descendant parameters. Ordinary instructions then execute that controller. On its return, local erasure and the prescribed bounded label completion are performed by the preceding instruction types.

An endpoint action in an excursion may be a pure computation reading or toggling the catalyst. Such actions are not individual edges of a terminal program graph: that graph uses the complete restoring, catalyst-independent predicate which contains the excursion. Its other pure branches depend only on initialized scratch, the input, cursor and permitted tuple data. A value test is never combined with a cursor-changing step. Conditional validity can be a separate restoring branch; a partial rotation-map step may simply have no edge on an invalid domain.

Every record position erased by a local edit is marked before the edit, or is a known end of a delimited word. In particular, an instruction cannot erase an arbitrarily chosen position and then forget its index. Counters, carries and loop heads are manipulated by these local steps. Pure computations and restoring tests can be recomputed to select one output bit; their scratch states and physical input heads are not included in the states whose predecessors are enumerated.

Predecessor probes can reach artificial local states whose stored flags were not produced by a designated execution. The lower query interfaces must therefore be safe independently of those flags. We use the following entry conventions before constructing the graph.

Total calls on invalid ports. An argument is *range-valid* when its tuple, operation type, cursor type, port code and digit position lie in the permitted finite ranges. A range-valid port can still name a hole. Every exported existence predicate is a total restoring test of its earlier defining masks. Every port-dependent quantity query freshly tests presence at structural digit position zero before entering its value calculation. If presence fails, the query returns the zero digit of the zero stream. A forward motion or excursion instead returns failure, with cursor and catalyst unchanged and without performing an endpoint action. An inverse rotation map uses its target-side incoming-count and inverse-rank validation. Excursion validity is checked before building its chain of terminal-path records, not anew at the remote endpoint or during rollback. An endpoint comparison returns false if either freshly tested descriptor is invalid. Wrong types or out-of-range codes are rejected by a pure check before any lower call.

Default queries and order-statistic padding inputs retain their separately specified values: an absent port is not a virtual default coordinate. In particular, incoming score order statistics are padded by zero, whereas a compressed-array order statistic uses copies of its default. Every opaque query tag invokes the canonical entry of one of these total procedures. No permitted call recipe targets an internal continuation after its presence test. A historical validity flag in a caller is never accepted as a certificate of presence. Lemma 12.11 proves that the fresh tests use earlier operations and fit the allowance bounds.

Lemma 12.4 (Guarded local graph)**.** *Suppose a fixed terminal recipe uses the preceding instruction types, has records determining descendant parameters, and obeys* (12.1) *on its designated executions. Suppose all its opaque transition queries are already implemented, are total on admissible arguments, and have the stated safe invalid-port behavior. Their returned bits, success flags, cursor outputs and port tokens are independent of incoming scratch and catalytic contents, and they restore the catalyst. Then there is a finite graph with outdegree at most one, uniformly associated with its root tuple, with these properties.*

1. *Its designated executions are precisely the prescribed terminal executions. Success states have no successor, and initiation and success tags are distinct.*
2. *Predecessor cases, ordered incidences, and rotation-map evaluation are computable with a bounded number of case labels and one cursor. An opaque predecessor query is made only after its local bound has been checked.*
3. *If a transition query at (w', j) is probed while the terminal root has parameters (w, ν), the complete live local portion of this probe, including a fixed number of edited or saved controller copies, has length at most*

$$A_{\mathrm{space}}(w - w') + D_{\mathrm{space}}(\nu - j) \tag{12.2}$$

for fixed enlarged coefficients $A_{\mathrm{space}}, D_{\mathrm{space}}$. Shared facilities and the outside stack are not copied.

The graph and all its guards are independent of an originating row, an outer stack's data, and catalytic contents.

Independence from the outer stack is understood after fixing the logical root tuple: two invocations that regenerate the same tuple define the same graph. Their physical replay histories may differ. The hypothesis on designated executions is proved for the concrete recipes in Lemma 12.11; the guards below extend query safety to artificial states reached while enumerating predecessors.

Proof. Use as potential states the cursor and delimited words having the specified finite tags. Check absolute parameter ranges, descriptor ranges, stage and suffix types, instruction modes, and the allowed parent-to-child recipes. Check the intrinsic word bounds of Definition 12.3. For each explicit chain also check the local bounds along that chain. Before any opaque query, check the bound from every ancestor of its local explicit chain to that query. All absolute parameters, digit positions and lengths needed for these checks are regenerated or counted in scratch. No graph query is needed for a guard. At a state with an unavailable instruction omit that edge.

The designated paths pass the guards by hypothesis: their boundary bounds add along an explicit chain and telescope. Enlarging the coefficients once covers fixed instruction tags and bounded intermediate edits. The resulting graph is finite since its local words have $O(w + \nu)$ bits and its cursor has $O(B)$ bits. It is deterministic since every branch bit and partial rotation map is deterministic. Guards may create additional dead ends, but they cannot create a new path from a canonical initiation to a success state.

For a local edit, predecessor candidates are obtained by guessing the bounded overwritten symbols, preceding finite mode, and bounded head moves, then testing the instruction forward. Marked positions and incremental erasure make the number of cases bounded. For a branch, perform the same fresh restoring query at the unchanged cursor and test the guessed preceding symbols. Incorrect historical branch bits do not resume the interior of a lower procedure: the query is always started again on its regenerated arguments. The guards are checked before the query, including for a candidate predecessor.

For a cursor step, first construct the possible preceding local record by a bounded inverse edit, guessing the bounded preceding token if needed. Before any motion, replay that candidate's active records from the fixed root tuple. Check the instruction and stratum, permitted child kind, budget, scale and suffix ranges, seed permission, numerical digit position and every ancestor-to-query raw length bound. These allocation and recipe checks do not use the predecessor cursor. The lower motion tuple is then fixed by the candidate record and the present output token. Apply the partial inverse at the present cursor and output token. If it succeeds, the returned token and preceding local symbols specify the candidate predecessor. Static checks and token comparisons can be made there.

A guessed preceding port is compared with the actual port returned by the inverse. Any remaining cursor-specific type or domain check is pure at that cursor; a domain condition requiring another restoring query belongs to a separate branch edge. Thus it does not conceal a second high-cost query inside this rotation-map step. To restore the probe, apply the forward rotation map with that same tuple and the actual returned token. No saved copy of the cursor is used. Conversely, an outgoing rotation-map probe checks the proposed target guards after its motion; if a guard fails, or if the probe is only a restoring incidence test, its inverse with the actual output token restores the starting cursor. Both directions have been allowed the same lower-operation cost. No unrelated value query is part of this cursor step.

Label an edge by its finite instruction case and bounded erased symbols. Its forward instruction determines a canonical reverse case code. An inverse probe accepts only that code. This removes any duplicate descriptions of the same predecessor edge without comparing two retained addresses. The finite case codes give a fixed local order and a bounded incidence token; a forward edge knows its reverse token.

For the space assertion, first ignore saved probe copies. Adding the boundary inequalities along the current explicit chain gives the bound $a_0(w - w') + b_0(\nu - j)$. A current state and a candidate predecessor differ in length by a constant: this follows from the bounded edit used to *construct* every candidate, before it is known to be an actual predecessor. Consequently retaining or editing both costs only a fixed multiple of that bound, with a constant adjustment absorbed by the strict allowance decrease. More explicitly, write $\Delta w = w - w' > 0$ and $\Delta\nu = \nu - j \geq -a'\Delta w$. For a fixed copy factor k and additive bounded metadata cost h, choose

$$D_{\text{space}} \geq kb_0, \qquad A_{\text{space}} \geq ka_0 + (D_{\text{space}} - kb_0)a' + h. \tag{12.3}$$

Then $k(a_0\Delta w + b_0\Delta\nu) + h \leq A_{\text{space}}\Delta w + D_{\text{space}}\Delta\nu$. This also handles a negative digit position decrement. These coefficients are uniform over all finite instruction cases.

The constants a_0, b_0 describe the raw skeleton uniformly over the whole recipe family. They have not been replaced by the physical probe constants $A_{\text{space}}, D_{\text{space}}$ in a lower skeleton's guards. Fix the maximum finite copy factor k and metadata cost h and choose $A_{\text{space}}, D_{\text{space}}$ once. A lower compiled query contributes a new physical stack segment when it runs; its traversal state and saved copies are not words in the calling graph. Thus this estimate copies raw records, not another $A_{\text{space}}, D_{\text{space}}$-sized physical segment.

Only the local explicit chain is saved or edited. A nested query has its own root boundary and appends its local portion below the existing stack; its program graph excludes the caller's saved copies. A tag selects the candidate chain for parameter replay, ignoring an inactive saved copy. Therefore a later neighbor probe does not copy the older outside portions again. Finally, all guards and forward case codes use just the fixed tuple and its permitted records. Imposing the same seed-visibility convention as the underlying recipe preserves this independence even at synthetic states. □

12.3 Depth-first traversals and incoming operations

The rotation–edge-flip traversal below is the forest permutation of Cook and McKenzie [17, Proposition 1, p. 387], with a sentinel to include an isolated root. Traversal of configuration trees also underlies space-preserving reversible simulation [20]. The traversal below recovers the endpoint at its success halt, giving incoming enumeration without a saved endpoint address. Lemma 12.4 bounds the live records of its neighbor queries.

Lemma 12.5 (Depth-first traversal)**.** *Let a finite deterministic graph of the form in Lemma 12.4 have a specified canonical success halt. Its undirected component is a tree directed toward that halt. There are mutually inverse next-mark and previous-mark operations on its traversal, where the marks are the root sentinel and one distinguished occurrence of each canonical initiation in the component. These operations require the current program state and bounded incidence control, but no saved root or traversal-length counter.*

Proof. Following the unique successor from any node in the undirected component reaches its sink. Indeed a directed cycle in a graph of outdegree at most one cannot be joined by a path to a sink: at the first connection the unique outgoing edge is already used toward the cycle. Equivalently, on an undirected cycle all its vertices must use their outgoing edge on that cycle, so no sink lies in its component. The same argument excludes two sinks in one component. Thus the component is a finite tree with the given unique sink.

Take the incidences of its edges, with an additional sentinel incidence at the root. Edge flip exchanges the two incidences of an edge and fixes the sentinel. At each vertex cyclically rotate its incidences in the fixed local order. Write R for rotation and F for edge flip; the traversal step is $F \circ R$, and its inverse is $R^{-1} \circ F$. This agrees with Cook and McKenzie's left-to-right product convention for their rotation and flip. On a tree with an edge their proposition gives one cycle, also obtained by inserting the two incidences of each new leaf into the old cycle. To check the sentinel modification, insert a sentinel s between two consecutive root incidences a, b in the rotation. The old traversal step $a \mapsto F(b)$ is replaced by $a \mapsto s \mapsto F(b)$ because $F(s) = s$; all other steps are unchanged. Thus this insertion still gives one cycle. For an isolated root, take the single fixed sentinel.

At a canonical initiation mark its unique successor incidence; mark the sentinel at the root. Each initiation is therefore counted once. Iterating the traversal successor to the next mark always terminates, since the finite traversal contains the root mark. Iterating its inverse to the preceding mark is the inverse operation. A success tag and the sentinel token identify the root in this component; its address never has to be compared with an earlier address. Constant-degree adjacency and edge flip are provided by Lemma 12.4. □

Lemma 12.6 (Capped incoming ports)**.** *Let a fixed row class have a constant port alphabet, distinct entries within each row, and a deterministic terminal program which maps its valid row ports to their endpoints. Assume the program is of the form in Lemma 12.4. For every fixed K_c there are restoring tests and mutually inverse rotation-map evaluations for precisely those class entries whose endpoint has at most K_c incoming class entries. The inverse ports belong to $\{1, \ldots, K_c\}$. No additional endpoint address or path without a bounded inverse port is held across these operations.*

Proof. Give invalid row ports a distinct failure execution. The canonical initiations in a success-root component then correspond exactly to valid class entries at that endpoint. Class validity is tested before using a forward initiation as a traversal starting point.

Start at a valid initiation mark. Go forward toward the sentinel, counting other initiation marks, stopping either at the sentinel or after K_c other marks. Store the number of mark jumps, at most $K_c + 1$, and restore the start by that many inverse jumps. Do the same in the backward direction and restore again. Reaching K_c other marks certifies excess. Otherwise, let a and b be the numbers of other initiations before the sentinel in the forward and backward directions. The total is $1 + a + b$; accept exactly when this is at most K_c. The initiation's rank after the sentinel in forward order is $r = b + 1$. A forward rotation-map evaluation goes to the sentinel in $a + 1$ mark jumps and returns the inverse port r.

From a canonical endpoint halt, count forward marks until returning to the sentinel or encountering $K_c + 1$ initiations. In the latter case restore the sentinel by the bounded number of inverse jumps and reject. In the former case the total $n \le K_c$ is known. For an inverse port $r \in \{1, \dots, n\}$, go forward by r mark jumps to its initiation and return the row port stored in that initiation state. Invalid ranks are rejected at the sentinel. A halt with no preimages has $n = 0$ and is harmless. Wrong vertex types can be rejected before creating a halt state.

Every unsuccessful inverse request therefore leaves its endpoint unchanged: an excessive incoming count is undone before failure, and an invalid rank is detected at the recovered sentinel. For a successful request the returned row port is the one in the reached initiation, not a guessed port. This is the token used by a later forward restoration in Lemma 12.4.

These are inverse maps by the common traversal order. Existence tests make the corresponding visits and reverse them, so they restore the cursor. All counts, ranks and incidence tokens have a fixed alphabet. The terminal program graph has the common table tuple as its root; neither direction fixes the starting row or port as an external parameter. Consequently the two directions use exactly the same graph. Restoring lower queries restore the catalyst, and the traversal itself does not otherwise change it. □

The incoming scores need not have bounded support in a column, so a bounded incoming-port list does not suffice to compute their order statistic. We instead give the terminal program a success state containing both the endpoint and a rounded score. A traversal at that halt counts the entries with that rounded score; successive traversals visit the scores in decreasing order. The endpoint and score belong to the current program state. During a traversal they may be replaced by an earlier state of the score computation, and the traversal recovers them when it returns to its unique halt. Thus neither is retained separately while an earlier state requests a score digit. The next proof combines this observation with an assembly order that stores only a short suffix before a late digit query.

Lemma 12.7 (Incoming score order statistic)**.** *Let a constant-size forward row class cover every positive entry of a bounded nonnegative score table, with distinct entries in each row. Suppose pre-incoming score digits are restoring queries and the class has a terminal program with the preceding interfaces. For each fixed positive integer k, the incoming kth-largest value, with zero padding, has exact redundant digits computed by a restoring procedure. Its local record above a place-j score query at output approximation digit position ν is $O(1 + (\nu - j)_+)$, with a fixed overshoot $j \le \nu + c$, in addition to the structural allowance costs. A complete $O(\nu + 1)$-bit score word is retained only over structural calls.*

Proof. Normalize the score range to $[0, 1]$, using a fixed public factor if necessary. At approximation place ν, choose a fixed extra margin c. For the digit truncation $v^{[\nu+c]}$ define the integer

$$q_\nu(v) = \mathrm{clip}_{[0,2^\nu]}\left(\left\lfloor 2^\nu v^{[\nu+c]} \right\rfloor\right). \tag{12.4}$$

The digit tail has absolute value at most a fixed constant times $2^{-\nu-c}$. Choose c so that this is less than $2^{-\nu}$. Clipping is nonexpansive and fixes $2^\nu v$, so $|q_\nu(v) - 2^\nu v| < 2$. The same margin gives $q_\nu(0) = 0$: a truncation of the zero value has scaled magnitude less than one, so flooring and nonnegative clipping give zero. Since the forward class contains every positive score, enumerating its entries and padding by zero therefore gives the order statistic of the whole quantized score column.

Use a forward terminal program which first tests the entire class and membership domain at its canonical initiation, sending invalid inputs to failure. Its success *configuration* contains the endpoint in the cursor and $q_\nu(v)$ in its local state. Before making its final structural motion, assemble the

score digits from least to most significant. Put $K = \nu + c$. In units 2^{-K}, the word before the place-j query is

$$Q_{j+1} = \sum_{r=j+1}^{K} v_r 2^{K-r}, \qquad Q_j = v_j 2^{K-j} + Q_{j+1}.$$

Since $|v_r| \le 6$, we have $|Q_{j+1}| < 6\,2^{K-j}$. Thus the signed suffix, the update carry, and a self-delimiting offset from K use $O(1 + K - j)$ bits at that query. The output digit position reconstructs K; it is not copied into the suffix word. After assembly, division by 2^c, flooring, clipping and loop setup are pure local computations. The complete word is then held only through structural motion and local clearing to success. Thus all its graph-query boundaries satisfy the claimed record bound. The success word is part of the graph state, not a parameter of the graph. At this fixed ν the graph includes every allowed quantized value.

At the requested endpoint, initialize the success state at $q = 2^\nu$. Traverse that root component and count canonical initiations, saturating at the desired fixed rank. Continue to the sentinel even if the saturated count has already been reached. At the sentinel the same endpoint and quantized value have been recovered; no other success halt occurs in that component. If the cumulative count has not yet reached the requested rank, decrement the value field by pure local steps and traversal the next success component. The value field itself is the loop variable. No separate copy of it survives the traversal. The saturated count and control bits have constant size. At value zero apply the stipulated zero padding.

If the desired rank has been reached, use the value field only after the traversal has returned to the sentinel. The digit position ν is a root parameter throughout; the loop value q is not. Consequently an inverse step which reaches an earlier partial-suffix state replaces the current score state by that suffix state, and need not retain the final q or the endpoint address alongside it. Unique-sink recovery supplies both again when the component traversal ends.

This computes the integer order statistic N_ν of the values $q_\nu(v)$. A uniform perturbation of a finite list changes any order statistic by at most the same perturbation, including after zero padding. Thus N_ν has error less than 2 as an approximation to 2^ν times the desired order statistic. Keep only the fixed number of residue bits needed to form its redundant digit, clear the other local data without a graph query, and return at the unchanged endpoint. Computing $N_{\nu-1}$ uses its own quantization convention; the difference-digit construction needs only the bounded errors, not nested quantizations.

During a traversal the live program state may be a partial suffix state preceding a high-place query. It then contains that short suffix, rather than a saved copy of the final score. Lemma 12.4 therefore also gives the required bound for inverse neighbor queries. All root components are finite, including isolated success states, and the decreasing finite value loop terminates. Fresh lower procedures and the traversal preserve the catalyst. □

12.4 Certificates for the estimator recipes

We next verify the hypotheses of the graph construction for each of the actual recipes. These certificates concern syntax, storage and dependencies; they do not use the statistical accuracy of an estimated entry. Thus they apply also when its numerical value is inaccurate.

Lemma 12.8 (Ordered terminal paths and excursions)**.** *Every path without a bounded inverse port required by the estimator has the following ordered form: restoring selection and validity tests; local label maps and, if necessary, a left-factor rotation-map evaluation; further restoring tests at that midpoint; at most one final terminal delegation; and then only local label completion. An excursion to its endpoint and back, for a pure local action, has records determining descendant parameters. It preserves the same allowance and digit position bounds as this ordered path, up to fixed factors. No path without a bounded inverse port is retained over a new high-cost query at its endpoint.*

Proof. For a direct summand the path is a local base path, a projected child path, or a child row port. A projection stores its bounded symbol before removing it and appends the prescribed destination symbol after the child terminal motion. For a product the left factor is a correction with rotation-map access. Its inverse token is stored at the midpoint; right-factor validity and weight queries are restoring there. The right terminal path is then the last delegation. An old prefix port delegates to that preceding prefix; after its return it requires no new graph query. These are exactly the path cases of Lemma 8.6.

There are two compiled continuations of this same terminal syntax. A one-way continuation clears its path records after performing its prescribed local completions. A visiting continuation retains the immediate choices and inverse tokens on its explicit terminal spine. After the local endpoint action it undoes the bounded completions, undoes the last delegated spine, and then uses the inverse-port token of an earlier motion. Each deeper record is cleared before invoking an earlier inverse map. For a projection, the saved symbol and prescribed append symbol give its local inverse. The identical lower rotation-map tuple and actual returned token give the inverse of a correction move.

This is an induction on terminal delegations. A frame stores only its own immediate descriptor, bounded choices and phase. Its returned child spine is used for local completion and the endpoint action, which make no graph calls. When a further graph query is made during rollback, the deeper spine has already been discarded. Thus the suspended records are those at the corresponding forward boundary, enlarged by bounded phase and inverse-token data. The choices can be re-created or kept on their own frame; no concatenated description of all old prefixes is passed to a new tight query.

In a restoring use, intrinsic constituent validity is checked before the visit. A product can test its right port by crossing the left edge using its rotation map and returning; this test does not use the right terminal path. Outer slice and wrap tests then use a valid intrinsic excursion. Hence none of these validity checks recursively invokes the very bucket or stored list membership it helps to define. The endpoint action's owner is marked in the outer frame, so its tuple can be regenerated by a scratch scan even while a descendant path is retained. The action's result does not alter the selected paths or their undo. □

12.5 Catalytic fingerprint equality

The fingerprint is short, but retaining a $\log B$-bit fingerprint or index at each tight recursive frame would still be too costly. The following exact identity gives a comparison with only a constant number of endpoint excursions. Its use in the space accounting requires the restoring endpoint interfaces of Lemma 12.8. The cancellation of arbitrary initial contents also appears in transparent programs for catalytic computation [9, Section 3]. Here the shared bit vector is included in the ordinary work-space bound.

Lemma 12.9 (Restoring equality test)**.** *Let $V \in \{0,1\}^B$ be a shared vector with arbitrary initial contents. Suppose an excursion to either of two deterministically specified endpoints returns the cursor, and its path and result are independent of the incoming contents of V. Such excursions may use nested procedures which restore V. There is a procedure which xors the bit $\mathbf{1}_{k(v)=k(w)}$ into a local result bit, restores V, and needs only constant local data across four endpoint excursions.*

Proof. Starting with result bit r_0, perform the following four excursions: flip $V_{k(v)}$; xor $V_{k(w)}$ into the result; flip $V_{k(v)}$ again; xor $V_{k(w)}$ into the result again. The fingerprint is evaluated only at the endpoint currently visited. If V^0 is the incoming vector, the first read equals $V^0_{k(w)} \oplus \mathbf{1}_{k(v)=k(w)}$ and the second equals $V^0_{k(w)}$. Thus the final result is $r_0 \oplus \mathbf{1}_{k(v)=k(w)}$ and both flips restore V^0.

For nesting, use induction on the well-founded procedure order. A nested call sees some arbitrary incoming vector, restores precisely that vector, and returns a result independent of it. Hence the four outer endpoint paths and their fingerprints are the same on repeated visits, even between the two outer flips. The terminal evaluation, read, or flip is a local operation with no further structural call. Only a phase and a result bit must survive across the excursions; a fingerprint index is recomputed at each visit in shared scratch. The result bit being accumulated is not used to choose either endpoint path. These facts give the induction invariant and the claimed local storage bound. □

For $d > 2h_*$, exact bit-by-bit ID comparison is retained. A bit position then takes $O(\log B) = O(d)$ suspended bits, with constants allowed to depend on the already fixed ϵ. For small slack, Lemma 12.9 avoids that per-frame index. The shared vector itself occupies B bits and, by the lemma, may be reused by all nested comparisons. This algebra establishes restoration; the ordered paths of Lemma 12.8 supply the excursions; the operation order is verified in Lemma 12.11.

The endpoint action evaluates the fingerprint with the invoking comparison's effective seed, even when the excursion visits a lower-budget table. It makes no further access query and does not choose or alter the excursion. Thus the lower path remains independent of later seeds. Lemmas 12.10 and 12.12 implement this separation for all restoring accesses, including inverse queries.

Lemma 12.10 (Implemented endpoint comparison)**.** *At slack d, comparisons of two valid candidate or old-list endpoints are restoring, catalyst-independent tests with $O(d)$ suspended structural control, in addition to the costs of their lower endpoint visits. They use no saved endpoint address. Their choice between an exact comparison and the designated fingerprint comparison is independent of a surrounding digit position.*

Proof. The two descriptors occupy $O(d)$ bits. By Lemma 12.8, each endpoint can be visited and restored for a local bit or hash action. If $d > 2h_*$, compare the $O(B)$ identifier bits in order, retaining only the bit-position counter and bounded flags. Its counter uses $O(\log B) = O(d)$ bits, since $h_* = \lfloor \epsilon \log_2 B \rfloor$ with a fixed positive ϵ and sufficient minimum padding.

Otherwise use the four visits in Lemma 12.9: flip the first fingerprint's bit, xor the second fingerprint's bit into a result, and repeat. The final result is fingerprint equality for every incoming catalyst and that vector is restored. Fingerprint evaluation and the indicated vector-bit action are pure endpoint operations in scratch. No fingerprint index is saved across another visit. Nested structural procedures already restore the catalyst and give results independent of its contents, by their lower operation order. The outer descriptors and all path choices ignore the accumulated result bit. The two repeated visits therefore use identical paths. This proves the same independence and restoration for the completed comparison, supplying the induction hypothesis at the next layer. The comparison mode is determined by M, d and the public block boundaries, not by the requested numerical place. □

Operation	Data retained across a lower call	Local space bound
Structural tests and list selection	Immediate descriptors, loop indices, counts and flags	$O(d)$
Numerical recipes	Gate paths, pending bits and permitted early operand words	$c_{\mathrm{str}} d + c_{\mathrm{num}}(\nu - j)$
Ordered terminal paths	One frame's choices, phase and bounded inverse tokens	$O(d)$ per immediate frame
Incoming score order statistics	A partial score suffix; the full word only over structural calls	$O(1 + (\nu - j)_+)$ plus structural costs
Graph transition probes	A fixed number of local raw-state copies and bounded incidence data	$A_{\mathrm{space}}(w - w') + D_{\mathrm{space}}(\nu - j)$

Table 2: Suspended records at a lower call. The bounds exclude shared facilities and the callee's physical state. Terminal-path frames are charged separately along their explicit chain; graph probes copy only that local raw chain.

For the block comparison in Section 10, the comparison calculation replaces designated fingerprint comparisons by exact endpoint equality. Endpoint comparison is an opaque restoring bit query in the terminal program graph: its internal states are not graph vertices. The exact comparison can therefore use a longer bit-position counter without enlarging the guarded raw graph states. It still returns its bit and restores the cursor and catalyst. Only the actual algorithm must satisfy the tight physical-space bound; the comparison calculation supplies comparison values and paths for the error analysis.

Lemma 12.11 (Concrete recipe certificates)**.** *All actual estimator operations admit records determining descendant parameters and a fixed finite ordering of operation strata. At a prefix of slack d, each ordinary layer has $O(d)$ structural control, and a numerical path to an operand at digit position j has suspended record*

$$c_{\mathrm{str}} d + c_{\mathrm{num}}(\nu - j), \qquad j - \nu \le c_{\mathrm{dig}} d, \tag{12.5}$$

with fixed coefficients. Beyond the fixed strata of this prefix, a call is to the preceding prefix of slack $2d$, or to an estimator of budget at most $M - d$. The final row and incoming wrappers add a fixed number of strata at $d = 1$. Every designated terminal execution obeys the corresponding boundary bounds before guards are imposed.

Proof. Table 2 summarizes the records to be charged. The list below runs from exported operations down to their dependencies; the verification constructs these operations in the reverse order. All constants depend only on the fixed construction constants and the base machine.

Finite strata within a group. For clarity, a permissible ordering, from an exported operation down to its dependencies, is:

1. compressed entries, default and list-port operations;
2. ordinal remapping, compression and exception selection;
3. size and first-signal order statistics of intermediate entries;
4. intermediate domain flags and entry or bucket-sum values;
5. first-representative and fingerprint-match predicates;
6. endpoint comparisons of outer-valid paths;

7. outer-valid visits, then outer path validity;
8. intrinsic visits, constituent tests, factor digits and ordered motions, with the intrinsic and outer-valid subtypes detailed below.

The wording of an interface name does not force a call at the same stratum. For example, the "visit" used to verify outer validity is the intrinsic visit, while the visit used by fingerprint comparison is an outer-valid visit. Arrange these finite subtypes in their indicated dependency order: outer-valid visits use outer validity; outer validity uses intrinsic visits; fingerprint comparisons use outer-valid visits; and bucket predicates use those comparisons. Arithmetic on intermediate values uses only their domain flags and lower matching predicates. This refines the displayed list to a finite acyclic list. Repeated enumeration over descriptors stays in a loop of its layer, rather than creating one recursive frame per descriptor. Between these layers Theorem 11.2 gives (12.5); structural loops add $O(d)$ bits. All remaining path calls go to the preceding prefix or to the lower-budget factors of Lemma 8.6.

Immediate paths and their values. By Lemma 8.6, a group has at most $2^{O(d)}$ immediate descriptors: term and array part, scale and budget offsets, a variant word and wrap choices, bounded copy symbols, and a bounded number of factor ports. All factor budgets are at most $M - d$. A descriptor is stored literally in $O(d)$ bits; it never stores a discovered endpoint. Index and range tests are pure computations. Constituent existence tests call only these lower factors. Products cross a left edge using its rotation map for right-factor tests or digits, retaining a bounded inverse port; they return after each such query. Factor digits can therefore be requested separately by an arithmetic evaluation path. No assembled left weight remains over a late right-factor digit. Source vector factors are queried at the current source, with the prescribed local projection if needed. Their numerical coefficients and the factor products use a fixed arithmetic recipe.

For terminal motion the right path without a bounded inverse port, or the direct child path, is the last delegation. This is the ordered path of Lemma 12.8. To avoid a validity cycle distinguish intrinsic constituent existence, intrinsic motion or excursion, and outer validity. Outer validity first checks the intrinsic constituents, then uses an intrinsic excursion for its own hit and wrap conditions. A subsequent outer-valid motion may repeat that validity check and then use the intrinsic motion. Each of these is a separate stratum; none tests the updated bucket or stored list. Invalid descriptors return failure or a zero value without relying on a motion's promised inverse.

All rate factors multiplying these terms have logarithmic range $O(d)$: a variable bin has $|\log_2(b/a)| = O(d)$, while f/π_f is bounded. Resetting a source rate to the public stage rate does not multiply the numerical term by f/u; that ratio was a norm cost. It therefore causes neither a large numerical shift nor a new long choice field. The fixed number of bounded-factor products, coefficients and $O(d)$ scale shifts obey Theorem 11.2.

Candidate domain and bucket sums. The descriptor pool is the concatenation of the preceding list positions and this group's unmerged paths, with invalid slots masked. A new descriptor is a representative exactly when it is the first valid new descriptor with its fingerprint and no old valid position has that fingerprint. Old valid positions remain separate. To decide these flags, enumerate the $2^{O(d)}$ descriptors, keeping only the current indices, bounded flags and $O(d)$-bit counts. Each test uses lower validity and endpoint comparison operations. The comparisons use only those unmerged paths or the preceding prefix's already-defined positions. In particular they never inspect the new value of the bucket being defined.

An intermediate entry is the old entry at an old representative, or the previous default at a new representative, plus the sum of all valid matching increments in each array. That sum has a

fixed $O(d)$-bit descriptor domain and boundedly many $2^{O(d)}$ scale factors. It is evaluated by the balanced sum construction of Theorem 11.2, with the preceding domain tests as discrete operands. No high-precision sequential accumulator is held while requesting term digits. The bucket rule preserves distinct actual representatives, including when two old representatives have the same fingerprint. Thus it gives exactly the intermediate array used in Lemma 10.7.

Order statistics, compression, and list selection. The intermediate domain and its values are now earlier inputs. Add enough copies of the default for the size and first-signal order statistics. A finite domain of $2^{O(d)}$ indices suffices: the preceding list has size $2^{O(2d)}$ and the new group has $2^{O(d)}$ descriptors. The default is included as often as needed to represent the unlimited padding in the prescribed order statistics. The order-statistic, max, min, absolute-value and clipping operations of Lemma 9.1 are the finite arithmetic primitives. The ranges are $2^{O(d)}$ by Lemma 9.6; their known normalizing shifts have magnitude $O(d)$.

Here $u_0 \geq \rho_d = 2^{-c_1 d}$. A canonical finite approximation can test a threshold strictly between u_0 and $2u_0$: with error a sufficiently small fixed fraction of ρ_d, accept every size greater than $2u_0$ and accept only sizes greater than u_0. The required digit position, after normalizing the range, is $O(d)$. Fix this digit position and tie convention once for that structural test. Consequently every nondefault compressed position is included, and fewer than n_d positions are selected, as in Lemma 9.1. No default can expose a positive row score: its order-statistic dummies close the cutoff first, by the margins of Lemma 6.4.

Remap a requested list ordinal to its selected intermediate descriptor by enumerating the pool and counting the preceding accepted flags. The loop stores $O(d)$-bit indices and a count. Its only calls are the already-defined intermediate flags and finite-margin selection tests. Compressed entry digits then query that descriptor's intermediate value and the common compression thresholds; default digits use those thresholds directly. A terminal list motion selects its descriptor before delegating to an old prefix path or a new valid path. A descriptor from many prefixes ago is therefore represented by a chain of delegations, not retained as one long word above a tight child.

Presence before value or motion. Ordinal lookup returns a presence bit and a selected descriptor only when its finite enumeration reaches the requested ordinal. It starts with a false presence bit and returns false if enumeration ends first; the false branch supplies no descriptor. Every compressed-entry digit query and list motion branches on this fresh result before using a descriptor. The preceding-prefix lookup has the same convention. A hole denotes no real representative. Order-statistic operands retain their prescribed structural selection of an entry, default, or padding value; they do not infer that value from the zero stream exported at an absent real port.

The final uncapped row access class A and auxiliary class A_I use the bounded final list-position alphabet, allowing holes. Their presence tests use list presence, role-membership masks, and the fixed-precision raw-entry and raw-order-statistic tests of Lemma 6.4. Source membership is checked locally; endpoint membership uses the earlier list or intrinsic excursion with a pure membership action. It does not use the exposed-class motion whose validity is being defined. These tests precede the pre-incoming score formula and use neither its incoming cutoff nor its cap. Every pre-score digit invocation repeats the corresponding class test and returns zero on failure. Other uncapped row-value queries use their support-covering class in the same way. Each such class contains the entire nonzero support, so the extension agrees with the mathematical table. Existence predicates evaluate their earlier masks directly; they do not ask for their own existence. Analytical witness functions without an exported port interface retain their stated full-array values and are not restricted by this convention.

Immediate-path values and visits first use their earlier constituent and, where appropriate, outer-validity tests. Endpoint comparisons likewise test their two earlier descriptors before making visits. A capped correction-value query stops with zero if its forward rotation-map evaluation fails; only a successful move permits a cutoff or column-value query at the endpoint. The inverse rotation map keeps its target-side validator from Lemma 12.6. Incoming order statistics and vector queries return zero outside their prescribed vertex type or membership domain. These checks are repeated at canonical entry, including when the caller is a synthetic post-initiation state with a fabricated validity flag. An excursion checks before constructing its spine; its endpoint action and rollback retain the order of Lemma 12.8.

Place these entry wrappers a fixed number of strata above their validity predicates and value or motion cores. A wrapper at digit position ν calls its structural test at digit position zero and retains $O(d)$ descriptor and control bits; its boundary is therefore bounded by $O(d) + b\nu$. When it subsequently calls a numerical core at digit position j, the test has finished and only bounded status and descriptor data remain, so the bound $ad + b(\nu - j)$ persists after enlarging a. The predicates use the fixed structural precisions already proved. They do not call the score, cutoff, cap, new bucket or new stored list whose value is being checked. The finite stratum list is enlarged before choosing the allowance constants. Recipe guards permit opaque calls only to total entry wrappers, so a synthetic state cannot select a continuation that skips validation. No semantic predicate is evaluated by a guard itself.

These wrappers preserve the represented relations, values and outgoing endpoints. Their insertion may change the traversal ordering of incoming ports; both directions use the same updated graph, so pairing remains consistent. The tests use the same effective tuples and fixed structural digit positions, and restore the catalyst on both outcomes. They require no accuracy promise and add no retained endpoint address.

Final row classes and pre-incoming values. At the final slack $d = 1$, the list size, port alphabet and all threshold gaps are constants. The classes and witnesses in Lemma 6.5 are defined by canonical finite-margin entry and order-statistic tests on the final raw arrays. They can be enumerated without testing positivity of a computed score. Their tests and pre-incoming score digits query only the final buffer and a fixed arithmetic recipe. The same raw-array tuple is used for every row. A normalized bin entry uses its second raw argument, divided by the fixed scaling constant and clipped before the reciprocal. It does not start a new preparation experiment at its endpoint reached without a bounded inverse port.

The broader auxiliary class used to score an incoming cutoff depends only on the corresponding pre-incoming row calculation. Its validity and score queries do not depend on that incoming cutoff or cap. The uncapped row access class A for a correction has the same property. Its canonical terminal program begins with a restoring check of the full uncapped row access class and membership domain, and sends every invalid initiation to failure. Thus Lemma 12.6 counts exactly its valid entries, not merely entries promised valid by an external caller. The auxiliary scoring program has the same initial check. Its score digits are pre-incoming, so Lemma 12.7 invokes only lower forward operations. By Lemma 6.5, discarding an uncapped row access class with more than K_c incoming entries preserves its value.

Final incoming and vector wrappers. Place the capped-pair and inverse-score wrappers above their respective terminal programs, and those programs above their pre-class, selection and pre-score queries. They are separate terminal programs: the cutoff score program does not call this task's cap pairing, and the cap program does not call its cutoff order statistic. A final correction-value

query first completes the rotation-map evaluation. At the endpoint its motion-return data consist only of the bounded inverse port while it queries a cutoff-order-statistic digit or a lower-budget column-vector digit, and then returns. This can be done separately for every arithmetic operand bit. Only numerical path bits and immediate descriptors coexist with a late query. The factor r_i at a projected source is handled in the same way at that source.

A detector trial is an inverse-score wrapper of its lower-budget input row task. Reward terms query source factors locally and cross correction rotation maps before midpoint vector queries. Vector prefix sums use the simpler balanced arithmetic form of the same dyadic schedule. These are all the vector and incoming uses in the additive formulas. There are only a fixed number of final wrapper types. Their short counts and return ports cost a fixed allowance gap. In an inverse score program the saved numerical suffix satisfies the digit position bound in Lemma 12.7; its full word is held only over structural motion. The cap wrapper's counts have fixed size.

Uniform generation and record bounds. Each enumerated object above has a literal $O(d)$-bit index or is a Boolean path in one of the stated arithmetic recipes. Uniform index-generation uses finite integer loops on these indices, including order-statistic comparison pairs and balanced-tree nodes. Constants, field/rate coefficients and walk probabilities have $O(B)$-bit rational descriptions. Their requested digits use integer arithmetic in scratch and make no recursive graph call. In particular, a walk word has length $O(d)$ and a fixed rational generator probability, so its probability denominator has $O(d)$ bits. Rate and field expressions involve fixed powers of $O(B)$-bit integers. The arithmetic theorem's uniformity and public-coefficient hypotheses are thus satisfied.

The finite arithmetic list covers the formulas used here explicitly. A fixed polynomial switch is evaluated on an affine-rescaled clipped argument; positive parts and absolute values use maxima, and a variable clip is a minimum followed by a maximum. The normalizing reciprocals have their arguments clipped to $[1/2, 1]$. The eighth roots used in the error estimates express regularity of the row functions and are not additional numerical operations requested by the controller. Order statistics inside a prefix use the finite descriptor construction above; the unbounded incoming classes use Lemma 12.7. This accounts for the arithmetic primitives.

Structural selection words and loop counters have $O(d)$ bits. The only longer local numerical words are the early operand records of Theorem 11.2 and the just-described score suffix; they obey their explicit digit position-decrement bounds. Late operand calls hold only numerical paths, pending bounded results, immediate descriptors and short inverse tokens. A fixed number of arithmetic and structural layers enlarges the constants in (12.5); its digit position differences add. Ordered terminal delegations add their own immediate records, with the same estimate at each boundary, including on rollback as proved in Lemma 12.8. This verifies the record certificate on the designated executions, independently of any graph guards. □

Lemma 12.12 (Parameter replay and seed visibility)**.** *The records in Lemma 12.11 determine descendant parameters as in Definition 12.2. With respect to their own effective environments, actual procedures at a budget in block b ignore all seed blocks later than b, including in structural orders, graph universes, guards and predecessor probes. Re-querying a table entry or using a port with a corresponding inverse label in either direction uses the same effective table tuple.*

Proof. Start with the fixed top parameters and scan the active records. An estimator edge specifies a task type, a stage change prescribed by the formula, a budget offset, a bin-scale offset if present, a bounded copy choice and sampling-matrix assignment. These determine the next absolute stage, rates and suffixes. A projection appends the stored symbol before the inherited suffix. A fresh

midpoint or vector role uses the specified public stage rate and its prescribed suffix; it does not carry the old source. These are the common-tuple recipes in Lemma 6.5 and Lemma 8.6.

An averaging edge also carries its generator and wrap word. Apply these fixed transformations in order to the current environment. Their descriptors are independent of the indicated endpoint; membership and wrap consistency are tested separately on visits to that vertex. Thus no discarded conditioning address is needed to regenerate the environment. For an old-prefix edge the tuple stays that of the same row calculation. At final assembly a cutoff or column multiplier uses this common tuple, rather than the internal variant which happened to lead to a selected representative. An action-owner tag allows an endpoint-local calculation to replay just to that ancestor tuple.

Numerical paths specify arithmetic operands, gates, digit places and the early-prefix modes of a reciprocal. Their finite index rules are those of Theorem 11.2; an absolute place is recovered from the root digit position and the path or offset, not saved afresh per frame. Descriptor selections which used the cursor were stored explicitly as their bounded choices before movement. No later tuple computation needs to repeat such a selection at an unknown old cursor.

At every step the tuple and the temporary integers have $O(B)$ bits in the actual parameter and digit position ranges. Keep a current tuple in scratch and overwrite it as each record is processed. Repeating the scan reconstructs an ancestor or a selected candidate chain. Delimiters and marked active-copy tags specify which chain is used. This costs one scratch facility, including its scanning counters, rather than one absolute pointer, input-head position or tuple per frame.

A pure endpoint action belongs to its outer invoking predicate and may read that owner's permitted seed. It is excluded from the lower program graph and cannot affect the lower path or rollback. Nested lower queries still restore their arbitrary incoming catalyst contents.

Finally, the only seed read by a new group comparison is its task's own block seed. Descendants do not increase their budget. Walks alter only the already designated earlier free seed blocks and leave the present and later ones frozen. All descriptor and instruction orders are fixed index orders; all range and length checks depend only on public bounds and accessible tuple fields. Give each recipe the syntactic permission to address only those seed blocks at most its budget block in its effective environment. This is a direct range check on a requested seed index, not a condition on the seed's value. A synthetic record cannot change that permission, choose a higher-budget recipe, or bypass the fixed parent-to-child rules. The same replay and checks are used before predecessor queries. Consequently the entire lower-block structure is independent of later seeds. The comparison calculation in Lemma 10.4 uses the same fixed descriptor orders and structural digit positions, with its stipulated replacements of comparison predicates. The error proof uses the seed independence of the actual lower algorithms and of the comparison recipes. □

Remark 12.13 (Effective coordinates and physical replay). The seed assertion in Lemma 12.12 is a statement about a task as a function of its effective environment. For fixed other task parameters, write this function schematically as $F_b(\eta)$. Its values, access orders and guarded program graph use only the sampling matrices and the permitted seed coordinates of η. An ancestor can invoke it at $\eta = \varphi(\sigma)$, where σ is the master environment and φ is a recorded environment transformation. Computing the permitted coordinates of $\varphi(\sigma)$ can read physical master bits whose original block indices exceed b. Those reads derive the effective argument; they do not give F_b a new semantic parameter or permission to inspect later coordinates of that argument.

For a comparison calculation in the current block, the current task's own walks vary the earlier effective seed coordinates and freeze its current and later ones. Same-block exact comparisons omit the current fingerprint, and lower-block procedures obey the local permission rule just proved. This is the coordinate convention used for the slice means in Lemma 10.4. A callback owned by a higher

task remains the separate pure endpoint action described above; it cannot enter the lower graph, change its selected path or affect rollback through catalytic contents.

12.6 The uniform space theorem

Proof of Theorem 12.1. Two linear quantities organize the final accounting. For positive physical-space coefficients $A_{\mathrm{space}}, D_{\mathrm{space}}$, chosen below, define

$$\Phi(w,\nu) = A_{\mathrm{space}} w + D_{\mathrm{space}} \nu, \qquad \Psi(w,\nu) = \nu + a' w, \tag{12.6}$$

where a' is the digit position-overshoot coefficient in Definition 12.3. The decrease of Φ will pay for a suspended physical segment, including its local probe copies. The nonincrease of Ψ will bound every descendant digit position and hence the shared arithmetic scratch.

Let S exceed the finite number of operation strata in Lemma 12.11, including the subdivision of path validation and all final wrappers. Choose fixed integers $K_0 > S+1$ and $K_1 > 2(K_0+S+1)$, increasing them as needed for the local constants. Assign to an operation at budget M, prefix slack d and stratum $s_0 \in \{0,\dots,S\}$ the allowance

$$w = K_1(1+M) - (K_0+s_0)d. \tag{12.7}$$

Final wrappers use $d=1$, with increasing s_0 down their dependencies. For a nontrivial prefix, $1 \le d \le M$; trivial tasks use fixed top strata and terminate locally.

A same-slack descent lowers w by at least d. A preceding-prefix descent to $2d$, even with a reset to an earlier stratum, lowers it by at least $(K_0-S)d > d$. A statistical descent to $m \le M-d$ lowers it by at least $(K_1-K_0-S)d$, since the child allowance is at most $K_1(1+m)$ minus a nonnegative stratum term. Hence every required boundary has $w - w' \ge cd$ for a fixed $c>0$, and all allowances remain positive and $O(1+M)$. The estimates (12.5) and the ordered-path record certificates give Definition 12.3, with common coefficients. When a common digit position coefficient is increased, compensate the allowance coefficient using the bounded digit position overshoot, just as in (12.3). Add a fixed allowance gap for a traversal wrapper's bounded incidence, count and status data.

There are two distinct estimates in this induction. First, Lemma 12.11 certifies the raw caller words with the same a_0, b_0 at every budget; a lower opaque operation contributes its call recipe, but none of its internal working states, to that certificate. Second, Lemma 12.4 converts a copied raw portion to its physical bound with one fixed pair $A_{\mathrm{space}}, D_{\mathrm{space}}$. The induction below appends a lower operation's physical portion; it never substitutes that physical portion into an upper raw graph state. The raw certificate therefore does not depend recursively on the already enlarged physical constants.

Construct the procedures in increasing allowance. The induction covers all recipe instances admitted by the finite range and parent-to-child checks: both designated calls and calls made when probing candidate predecessor records. It does not require such a candidate record to have occurred on a designated execution. Each opaque call still enters the lower procedure at its canonical entry, where its semantic presence tests are performed. At the bottom are local base ports, sampling-matrix and seed evaluation, rational arithmetic without graph operands, and trivial zero tasks. Suppose lower operations are total, have their claimed cursor and catalyst behavior, and obey the local bounds. The finite restoring and numerical recipes of Lemma 12.11, using Theorem 11.2, then compute exactly their specified bit or digit. Their finite loops and lower calls terminate. Endpoint visits use Lemma 12.8, and completed endpoint comparisons use Lemma 12.10. The latter restores arbitrary catalyst contents and has an independent answer, preserving the induction hypothesis. The other recipes make no direct catalyst-dependent choice.

The canonical terminal programs have finite loops and only lower terminal delegations or opaque calls, so their prescribed executions terminate. Their class checks send invalid canonical initiations to failure. The certificates already proved show that all their designated boundaries fit the local bounds. By Lemma 12.12, their parameters and static guards are uniform and independent of a discarded cursor. We may therefore apply Lemma 12.4. Guards remove no designated execution. At synthetic states they ensure that every probed query has an admissible lower tuple and available space; fresh safe lower queries do not rely on the truth of an earlier stored flag.

The finite graph traversals of Lemma 12.5 now implement the higher cap and inverse-rank wrappers in Lemmas 12.6 and 12.7. They terminate on success-root components even if other graph components contain cycles or guard-created dead ends. They count only the canonical initial executions of these programs, so their rotation maps and order statistics are the mathematical ones specified by the tables. By the common root tuple and seed convention the same maps are obtained in either direction and on every re-query. This completes the well-founded construction. An opaque query in this induction denotes its already constructed finite procedure; recursively substituting these procedures gives an ordinary deterministic machine.

The allowance is a runtime parameter of this fixed finite collection of instruction types, rather than an index for input-dependent program text. Operation tags select the finite recipe family; loop counters, gate paths and descriptors determining descendant parameters select its current instance. Thus the induction constructs one uniform interpreter for these instructions.

It remains to add all simultaneously live data. By Lemma 12.4, a traversal root at (w_i, ν_i) with a pending query at (w_{i+1}, ν_{i+1}) contributes at most

$$A_{\mathrm{space}}(w_i - w_{i+1}) + D_{\mathrm{space}}(\nu_i - \nu_{i+1}) = \Phi(w_i, \nu_i) - \Phi(w_{i+1}, \nu_{i+1}) \tag{12.8}$$

local bits, including its fixed number of neighbor-state copies. Ordinary restoring and numerical calls have the same bound after one enlargement of $A_{\mathrm{space}}, D_{\mathrm{space}}$. Inverse traversal copies only its local explicit chain, not older stack segments or the internal configurations of a transition query. Choose $A_{\mathrm{space}}, D_{\mathrm{space}}$ also to dominate the bottom operation's intrinsic linear bound. If the active chain has boundaries $0, \dots, r$, with boundary zero at the top, put $\Phi_i = \Phi(w_i, \nu_i)$. The total local space is at most

$$\sum_{i=0}^{r-1} (\Phi_i - \Phi_{i+1}) + \Phi_r + O(1) = \Phi_0 + O(1) = A_{\mathrm{space}} w_{\mathrm{top}} + D_{\mathrm{space}} \nu_{\mathrm{top}} + O(1).$$

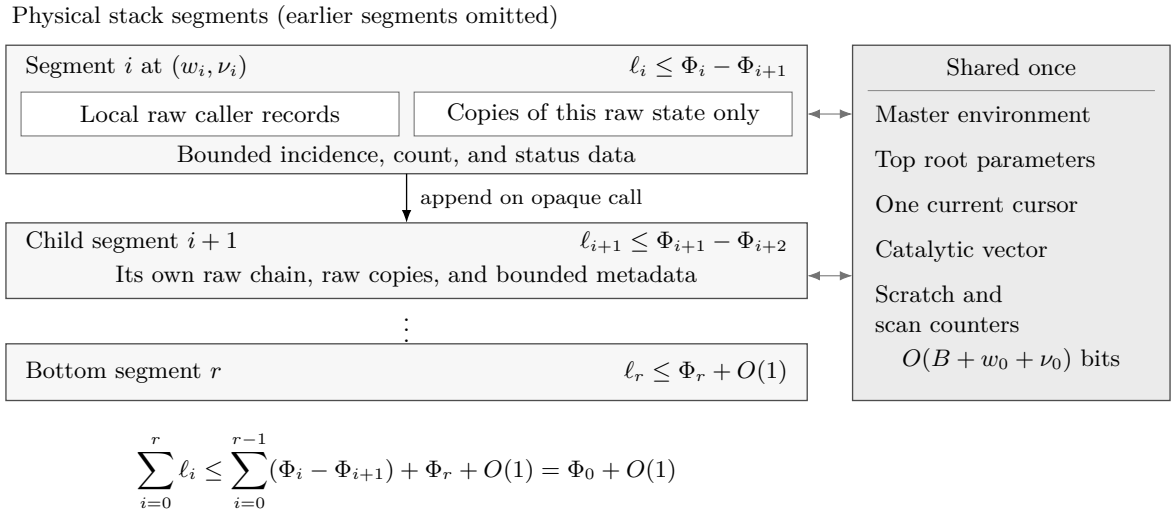
Figure 1 shows which records enter each summand.

The digit position inequality in (12.1) gives

$$\Psi(w_{i+1}, \nu_{i+1}) \le \Psi(w_i, \nu_i). \tag{12.9}$$

Since $\nu_i \le \Psi(w_i, \nu_i)$, all digit positions are $O(\nu_{\mathrm{top}} + w_{\mathrm{top}})$. This also bounds the public scratch calculations in the arithmetic theorem. An uncharged full coefficient or cursor copy cannot persist there: any operand-dependent early word crossing a query was included in its record certificate, while pure public computations have no recursive query. The master environment, cursor, catalyst, scratch and their global scan counters are single facilities. Their total space is $O(B + \nu_{\mathrm{top}} + w_{\mathrm{top}})$.

For exact-stream semantics the induction is valid for each finite requested digit position, allowing scratch to scale with that digit position. Every structural domain uses its fixed finite-margin digit position, so its definition does not change in this argument. In particular the integer N_ν of a score order statistic or arithmetic operation is always evaluated by its own place convention, even when used in a request for the next digit. The actual application has $M_{\mathrm{top}}, \nu_{\mathrm{top}} = O(B)$, and hence all these facilities and active records use $O(B)$ bits. □



$$\sum_{i=0}^{r} \ell_i \le \sum_{i=0}^{r-1} (\Phi_i - \Phi_{i+1}) + \Phi_r + O(1) = \Phi_0 + O(1)$$

Figure 1: Physical stack space, with ℓ_i the length of segment i and $\Phi_i = \Phi(w_i, \nu_i)$. Opaque child work is appended outside raw copies. The proved record certificates and local graph bound, with enlarged $A_{\text{space}}, D_{\text{space}}$, pay for copies and metadata. Shared facilities are counted separately. The displayed copies occur at traversal calls; older physical segments are not copied into the child. Ordinary restoring and numerical segments obey the same difference bound.

13 Fixed-precision approximation and accepting computations

We prove Theorem 3.1: for each fixed machine $\mathcal{M}$ and integer $d \ge 1$, a deterministic logarithmic-space transducer approximates $p_{\mathcal{M}}(x)$ within $(n+2)^{-d}$ in polynomial time, with logarithmic output length. The hierarchy gives a deterministic reward close to the acceptance probability. The estimator's moment bound gives a majority of accurate environments, and total evaluation on every environment permits median enumeration. Section 3 already derives the class equality, promise separation, and unary-precision approximation from this result.

13.1 Choosing constants and finite ranges

The construction has the following order of choices.

1. Fix $H, h_0, s = 8$ and the fixed number of independent sampling matrices. The finite-action gap is uniform in every later modulus and bit length.
2. Choose the correction thresholds and cutoff, then the iteration count k_* from the factorial bound and its resulting uniform magnitude constants. These estimates hold for every column multiplier in $[0, 1]$.
3. Choose D_0 for the entry bound reduction, the routing thresholds, and the finite ladders of entry and cutoff thresholds. The row-cutoff rank K can then be chosen from the row-mass bounds alone.
4. Choose the detector rank J large enough for the incidence bound on extra copies. Choose R_* large enough for overload detection, and finally the incoming rank K_c large enough for correction normalization. The earlier row-cutoff choice does not depend on K_c.
5. Fix the bounded-output constant C_b, the scale cutoff c_0 and precision coefficient K_{prec}, and the prefix mean exponent B_1. Choose the buffer floor and capacity exponents c_1, c_2 and the noise decay K_n so that all buffer sums converge. Choose the rank coefficient A_* of Equation (8.10)

and the walk-length coefficient ℓ sufficiently large for the small constants in the comparison estimation induction.

6. Fix the crude range and propagation constants, then the block gain coefficients D_2, D_1 and a sufficiently small rational block width coefficient ϵ. These choices close the coupled fingerprint induction of Theorem 10.2.
7. Choose the fixed operation-stratum and allowance coefficients and the guard constants in Theorem 12.1. Their finite enlargement does not alter the analytical tables or the distribution of the environment.
8. Choose the linear top-budget coefficient, the field-size exponent, and the minimum padding of B.

Within each step only finitely many strict inequalities between fixed constants are needed. The proofs in the corresponding sections give these inequalities. In particular, the row rank is fixed before the incoming rank, the block constants follow the crude error constants, and the field is chosen after the reachable resource ranges. There is no dependence of an earlier averaging walk gap on that last choice.

The coefficient ϵ in this order is the fixed fingerprint block-width coefficient. Once it has been chosen, increasing the fixed linear top budget increases the number of seed blocks but leaves the averaging walk's gap uniform: all earlier seed bits form one free component in Lemma 7.2. The storage estimate (10.4) therefore remains $O(B)$.

The allowed family in Section 8.1 bounds the stage by L, each inherited suffix length by $L - l$ at stage l, and every reciprocal-rate logarithm by

$$HL + C_{\mathrm{rate}}(M_{\mathrm{top}} - M). \tag{13.1}$$

The closure check after Lemma 8.6 verifies both this finite range and the task-specific inequalities $u \le f$, $a \le K_f f$ and $t \le a$, including stage resets, preparations, detectors and reward calls. The finitely many local operations keep their task tuple and use the stated widths and fixed subranks. The controller guards allow these designated executions.

There is consequently a fixed C_P such that a prime

$$2^{C_P B} \le P \le 2^{C_P B + 1}$$

is larger than all identifiers, makes the radix invertible, and gives $P 2^{-k} \ge 1$ for every rate ever requested. Bertrand's postulate provides such a prime, as it provides the prime of Lemma 10.1. Testing consecutive integers by trial division finds the prime using $O(B)$ space. The search range and running time are $2^{O(B)}$. Field arithmetic and fixed-dimensional rank tests also use $O(B)$ space.

13.2 A dyadic approximation at the specified start

Lemma 13.1 (Conditioning on one start)**.** *Let V be finite, let $W \in [0,1]^V$, and use the sampling matrix indicators X_f at an admissible rate f, with marginal $\pi_f > 0$. Suppose an estimator satisfies*

$$\frac{1}{N}\mathbb{E}\sum_{v \in V}\frac{X_f(v)}{f}|\bar{W}(v) - W(v)|^8 \le C_b^8 2^{-8p}.$$

For any specified $z \in V$ and every $\eta > 0$,

$$\mathbb{E}\big[|\bar{W}(z) - W(z)|^8 \mid X_f(z) = 1\big] \le N\frac{f}{\pi_f}C_b^8 2^{-8p}, \tag{13.2}$$

$$\Pr\big[|\bar{W}(z) - W(z)| > \eta \mid X_f(z) = 1\big] \le N\frac{f}{\pi_f}C_b^8 2^{-8p}\eta^{-8}. \tag{13.3}$$

Proof. Keep only the nonnegative summand at z. Since $\Pr[X_f(z) = 1] = \pi_f > 0$,

$$\frac{\pi_f}{f}\mathbb{E}[|\bar{W}(z) - W(z)|^8 \mid X_f(z) = 1] = \mathbb{E}\left[\frac{X_f(z)}{f}|\bar{W}(z) - W(z)|^8\right] \le NC_b^8 2^{-8p}.$$

Dividing by π_f/f proves Equation (13.2). In particular, choosing one specified coordinate costs the factor N from the integrated norm. Markov's inequality gives Equation (13.3). □

Proof of Theorem 3.1. Apply Lemma 2.1 to $\mathcal{M}$. Set

$$r = d\lceil \log_2(n+2)\rceil, \qquad \Delta = 2^{-r}, \qquad L = \left\lceil \frac{C_0 B + r + 3}{H - 1} \right\rceil,$$

where $|V_0| \le 2^{C_0 B}$ is an explicit configuration bound. Since d is fixed, $r, L = O(B)$. Let z be the copy with every appended digit zero of the start at stage L. Proposition 5.7 gives

$$0 \le p_{\mathcal{M}}(x) - W_L(z) \le \Delta/8. \tag{13.4}$$

This bound holds for every acceptance probability. Choose a fixed C_N large enough to bound all the resulting stage domains by $N = 2^{C_N B}$.

For the final reward task take $a = u = f = b_L$. Its rank satisfies $i_* = O(L+1)$, and Equation (8.10) reads

$$p = M_{\mathrm{top}}/K_{\mathrm{prec}} - HL - A_* i_*.$$

By Lemma 4.1, fix an integer $C_{\mathrm{tail}} \ge 1$ bounding $(f/\pi_f)C_b^8$ throughout the admissible range. Choose the fixed linear coefficient of $M_{\mathrm{top}} = O(B)$ large enough that

$$p \ge r + 3 + \frac{C_N B + \lceil \log_2 C_{\mathrm{tail}} \rceil + 3}{8}. \tag{13.5}$$

The top-budget and field-size choices in the preceding subsection allow this enlargement; Equation (13.1) still bounds every descendant rate. Theorem 10.2 supplies the reward moment estimate. Lemma 13.1, with $\eta = \Delta/8$, now shows that

$$\Pr\big[|\bar{W}_L(z) - W_L(z)| > \Delta/8 \mid X_f(z) = 1\big] \le 1/8. \tag{13.6}$$

Thus the factor N lost on selecting one coordinate has been paid by the statistical precision. Explicitly, the bound in Equation (13.3) is at most

$$C_{\mathrm{tail}}\, 2^{C_N B - 8p + 8r + 24} \le 2^{-3},$$

by Equation (13.5).

We next turn the estimator values into a finite list of dyadic numbers. For each master environment, Theorem 11.2 gives the digits $w_j \in \{-6, \dots, 6\}$ of its one fixed value $\bar{W}_L(z)$. Structural choices do not depend on the requested digit. Put $K = r + 6$, compute

$$A_\sigma = \sum_{j=0}^{K} w_j 2^{K-j}, \qquad a_\sigma = \min\{2^K, \max\{0, A_\sigma\}\}, \qquad v_\sigma = a_\sigma 2^{-K}.$$

The integer A_σ is obtained by starting at zero and applying $A \leftarrow 2A + w_j$ in order, so it uses $O(K)$ bits. By Equation (11.2), the truncation error is at most $6\,2^{-K} < \Delta/8$. Clipping cannot increase

error relative to $W_L(z) \in [0,1]$. Equations (13.4) and (13.6) imply that, on more than three quarters of the environments conditioned on $X_f(z) = 1$,

$$|v_\sigma - p_{\mathcal{M}}(x)| < 3\Delta/8. \tag{13.7}$$

Enumerate the conditional law exactly. Enumerate each field-matrix entry in $\{0, \dots, P-1\}$ and each raw seed bit string, retaining precisely those encodings whose positive-degree columns are independent and for which $X_f(z) = 1$. The retained encodings are equally weighted and give the law in Equation (13.6). In particular, enumerate the original seed strings, not their reduced fingerprint parameters. Different raw strings that induce the same fingerprint remain separate encodings; their multiplicities reproduce the original probability law. At least one encoding is retained: an independent column tuple exists, and its independent constant column can place the designated evaluation in the nonempty retention box. The matrices have fixed dimensions and $\log P = O(B)$; together with Equation (10.4), this gives $O(B)$ bits per master environment and at most $2^{O(B)}$ encodings to enumerate.

Let $E \geq 1$ be the number of retained encodings, found by one pass, and let $h = \lfloor E/2 \rfloor + 1$. Binary-search the integer interval $[0, 2^K]$ for the least a such that at least h retained encodings satisfy $a_\sigma \leq a$. Each count is computed by a fresh pass, recomputing each numerator a_σ. This finds the upper median of the finite multiset, including all repetitions. Since more than three quarters of its members satisfy Equation (13.7), so does its median: fewer than $E/4$ values can lie below the indicated interval, and fewer than $E/4$ can lie above it. The algorithm retains only the current count and search registers; it never stores the multiset. Output the integers (a, K) representing $z_x = a2^{-K}$. Then

$$|z_x - p_{\mathcal{M}}(x)| < 3\Delta/8 \leq (n+2)^{-d}.$$

We account for the memory held during this computation. The master environment, current cursor, catalyst, shared scratch, and suspended controller records together use $O(B)$ bits by Theorem 12.1, because $M_{\mathrm{top}}, K = O(B)$. The outer computation keeps only a fixed number of additional $O(B)$-bit registers: its top parameters and designated start, the environment enumeration index, E, h, the current count, the two binary-search endpoints and candidate, the digit index, and the signed numerator accumulator. Each count is bounded by the number of environment encodings, so it also has $O(B)$ bits. These outer registers remain live while a digit query runs; in particular, both its environment-enumeration state and its numerator accumulator are included. They are present once for the whole computation, not once per recursive level. Every query terminates on every environment, including the inaccurate environments, by the same realization theorem. The output is written only after the median is computed.

Each digit query is a halting deterministic computation with $O(B)$ work bits and physical input heads confined between the real endmarkers. Counting its finite control, work-tape contents, and all work and input head positions gives at most $2^{O(B)}$ complete configurations on its fixed input. A repeated complete configuration would force a deterministic loop, so totality bounds the query by $2^{O(B)}$ steps. There are $2^{O(B)}$ environment encodings, $O(K+1)$ binary-search passes, and $K+1$ digit queries per numerator. Their product is still $2^{O(B)} = (n+2)^{O(1)}$. All finite parameters are generated uniformly, and all constants are fixed for $(\mathcal{M}, d)$. □

13.3 Certified accepting computations

We return to fixed inverse-polynomial accuracy to construct an accepting computation when the machine's acceptance probability has an inverse-polynomial lower bound. For a transducer, the conclusion concerns outputs certified by an accepting run: its output tape must be write-only, so past emitted symbols do not influence later transitions.

Corollary 13.2 (Certified accepting computations)**.** *Fix a randomized logarithmic-space machine $\mathcal{M}$ that halts within a fixed polynomial bound $T(n) \geq 1$ on every input and every coin sequence, and fix an integer $c \geq 0$. There is a uniform deterministic logarithmic-space, polynomial-time transducer that outputs an accepting computation of $\mathcal{M}$ whenever*

$$p_{\mathcal{M}}(x) \geq (n+2)^{-c}.$$

On every other input it outputs either an accepting computation or a failure symbol. If every accepting run of $\mathcal{M}$ emits a valid witness for a specified relation, the transducer can instead output such a witness with the same guarantee.

Proof. Give halted configurations constant self-transitions through time $T(n)$. A configuration and its elapsed time use $O(\log(n+2))$ bits. There is a fixed randomized logarithmic-space continuation machine which, on a valid encoding of (x, v, t), starts from configuration v and simulates the remaining $T(n) - t$ steps with fresh coins. It checks the fixed work-space box and time bound, rejecting malformed encodings and any attempted violation. Its acceptance probability on the reachable configurations is the continuation value $p_t(v)$, and it has a polynomial worst-case time bound in its input length.

Write $\gamma = (n+2)^{-c}$. Choose a fixed exponent d large enough that Theorem 3.1, applied to the continuation machine, approximates every queried value within $\delta \leq \gamma/(8T(n))$. For an explicit choice, fix $C_T \geq 1$ and an integer $a_T \geq 0$ such that $T(n) \leq C_T(n+2)^{a_T}$ for every n, and take

$$d \geq a_T + c + 3 + \lceil \log_2 C_T \rceil.$$

If m is the continuation input length, then $m \geq n$, and

$$(m+2)^{-d} \leq (n+2)^{-d} \leq \frac{(n+2)^{-c}}{8C_T(n+2)^{a_T}} \leq \frac{\gamma}{8T(n)}.$$

The extra fields have logarithmic length. We supply them virtually from saved configuration and time registers, so the approximation and those registers together use logarithmic space.

Starting at the initial configuration, at each time estimate the continuation value of every positive-probability successor and choose one with maximum estimated value, breaking ties by a fixed port order. Since $p_t(v)$ is the weighted mean of the successor values, the chosen successor v' satisfies

$$p_{t+1}(v') \geq \max_w p_{t+1}(w) - 2\delta \geq p_t(v) - 2\delta.$$

On an input satisfying the stated lower bound, after $T(n)$ choices the terminal continuation value is at least $\gamma - 2T(n)\delta \geq 3\gamma/4 > 0$. A terminal continuation value is zero or one, so this computation accepts.

First run this construction without writing output. If its terminal state rejects, output the failure symbol. Otherwise repeat the same deterministic choices and stream the configurations through the original first halting state, or the symbols emitted by the simulated transducer. The artificial self-transitions after halting are omitted from the output. This second pass follows the

same accepting computation. Consequently no rejecting computation is output, even outside the probability promise.

At any time the current, candidate, and best successor configurations, the time counter, a constant number of dyadic scores, the virtual input address, and the approximation workspace occupy a constant number of $O(\log(n+2))$-bit records. The finite successor lists have constant size, and there are only $T(n)$ steps in each pass, so the total running time is polynomial. Each approximation has a deterministic guarantee at its queried configuration; no simultaneous accuracy event for all adaptive queries is needed. □

14 The effective compiler

The probability-approximation theorem fixes the source machine. We now construct one effective map from source descriptions and supplied resource bounds to deterministic simulations with explicit bounds.

Theorem 14.1 (Effective simulation with explicit bounds)**.** *There is a terminating algorithm which, given a finite randomized machine description $\mathcal{M}$ and positive integers a, b, emits a deterministic machine $\mathcal{D}_{\mathcal{M}}$ and positive integers $K_{\text{space}}, H_{\text{time}}, c$ with the following guarantee. If, on every input of length n and every random tape, $\mathcal{M}$ halts within $(n+2)^a$ steps, uses at most $b\lceil\log(n+2)\rceil$ work bits, and has acceptance probability at most $1/3$ on no inputs and at least $2/3$ on yes inputs, then $\mathcal{D}_{\mathcal{M}}$ decides the same language. On each input, that same execution uses at most $K_{\text{space}}\log(n+2)$ work bits and $H_{\text{time}}(n+2)^c$ elementary bit operations.*

The compiler's own running time need only be finite, and its output exponent may depend on the source.

We prove Theorem 14.1 by specializing one fixed deterministic program. First we construct a randomized interpreter $\mathcal{U}$ that enforces polynomial-time and logarithmic-space bounds on every input. Theorem 3.1 supplies a deterministic machine $\mathcal{A}_{\text{lib}}$ that separates the inputs on which $\mathcal{U}$ accepts with probability at least $2/3$ from those on which it accepts with probability at most $1/3$. Both $\mathcal{U}$ and $\mathcal{A}_{\text{lib}}$ are fixed before the compiler receives a source description $\mathcal{M}$ and bounds a, b.

When the source satisfies its supplied resource bounds, padding places each encoded computation within $\mathcal{U}$'s caps without changing its acceptance probability. The generated machine runs $\mathcal{A}_{\text{lib}}$ on that padded input, generating each requested symbol from the real input. We then compute the output bounds $K_{\text{space}}, H_{\text{time}}, c$ from the generated finite transition table and the fixed library data.

Universal deterministic estimation has a substantial antecedent in Pyne, Raz and Zhan [25, full version, Theorem 5.1]. They construct one deterministic algorithm that, on input $(1^n, \mathcal{B})$ with $n \geq 1$ and $\mathcal{B}$ an ordered branching program of length and width at most n, approximates its acceptance probability to error less than $1/n$. For every space-constructible $S(n) \geq \log n$, this single algorithm uses $O(S(n))$ space exactly when $\mathsf{prBPL} \subseteq \mathsf{SPACE}[O(S(n))]$. Here the preceding sections establish the separator used by one fixed library, and we compute explicit resource bounds for its source-specific specialization.

All quantities in this section are local to the compiler construction. In particular, P_{pad} is a padding exponent, d_{code} is a program length, $\ell_n = \lceil\log_2(n+2)\rceil$, and N_{pad} is a virtual input length. They do not denote the statistical budgets, field modulus, or domain sizes used in the preceding sections. We use the usual counted-cell convention for work tapes: traversed writable cells count toward space, whether or not they contain a nonblank symbol. The finite control and the read-only input have their standard treatment.

14.1 One fixed interpreter with enforced resource bounds

Fix a binary coding of ordinary finite transition lists. One concrete choice uses a fixed record grammar and the field encoding $w \mapsto 1^{|w|}0w$. Lists specify the finite state set, tape and alphabet declarations, initial and halting states, blanks and endmarkers, and transition records. A transition record lists its tested state, scanned symbols and coin pattern, followed by its next state, written symbols and local head movements. The grammar has fixed nesting depth; its fields are actual listed data, not recursively executable descriptions. State and symbol IDs are canonical binary integers used only as names. The first matching transition is used; a missing transition causes rejection. Wildcards for ignored coins can equivalently be expanded into a finite explicit list.

Write z for a program in this coding and $d_{\mathrm{code}} = |z| \geq 1$. The coding permits at most d_{code} work tapes and a block of at most d_{code} fresh fair bits per simulated transition. Every ID has at most d_{code} bits. A harmless padding field can enlarge the program to meet these inequalities. All tape movements are local. Work coordinates are measured from their initial origins; the ordinary model has one input head and one head per work tape. For a source input x of length n, let $E_{d_{\mathrm{code}}}(x)$ encode each of its symbols by its ID padded to exactly d_{code} bits.

Preserving the supplied source bounds. An ordinary source machine $\mathcal{M}$ can be converted effectively to this format by renaming its states and symbols and listing its transitions. Its tapes are represented directly, so a source transition retains its work writes and head movements and counts as one interpreted transition. The conversion does not pass through a slower single-tape simulation.

Under the optional one-way read-only random-tape convention, the source may reuse its current random bit. Store a cached bit and a validity flag in finite control, initially invalid. At each transition obtain one fresh bit. Use it if the cache is invalid; otherwise use the cached bit and discard the fresh one. Execute the source transition, then retain the used bit if the random head stayed and invalidate the cache if it advanced. This samples a newly visited random cell lazily on its next use. It preserves the joint distribution of the source run and needs no initialization transition. An initially halting state is recognized before the first transition. The same argument uses a fixed finite block for a convention allowing finitely many fresh bits per elementary transition, discarding unused bits. The finite cache changes neither counted work space nor the number of source transitions. Thus the supplied parameters a, b remain valid; only the static description length can increase.

Lemma 14.2 (A total resource-capped interpreter)**.** *There is one fixed randomized Turing machine $\mathcal{U}$ with polynomial worst-case running time and $O(\log(m+2))$ counted work bits on every binary input of length m, with the following behavior. Set*

$$\ell_m^{\mathrm{in}} = \lceil \log_2(m+2) \rceil, \qquad R(m, d_{\mathrm{code}}) = \left\lfloor \frac{\ell_m^{\mathrm{in}}}{100(d_{\mathrm{code}}+1)^3} \right\rfloor. \tag{14.1}$$

On an input with prefix

$$1^{d_{\mathrm{code}}}0\, z\, 1^n 0\, E_{d_{\mathrm{code}}}(x), \tag{14.2}$$

it first requires $1 \leq d_{\mathrm{code}} \leq \ell_m^{\mathrm{in}}$ and $R(m, d_{\mathrm{code}}) \geq 1$. It then simulates the encoded machine for at most m transitions, allowing each counted work tape only coordinates in $[-R, R]$. It rejects malformed data, a requested move outside these bounds, or failure to halt by the end of the transition budget. Otherwise it returns the encoded machine's decision. The suffix after the complete prefix has no parsing significance.

Proof. Count m before interpreting the prefix. Scan the unary code-length header with a bounded counter, rejecting if its length exceeds ℓ_m^{in} or the required delimiter is absent. Check the two eligibility

conditions before copying z or allocating simulation storage. Then verify that the following d_{code} code bits, the unary data-length header, and the required data bits are present. The parsed n is at most m, and $d_{\mathrm{code}} n \le \ell_m^{\mathrm{in}} m$, so even size calculations preceding a failed length check use only $O(\ell_m^{\mathrm{in}})$ bits. List counts, ID lengths, tape declarations, and movements are checked against the actual code and the stated bounds before any loops or allocations using them. There is no enumeration of an implicit domain of symbol or state names.

Store the encoded tapes sequentially on a fixed number of real work tapes of $\mathcal{U}$. At most d_{code} strips, each with $2R+1$ cells of d_{code} bits, require

$$d_{\mathrm{code}}^2(2R+1) = O(\ell_m^{\mathrm{in}}) \tag{14.3}$$

bits. Indeed, eligibility gives $d_{\mathrm{code}}^3 \le \ell_m^{\mathrm{in}}$ and $d_{\mathrm{code}}^2 R \le \ell_m^{\mathrm{in}}$. The copied program, current state, scanned and written symbol tuples, coin block, strip coordinates, step counter, and parsing scratch fit simultaneously in $O(\ell_m^{\mathrm{in}})$ bits. A single simulated input-head coordinate has magnitude $O(m)$ by the transition cap and therefore also uses $O(\ell_m^{\mathrm{in}})$ bits. Read the corresponding source symbol by rescanning the encoded data, supplying the appropriate markers or off-data blanks according to the encoded input-tape convention.

All routines are bounded scans and ordinary binary arithmetic. They iterate over actual code positions, input positions, a bounded tape/head list, strip positions, and bit positions, with a fixed nesting depth. Each range is polynomially bounded in $m+2$; binary arithmetic is implemented by bit scans, not unit-cost operations on growing words. Matching a listed transition, fetching input symbols, updating the strips, and obtaining its finite coin block therefore take polynomial time in $m+2$, with one fixed exponent determined by these routines. The same bounds hold when a check fails. There are at most m interpreted transitions. Checking the initial halting state and the state reached by the last permitted transition gives the asserted total behavior, including $m=0$. The resulting fixed $\mathcal{U}$ has the required polynomial worst-case time on every random tape. □

No bounded-error language promise is asserted for $\mathcal{U}$ on arbitrary inputs. Its acceptance probability may lie between $1/3$ and $2/3$. This is why we use probability approximation rather than a bare language-class containment.

Remark 14.3 (Optional head conventions). The construction also applies when the fixed coding and $\mathcal{U}$ are chosen to allow at most d_{code} heads per tape, with the usual common initial origin. These conventions are fixed before choosing the deterministic library in Section 14.2. There are then at most d_{code}^2 work-head coordinates and scanned symbols. Their storage is bounded by

$$O(d_{\mathrm{code}}^2 \log(2R+1) + d_{\mathrm{code}}^3) = O(\ell_m^{\mathrm{in}}).$$

For multiple input heads, or an optional entirely blank tape whose cells are not charged, use signed R-bit coordinates and reject on overflow. At most d_{code}^2 such coordinates still cost $O(d_{\mathrm{code}}^2 R)$ bits. These optional conventions do not change the interpreter's bounds. The instances in Section 14.3 never overflow those coordinates by Equation (14.8).

14.2 Fixing one deterministic library

Apply Theorem 3.1 to the fixed machine $\mathcal{U}$ and accuracy exponent 3, and compare its dyadic answer with $1/2$. The error on an input of length m is at most $(m+2)^{-3} \le 1/8$. Hence the resulting deterministic machine accepts whenever $\Pr[\mathcal{U} \text{ accepts}] \ge 2/3$ and rejects whenever that probability is at most $1/3$, since

$$\frac{1}{3} + \frac{1}{8} = \frac{11}{24} < \frac{1}{2} < \frac{13}{24} = \frac{2}{3} - \frac{1}{8}.$$

It halts on every input, including inputs between the two thresholds, as also stated in Corollary 3.2. Its logarithmic space bound counts all simultaneously live work in the approximation algorithm. Store the $O(\log(m+2))$-bit dyadic output in work space for the threshold test.

We fix this library once in the following convenient form. It is an ordinary finite-tape deterministic decision machine $\mathcal{A}_{\mathrm{lib}}$ with one input head confined to the endmarked input, initially at its left endmarker, such that every work head stays in

$$[-(\ell_m^{\mathrm{in}} + 1), \ell_m^{\mathrm{in}} + 1] \tag{14.4}$$

relative to its origin. To obtain this form, first consolidate any input heads by keeping their coordinates in logarithmic work space and scanning for each read. Even an off-data coordinate under an unbounded-input convention has logarithmic length because the original library has polynomial running time. A fixed change of initial control state handles its input-head start convention.

Next choose an integer w at least a coefficient bounding each counted work head's distance from its origin by $w\ell_m^{\mathrm{in}}$. Such one fixed w exists from the logarithmic bound, enlarged if necessary for the finitely many small lengths. Group consecutive work cells into blocks of width w, storing within-block offsets in finite control. The resulting finite alphabet and finite transition list realize Equation (14.4), also for two-sided tapes. All symbol widths will be charged in bits below; grouping only exposes a uniform address bound and gives no uncharged bit compression.

This choice of $\mathcal{A}_{\mathrm{lib}}$, including its one blocking width, precedes every compiler input. More formally, the remaining construction is a computable syntactic function of a finite library list and $(\mathcal{M}, a, b)$. Fixing one valid finite list yields one ordinary computable compiler on all those source inputs. The existence of the fixed list was proved by Theorem 3.1; no constant is selected as a noncomputable function of the source machine or its ordinary input.

The fixed finite data are the program grammar, the transition list of $\mathcal{A}_{\mathrm{lib}}$ with its blocked work alphabets and head conventions, and the finite callback templates described below. The construction constants, cutoffs, generator choices and finite instruction routines used by the separator are included in that list. Its blocking width is fixed at the same time. These are data specifying one compiler; subsequent source-dependent integers are computed from these data and the supplied $(\mathcal{M}, a, b)$.

14.3 Padding and exact preservation of acceptance probability

Given the source description and its supplied positive integers a, b, construct z as above and compute

$$\begin{aligned} P_{\mathrm{pad}} &= 1000(d_{\mathrm{code}} + 2)^4(a + b + 2), \\ \ell_n &= \lceil \log_2(n+2) \rceil, \qquad N_{\mathrm{pad}} = 2^{P_{\mathrm{pad}}\ell_n}. \end{aligned} \tag{14.5}$$

The generated machine will run $\mathcal{A}_{\mathrm{lib}}$ on a virtual binary input y_x of length N_{pad} whose prefix is Equation (14.2) and whose remaining bits are zero. Its prefix length is exactly

$$L_{\mathrm{pre}} = (d_{\mathrm{code}} + 1) + d_{\mathrm{code}} + (n+1) + d_{\mathrm{code}} n = (d_{\mathrm{code}} + 1)(n+2). \tag{14.6}$$

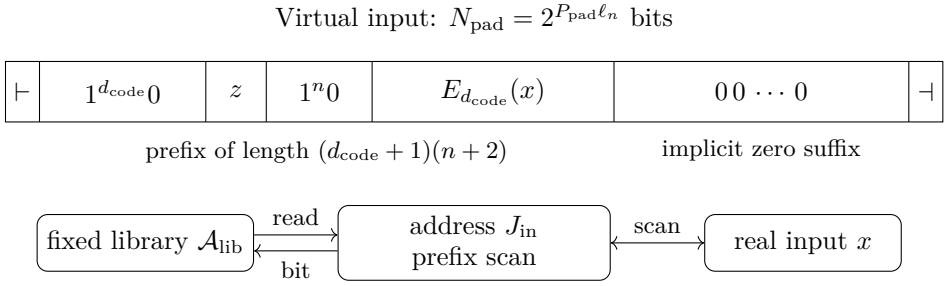


Figure 2: The fixed library reads the padded input at virtual input-head address J_{in} through a callback. The callback regenerates prefix symbols from the real input and returns zero for an address in the padding interval.

Since $\ell_n \geq 1$, $2^{\ell_n} \geq n+2$, and $2^{P_{\mathrm{pad}}-1} \geq d_{\mathrm{code}}+1$, this length is at most N_{pad}. The interpreter's length parameter is

$$\ell_{\mathrm{pad}} := \ell^{\mathrm{in}}_{N_{\mathrm{pad}}} = P_{\mathrm{pad}}\ell_n + 1. \tag{14.7}$$

All eligibility checks pass. In particular,

$$\frac{P_{\mathrm{pad}}}{100(d_{\mathrm{code}}+1)^3} = 10\frac{(d_{\mathrm{code}}+2)^4}{(d_{\mathrm{code}}+1)^3}(a+b+2) \geq 30(a+b+2),$$

so

$$R(N_{\mathrm{pad}}, d_{\mathrm{code}}) \geq 30(a+b+2)\ell_n - 1 \geq (a+b)\ell_n + 4. \tag{14.8}$$

Also $P_{\mathrm{pad}} > a$, whence $(n+2)^a \leq 2^{a\ell_n} < N_{\mathrm{pad}}$. Local motion from an initial work origin to a cell entails visiting the intervening counted cells. The source bound of $b\ell_n$ work bits therefore places every source work cell in the allocated strip; the slack in Equation (14.8) also covers a final one-cell move. This uses the source's original cell costs, each at least one bit, and does not charge it for the larger interpreter IDs.

Under the optional head conventions, local motion bounds an input or uncharged blank-tape coordinate by the source time, up to the fixed endmarker offset. Its signed encoding fits because $R \geq a\ell_n + 4$.

Consequently no resource cutoff changes a promised source run, and

$$\Pr[\mathcal{U}(y_x) \text{ accepts}] = \Pr[\mathcal{M}(x) \text{ accepts}]. \tag{14.9}$$

The decision of $\mathcal{A}_{\mathrm{lib}}(y_x)$ is therefore the decision required for the source language. Although N_{pad} is given exponentially, it is at most $(n+2)^{2P_{\mathrm{pad}}}$ for the fixed source. More importantly, it need never be written as an input string.

Figure 2 shows how the generated machine supplies the virtual input requested by $\mathcal{A}_{\mathrm{lib}}$. The library's work tapes remain live while the callback scans the real input; the resource bound below counts the library and callback storage simultaneously.

14.4 Compiling the virtual input operations

Inline the finite transition list of $\mathcal{A}_{\mathrm{lib}}$, giving it its own work tapes. The generated machine $\mathcal{D}_{\mathcal{M}}$ suspends its library control state in finite control during an input operation. Four auxiliary binary words suffice: the length count $n+1$, the virtual right-endmarker address $N_{\mathrm{pad}}+1$, the virtual input-head position J_{in}, and a prefix position J_{pre}. Each has a separate tape, with origin and termination marks and digits in low-end-first order.

Initialize the length count to one and increment it once per real input symbol. Its bit length is exactly

$$\mathrm{bitlength}(n+1) = \lfloor \log_2(n+1) \rfloor + 1 = \lceil \log_2(n+2) \rceil = \ell_n \qquad (n \geq 0). \tag{14.10}$$

For every bit place in that count, write P_{pad} zero bits on the bound tape, then append a one. This produces the low-end-first representation of $2^{P_{\mathrm{pad}}\ell_n} = N_{\mathrm{pad}}$; increment it to obtain $N_{\mathrm{pad}}+1$. The repetition of P_{pad} is hardwired finite control. Initially set $J_{\mathrm{in}} = 0$, the library's left-endmarker position.

At a virtual read, first test $J_{\mathrm{in}} = 0$ and $J_{\mathrm{in}} = N_{\mathrm{pad}}+1$ and return the corresponding endmarker. Otherwise initialize $J_{\mathrm{pre}} = 1$ and regenerate the prefix in its natural order:

1. emit the fixed string $1^{d_{\mathrm{code}}}0z$;
2. scan x, emitting one 1 per symbol, and then emit 0;
3. rewind and scan x again, emitting each fixed-width symbol code in $E_{d_{\mathrm{code}}}(x)$.

Here "emit" means compare the current J_{pre} with J_{in}, return the bit if they agree, and otherwise increment J_{pre} and continue. If the scan finishes without a match, return zero. The strings z, the integer d_{code}, symbol codes and emission phases are fixed in the generated control. The real input head is rewound by scanning to its endmarker, without storing its position. In particular, $J_{\mathrm{in}} = N_{\mathrm{pad}}+1$ is answered before regeneration, while a padding address such as $J_{\mathrm{in}} = N_{\mathrm{pad}}$ returns zero after the finite prefix scan. The largest prefix counter value is $L_{\mathrm{pre}}+1 \leq N_{\mathrm{pad}}+1$.

For explicit counter routines, an increment uses a finite carry flag. Compare two low-end-first words by scanning both, retaining only a finite comparison flag that is overwritten by each higher differing bit. After one terminator, treat its later bits as zero using a finite "ended" flag. Rewind each tape to its origin mark afterward. Zeroing and copying are likewise bounded bit scans. No growing arithmetic register beyond the named words is required. All words have at most $P_{\mathrm{pad}}\ell_n + 2$ bits; including delimiters and rewinds, their heads need visit only positions through $P_{\mathrm{pad}}\ell_n + 10$.

After the callback, perform the indicated library transition and update J_{in} in place for its input-head motion. The library's work heads and contents are untouched by the callback. Its state, the returned input symbol, and any step-local scanned tuple are fixed finite-control data. The callback does not invoke $\mathcal{A}_{\mathrm{lib}}$ recursively or maintain a stack of callbacks. The real input head stays between its real endmarkers.

Every listed routine terminates. For example, a complete regeneration uses at most L_{pre} emissions, each with an $O(P_{\mathrm{pad}}\ell_n + 1)$-bit comparison and increment, together with two real-input scans. Initialization makes n increments and writes $P_{\mathrm{pad}}\ell_n + 1$ bound bits. These observations give direct polynomial bounds for the auxiliary operations. The whole simulation halts because $\mathcal{A}_{\mathrm{lib}}$ is total. Its more uniform explicit time bound, including every such operation, is derived next from the complete configurations of this same machine.

All the above instructions can be generated as finite transition tables, unrolling fixed repetitions and strings if necessary. Formation of the source list, the integer P_{pad}, and this transition table is a finite effective process. Syntactically invalid compiler inputs may be assigned a fixed default output; correctness is required on the stipulated class. The generated simulation is total and obeys

the bounds below even when the supplied source promises fail; those promises are used to establish that its decision agrees with the source.

14.5 Explicit simultaneous bounds

Use the finite-alphabet description just constructed to compute n_{tapes}, its number of work tapes; $g_{\mathrm{alph}} \geq 2$, the maximum of two and its work alphabet sizes, including all blank, origin and termination symbols; and q_D, its number of states, including callback phases. Let $u_{\mathrm{code}} \geq 1$ dominate its explicit transition-table bit length, all alphabet sizes (including the input alphabet), the number of states, and the number of tapes. Padding this integer is effective. Return the positive integer bounds

$$\begin{aligned} \rho_{\mathrm{tape}} &= 100(P_{\mathrm{pad}} + 2), \qquad S_{\mathrm{code}} = 1 + (n_{\mathrm{tapes}} + 1)(2\rho_{\mathrm{tape}} + 1)(g_{\mathrm{alph}} + 2), \\ K_{\mathrm{space}} &= 4S_{\mathrm{code}}, \qquad c = 2S_{\mathrm{code}} + 1, \qquad H_{\mathrm{time}} = 2^{(u_{\mathrm{code}}+100)^2}(q_D + 1). \end{aligned} \tag{14.11}$$

These depend on the fixed source and the fixed library, and are computed from finite syntax. They have no dependence on n or x.

All live work bits. Every work head stays in $[-\rho_{\mathrm{tape}}\ell_n, \rho_{\mathrm{tape}}\ell_n]$. For a library tape this follows from Equations (14.4) and (14.7), since $\ell_{\mathrm{pad}}+1 = P_{\mathrm{pad}}\ell_n+2 \leq \rho_{\mathrm{tape}}\ell_n$. For an auxiliary tape it follows from $P_{\mathrm{pad}}\ell_n + 10 \leq \rho_{\mathrm{tape}}\ell_n$, using $\ell_n \geq 1$. Thus the library's live data and all callback data satisfy the same bound at the same time. Encoding a work alphabet of size g_i in $\max\{1, \lceil \log_2 g_i \rceil\} \leq g_{\mathrm{alph}}$ bits per symbol bounds their total data by

$$n_{\mathrm{tapes}}(2\rho_{\mathrm{tape}}\ell_n + 1)g_{\mathrm{alph}} \leq S_{\mathrm{code}}\ell_n. \tag{14.12}$$

Even twice this quantity satisfies the stronger logarithmic bound: with $L_n = \log_2(n + 2) \geq 1$ we have $\ell_n \leq 2L_n$ and

$$2S_{\mathrm{code}}\ell_n \leq 4S_{\mathrm{code}}L_n = K_{\mathrm{space}}L_n.$$

All finite-alphabet transitions. For a fixed real input x, a complete configuration is specified by the state, the real input-head position, all work-head positions, and all work symbols in the stated intervals. Everything outside these intervals remains at its initial value. There are at most

$$q_D(n + 2)(2\rho_{\mathrm{tape}}\ell_n + 1)^{n_{\mathrm{tapes}}} {g_{\mathrm{alph}}}^{n_{\mathrm{tapes}}(2\rho_{\mathrm{tape}}\ell_n+1)} \leq q_D(n + 2)2^{S_{\mathrm{code}}\ell_n} \leq q_D(n + 2)^c \tag{14.13}$$

such configurations. For the first inequality use

$$2\rho_{\mathrm{tape}}\ell_n + 1 \leq (2\rho_{\mathrm{tape}} + 1)\ell_n, \quad \log_2 g_{\mathrm{alph}} \leq g_{\mathrm{alph}}, \quad \log_2(2\rho_{\mathrm{tape}}\ell_n + 1) \leq (2\rho_{\mathrm{tape}} + 1)\ell_n;$$

the second uses $\ell_n \leq 2\log_2(n + 2)$ and $c = 2S_{\mathrm{code}} + 1$. A deterministic halting computation cannot repeat a complete configuration. Therefore Equation (14.13) bounds its total transitions, including initialization, counter arithmetic, every regenerated input bit, and every library transition. The virtual head position is already in the counted word J_{in}; it is not an additional uncounted configuration component.

Elementary bit operations. The preceding count refers to ordinary finite-alphabet transitions. To charge them in bits, encode each work symbol by a fixed-width block, mapping blank to the all-zero block. Read and write within the current block, return to its first bit, and move to the first bit of the target block. Within-block phases and scanned finite tuples are stored in finite control. No

per-block writable delimiter is needed, and these operations visit only blocks whose macro positions occur in the proved work intervals. Existing origin and termination symbols have already been included in g_{alph}. Thus the block implementation obeys the bit-space estimate above; the factor two also allows the usual blank/bit tape convention. Input-symbol access has fixed width and is covered by the same finite description parameter u_{code}.

There are at most $u_{\mathrm{code}} + 1$ scanned symbols of at most u_{code} bits each. An explicit sequential comparison against at most u_{code} transition rows, followed by the fixed-width reads, writes and moves, uses at most a fixed polynomial in $u_{\mathrm{code}} + 1$ bit actions per transition. For example, the deliberately loose bound $100(u_{\mathrm{code}} + 1)^8$ suffices for these scans and comparisons, with all temporary phase and symbol information in finite control. It is dominated, for $u_{\mathrm{code}} \geq 1$, by

$$100(u_{\mathrm{code}} + 1)^8 \leq 2^{(u_{\mathrm{code}}+100)^2}.$$

The binary microcode may have more states than q_D, but no new configuration count is needed: multiply the number of original transitions by this per-transition bit cost. Equations (14.11) and (14.13) give the simultaneous bound

$$\mathrm{time}_{\mathrm{bit}}(\mathcal{D}_{\mathcal{M}}, x) \leq H_{\mathrm{time}}(n + 2)^c. \tag{14.14}$$

No growing-word arithmetic is charged as a single elementary bit action.

Completion of the proof of Theorem 14.1. The compiler returns the described machine $\mathcal{D}_{\mathcal{M}}$ and the integers $K_{\mathrm{space}}, H_{\mathrm{time}}, c$ in Equation (14.11). Its finite syntactic construction terminates. On every promised source input, Equation (14.9) and the fixed separator's threshold guarantee give the correct decision. The totality of $\mathcal{A}_{\mathrm{lib}}$ and the finite callback loops prove halting. Equations (14.12) and (14.14) give at most $K_{\mathrm{space}} \log_2(n + 2)$ work bits and $H_{\mathrm{time}}(n + 2)^c$ bit operations for this same machine.

All claims include $n = 0$: then $\ell_n = 1$, the real-input scans are empty, the count $n + 1 = 1$ has one bit, and every displayed padding, strip, space, and time inequality remains valid. The library's grouping width already includes its finitely many small-length exceptions. All construction constants are fixed before the ordinary input is read.

The work and time internal to probability approximation—environment enumeration, numerical digits, recursive interfaces, recomputation, and precision management—belong to the one complete library machine proved in Theorem 3.1. Its tapes remain counted throughout virtualization. The compiler adds only the exact input operations analyzed here. Configuration counting gives polynomial time because this entire deterministic machine uses exact logarithmic space. □

References

[1] AmirMahdi Ahmadinejad, Jonathan Kelner, Jack Murtagh, John Peebles, Aaron Sidford, and Salil Vadhan. High-precision estimation of random walks in small space. In *61st IEEE Annual Symposium on Foundations of Computer Science (FOCS 2020)*, pages 1295–1306. IEEE, 2020. Full version: arXiv:1912.04524.

[2] Miklós Ajtai, János Komlós, and Endre Szemerédi. Deterministic simulation in LOGSPACE. In *Proceedings of the Nineteenth Annual ACM Symposium on Theory of Computing*, pages 132–140. Association for Computing Machinery, 1987.

[3] Romas Aleliunas, Richard M. Karp, Richard J. Lipton, László Lovász, and Charles Rackoff. Random walks, universal traversal sequences, and the complexity of maze problems. In *20th Annual Symposium on Foundations of Computer Science*, pages 218–223. IEEE, 1979.

[4] Roy Armoni, Amnon Ta-Shma, Avi Wigderson, and Shiyu Zhou. An $O(\log(n)^{4/3})$ space algorithm for (s,t) connectivity in undirected graphs. *Journal of the ACM*, 47(2):294–311, 2000.

[5] Algirdas Avizienis. On a flexible implementation of digital computer arithmetic. In C. M. Popplewell, editor, *Information Processing 1962*, pages 664–670. North-Holland, 1963.

[6] László Babai, Noam Nisan, and Márió Szegedy. Multiparty protocols, pseudorandom generators for logspace, and time-space trade-offs. *Journal of Computer and System Sciences*, 45(2):204–232, 1992.

[7] Allan Borodin, Stephen Cook, and Nicholas Pippenger. Parallel computation for well-endowed rings and space-bounded probabilistic machines. *Information and Control*, 58(1–3):113–136, 1983.

[8] Mark Braverman, Gil Cohen, and Sumegha Garg. Pseudorandom pseudo-distributions with near-optimal error for read-once branching programs. *SIAM Journal on Computing*, 49(5):STOC18–242–STOC18–299, 2020.

[9] Harry Buhrman, Richard Cleve, Michal Koucký, Bruno Loff, and Florian Speelman. Computing with a full memory: Catalytic space. In *Proceedings of the 46th Annual ACM Symposium on Theory of Computing*, pages 857–866. Association for Computing Machinery, 2014.

[10] Jin-Yi Cai, Venkatesan T. Chakaravarthy, and Dieter van Melkebeek. Time-space tradeoff in derandomizing probabilistic logspace. *Theory of Computing Systems*, 39(1):189–208, 2006.

[11] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.

[12] Ben Chen, Gil Cohen, Dean Doron, Yuval Khaskelberg, and Amnon Ta-Shma. Improved error reduction for weighted PRGs. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2026)*, volume 392 of *Leibniz International Proceedings in Informatics*, pages 39:1–39:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026.

[13] Kuan Cheng and William M. Hoza. Hitting sets give two-sided derandomization of small space. *Theory of Computing*, 18(21):1–32, 2022.

[14] Kuan Cheng and Ruiyang Wu. SC derandomization for regular ROBPs and models beyond BPL, 2026. Version 1, September 20, 2026.

[15] Gil Cohen, Dean Doron, Oren Renard, Ori Sberlo, and Amnon Ta-Shma. Error reduction for weighted PRGs against read once branching programs. In *36th Computational Complexity Conference (CCC 2021)*, volume 200 of *Leibniz International Proceedings in Informatics*, pages 22:1–22:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021.

[16] Gil Cohen, Dean Doron, Ori Sberlo, and Amnon Ta-Shma. Approximating iterated multiplication of stochastic matrices in small space. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 35–45. Association for Computing Machinery, 2023.

[17] Stephen A. Cook and Pierre McKenzie. Problems complete for deterministic logarithmic space. *Journal of Algorithms*, 8(3):385–394, 1987.

[18] John Gill. Computational complexity of probabilistic Turing machines. *SIAM Journal on Computing*, 6(4):675–695, 1977.

[19] William M. Hoza. Better pseudodistributions and derandomization for space-bounded computation. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2021)*, volume 207 of *Leibniz International Proceedings in Informatics*, pages 28:1–28:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021.

[20] Klaus-Jörn Lange, Pierre McKenzie, and Alain Tapp. Reversible space equals deterministic space. *Journal of Computer and System Sciences*, 60(2):354–367, 2000.

[21] Noam Nisan. Pseudorandom generators for space-bounded computation. *Combinatorica*, 12(4):449–461, 1992.

[22] Noam Nisan. RL $\subseteq$ SC. *Computational Complexity*, 4(1):1–11, 1994.

[23] Noam Nisan, Endre Szemerédi, and Avi Wigderson. Undirected connectivity in $O(\log^{1.5} n)$ space. In *Proceedings of the 33rd Annual Symposium on Foundations of Computer Science*, FOCS '92, pages 24–29. IEEE Computer Society, 1992.

[24] Noam Nisan and David Zuckerman. Randomness is linear in space. *Journal of Computer and System Sciences*, 52(1):43–52, 1996.

[25] Edward Pyne, Ran Raz, and Wei Zhan. Certified hardness vs. randomness for log-space. In *64th IEEE Annual Symposium on Foundations of Computer Science (FOCS 2023)*, pages 989–1007. IEEE, 2023. Full version: arXiv:2303.16413.

[26] Edward Pyne and Roei Tell. Using hardness vs randomness to design low-space algorithms. Electronic Colloquium on Computational Complexity, Report TR26-045, 2026.

[27] Omer Reingold. Undirected ST-connectivity in log-space. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '05, pages 376–385. ACM, 2005.

[28] Omer Reingold. Undirected connectivity in log-space. *Journal of the ACM*, 55(4):17:1–17:24, 2008.

[29] Omer Reingold, Luca Trevisan, and Salil Vadhan. Pseudorandom walks on regular digraphs and the RL vs. L problem. In *Proceedings of the 38th Annual ACM Symposium on Theory of Computing*, pages 457–466. Association for Computing Machinery, 2006.

[30] Marcel Riesz. Sur les maxima des formes bilinéaires et sur les fonctionnelles linéaires. *Acta Mathematica*, 49:465–497, 1927.

[31] Michael Saks and Shiyu Zhou. $\mathrm{BP_HSPACE}(S) \subseteq \mathrm{DSPACE}(S^{3/2})$. *Journal of Computer and System Sciences*, 58(2):376–403, 1999.

[32] Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the ACM*, 27(4):701–717, 1980.

[33] Yehuda Shalom. Bounded generation and Kazhdan's property (T). *Publications Mathématiques de l'IHÉS*, 90:145–168, 1999.

[34] G. O. Thorin. *Convexity theorems generalizing those of M. Riesz and Hadamard with some applications*. PhD thesis, Lund University, 1948. Russian translation by Yu. T. Medvedev, *Matematika* 1 (1957), no. 3, 43–78. Translation and bibliographic record.